

GUIA DE ESTUDIO EXAMEN EGEL PLUS - ISOFT

UNITEC^{MR}

Universidad Tecnológica de México

FACULTAD DE
INGENIERÍA,
ARQUITECTURA
Y DISEÑO GRÁFICO

Contenido

Guía de estudio Examen EGEL PLUS	6
Análisis de Sistemas de Software.....	7
1.1 Tipos de requerimientos	9
Técnicas y herramientas para la obtención, análisis, priorización y validación de requerimientos.....	12
1.2.1 obtención:	12
Observación	12
Tormenta de Ideas (en grupo)	14
Cuestionarios o Encuestas	14
1.2.2 Análisis de Requerimientos:	16
2.- Modelado del Proceso:	17
3. Modelo de dominio	18
4.CASOS DE USO	19
técnicas y herramientas de documentación de requerimientos.	21
1.3.1¿Qué es un documento SRS?	21
1.3.2 Herramientas para gestionar requerimientos	23
Diseño de Sistemas de Software	24
Diseño Arquitectónico de Software (Información del libro Ing. de software)	24
PATRONES DE DISEÑO	28
Diseño de módulos, componentes y de datos de software.	32
Estructura de diseño de Módulos y Componentes	33
DISEÑO DE DATOS	36
>> MODELO ENTIDAD- RELACION (E-R).....	36
>> La Normalización:.....	36
>>Diccionario de Datos	37
>>Esquema.....	38
PRINCIPIOS DE DISEÑO (SOLID Y COMPLEMENTIOS)	38
Patrones de Diseño Relevantes	40
→ Creacionales	40

→COMPORTAMENTALES	41
Arquitecturas y Estilos Relacionado.....	42
Desarrollo de Sistemas de Software.....	49
3.1.2 Conceptos Fundamentales	49
3.1.3 <i>Clasificación de Lenguajes</i>	50
3.1.4 Tipos de Tipado	52
3.1.5 Compilación, Interpretación y Ejecución:	53
3.1.6 Gestión de Memoria.....	54
3.1.7 Concurrencia y Paralelismo: Conceptos Universitarios:	55
3.1.8 Ecosistema y Librerías	55
3.1.9 Seguridad y Robustez.....	56
3.1.10 Criterios de Selección de Lenguaje	57
3.2 Paradigmas de Programación.	59
>> Paradigma Concurrente / Paralelo.....	63
3.3 Entornos de Desarrollo de Software.....	64
3.3.1 Conceptos centrales.....	64
3.3.2 Tipos de Entorno en el ciclo de desarrollo.....	65
3.3.3 Componentes de un Entorno de Desarrollo (IDE).	66
3.3.4 Ejemplos de Entornos por Lenguaje.	67
3.4 Gestión de Datos	68
3.4.1 Conceptos Fundamentales	68
3.4. 3 modelos de Datos.....	68
3.4.4 Etapas de la gestión de Datos.....	69
3.4.5 Modelado Conceptual y Lógico.....	70
3.4.6 Lenguajes de Gestión de Datos (SQL).....	71
3.4.7 Transacciones y Concurrencia.....	73
3.4.8 Integridad de Datos y Seguridad de los Datos	75
3.4.9 Gestión de Datos en Aplicaciones (Conceptos)	77
3.4.10 Técnicas de Integración de Datos.....	77

3.4.11 Tipos de Bases de Datos NoSQL	78
3.4.12 Administración y Mantenimiento.....	80
3.5 Plataformas de Desarrollo.	82
3.5.1 Concepto central.....	82
Gestión de proyectos de Software.....	84
4.1.2 Gestión del tiempo (Project Time Management)	87
4.1.3 Gestion de Costos (Project Cost Management)	91
4.1.4 Gestión de recursos humanos (Human Resource Management)	94
4.1.5 Gestión de riesgos en proyectos.	96
Calidad del Software	98
4.2.2 Conceptos Clave.....	98
4.2.3 Fases de la Gestión de Calidad	98
4.2.4 Normas y Modelos de Calidad más importante	99
4.2.5 Atributos de calidad del software (según ISO/IEC 25010)	100
4.2.6 Estrategias y niveles de pruebas de software.....	102
4.2.7 Ciclo de mejora continua de la calidad (PHVA)	102
4.2.8 Herramientas Clave para la Gestion de Calidad (QA)	103
4.2.9 Métricas Calidad del Software	104
4.2.10 Roles involucrados	105
4.2.11 Documentos de gestión de la calidad	105
4.2.12 Enfoque DevOps en la calidad.....	105
Metodologías de Desarrollo de Software (completo)	106
>>Modelo de Cascada.....	106
>> Modelo en V (Verificación y Validación)	108
>> Modelo Espiral	110
>>Modelo Incremental	112
>>Modelo de Prototipos	113
Metodologías ágiles.....	114
SCRUM:	114

Extreme Programming XP:.....	116
Kanban:	117
Lean Software Development (LSD):	118
Feature-Driven Development (FDD):	119
El Dynamic Systems Development Method	120
Crystal Method:.....	122
El DevOps (Desarrollo y Operaciones) Metodología Ágil.	123
Metodologías Ágiles	125
Cocoma – cálculo de costos de desarrollo de software.....	126
ISO – Estanders de calidad.....	127
ISO 12207 “Ciclo de vida del software”	127
ISO 15504 SPICE (Internacional)	127
ISO 25010 (SquaRe)	128
ISO 29148.....	129
ISO 25002.....	129
ISO 20000 Gestión de servicios.....	130
ISO 27001 Seguridad de la información	130
ISO 37301 Compliance.....	130
ISO/IEC 9001 – Gestión de calidad	131
Anexos.....	132
Consejos	132
Consejo Final para el Examen	133
Fórmulas Críticas para el Examen EGEL Plus.....	136
PREGUNTAS PARA FORTALECER LOS CONOCIMINETOS DEL MODULO 1.....	138
PREGUNTAS PARA FORTALECER LOS CONOCIMINETOS DEL MODULO 2.....	145
PREGUNTAS PARA FORTALECER LOS CONOCIMINETOS DEL MODULO 3.....	152
PREGUNTAS PARA FORTALECER LOS CONOCIMINETOS DEL MODULO 4.....	159
Elaborada	166
ALUMNOS.....	166

PROFESORES	166
DIRECTOR ACADEMICO.....	166
Recursos Digitales	167
Bibliografía	168
Área 1. Análisis de sistemas de software.....	168
Área 2. Diseño de sistemas de software	170
Área 3. Desarrollo de sistemas de software	172
Área 4. Gestión de proyectos de software.....	174



Guía de estudio Examen EGEL PLUS

Áreas	Subáreas	Núm. de reactivos
1. Análisis de Sistemas de Software	1.1. Tipos de requerimientos	12
	1.2. Técnicas y herramientas para la obtención, análisis, priorización y validación de requerimientos	9
	1.3. Técnicas y herramientas de documentación de requerimientos	10
2. Diseño de Sistemas de Software	2.1. Diseño arquitectónico de software	10
	2.2. Diseño de módulos, componentes y de datos de software	16
	2.3. Diseño de interfaces	7
3. Desarrollo de Sistemas de Software	3.1. Lenguajes de desarrollo de software	10
	3.2. Paradigmas de programación	10
	3.3. Entornos de desarrollo	10
	3.4. Gestión de datos	9
	3.5. Plataformas de desarrollo	10
4. Gestión de Proyectos de Software	4.1. Gestión de tiempos, costos, recursos humanos y de riesgo	8
	4.2. Calidad de software	10
	4.3. Metodologías de desarrollo	12
Total de reactivos		143

Análisis de Sistemas de Software

Análisis y especificación de requerimientos dentro del ciclo de vida de un sistema de información.

El ciclo de vida de un sistema de información es un enfoque por fases que consiste en una serie de pasos, con tareas específicas, para la creación, mantenimiento o mejora de un sistema. La norma ISO/IEC 12207:2008 [2], busca establecer un estándar de calidad internacional para el proceso de ciclo de vida de desarrollo y define seis etapas claramente diferenciadas:



Planificación (planteamiento del problema). La fase de planificación es la que permite conocer sobre **el alcance que tendrá el proyecto** (**Que tan grande va a hacer así como sus límites**), que puntos abarcará, los posibles riesgos que puede llegar a presentar y el orden en el cual se ejecutarán todas las tareas en el proceso de su creación.

- **Análisis.** Esta fase es la que estudia **las necesidades de información de los usuarios finales**, consiste en documentar **los requerimientos específicos** para el software a desarrollar, tal como se fijaron en la etapa de planeamiento, constituyéndose la base del diseño de un sistema de información.

Diseño. En esta fase, se selecciona **la arquitectura** más adecuada para llevar adelante el desarrollo y cumplir con los requerimientos fijados en la

documentación (**Tipo de representación gráfica de manera ilustrativa**).

Implementación. Consiste en el **desarrollo (Codificación del sistema)** propiamente dicho del producto, tomando como referencia los documentos de diseño.

Pruebas. En esta etapa se evalúa el desempeño del programa, para verificar la existencia de fallas o posibilidades de mejora hasta alcanzar los estándares de calidad definidos en la planificación (**requerimientos del cliente**).

Instalación / Despliegue. Luego de obtener la certificación de las pruebas donde se indica que éstas fueron exitosas, se procede con la **instalación** del software para su puesta en producción y **utilización** por parte de los usuarios finales.



1.1 Tipos de requerimientos

¿Qué es un requerimiento?

El análisis de requerimientos establece el proceso de definición de requerimientos como una **serie de tareas o actividades mediante las cuales se busca comprender las necesidades y el problema real de los clientes**, usuarios y otros interesados y utilizar este entendimiento para producir una especificación formal, completa, precisa y documentada de los requerimientos del sistema, siguiendo un determinado estándar.

Tipos de Requerimientos

Los requerimientos corresponden a una condición o capacidad que debe cumplir un sistema o componente del sistema para satisfacer una necesidad del usuario. Atendiendo al objetivo de los requerimientos se diferencian dos tipos: **funcionales y no funcionales**.

→Requerimiento Funcional (¿Qué es lo que va a hacer mi sistema?)

Los requerimientos funcionales describen la interacción entre el sistema y su ambiente, donde pueden estar involucrados usuarios, interacciones con otros sistemas, respuestas automáticas o procesos predefinidos. **Describen la funcionalidad o los servicios que el software debe ser capaz de realizar**. En algunos casos, los requerimientos funcionales también pueden establecer explícitamente lo que el sistema no debe hacer y esto es correcto siempre y cuando el resultado corresponda a una respuesta funcional esperada por el usuario u otro sistema.

Ejemplo: - - - - -

- Se deberá disponer de una funcionalidad que permita la creación y administración de usuarios que podrán acceder a la aplicación de acuerdo con la configuración dada.
- El campo fecha contable deberá aceptar únicamente fechas que correspondan con periodos contables que se encuentren en estado abierto en el sistema.
- La funcionalidad de programación de citas médicas deberá estar habilitada solo para usuarios que registren en estado activo.
- El reporte de indicadores mensuales deberá ser exportable en formatos .xlsx y .pdf. Al momento del cargue de un documento

al expediente, el sistema deberá asignar un número único de identificación en la plataforma.

- El sistema deberá permitir la consulta a partir del nombre o título del documento.

→No Funcionales (como atributos de calidad)

Los requerimientos no funcionales o también conocidos como atributos de calidad son aquellos que no se refieren directamente a las funciones específicas brindadas por el sistema, sino que definen propiedades y restricciones de éste **como tiempo de respuesta, requisitos de almacenamiento, seguridad, disponibilidad, estabilidad, rendimiento, etc.** Si estos requerimientos no son satisfechos el sistema no resulta útil.



Ejemplos

- El sistema debe ser capaz de operar de forma normal con varias sesiones concurrentes. El requerimiento descrito de esta manera no establece un valor numérico que permita medir la eficiencia del sistema; por lo cual, se requiere que éste sea escrito en términos como “El sistema debe ser capaz de operar de forma normal con hasta 100.000 usuarios con sesiones concurrentes”.
- Se necesita un tiempo de respuesta aceptable. El requerimiento no permite establecer cuál es el tiempo de respuesta aceptable para un usuario u otro y podría mejor plantearse como “Toda funcionalidad del sistema debe brindar un tiempo de respuesta al usuario menor a 5 segundos”.
- El promedio de duración de fallas no podrá ser mayor a 15 minutos.
- El sistema debe ser capaz de procesar N transacciones por segundo.
- La aplicación web debe poseer un diseño “Responsive” a fin de garantizar la adecuada visualización en múltiples computadores personales, dispositivos tableta y teléfonos inteligentes.

Propiedades

Cada requerimiento debe alcanzar las siguientes propiedades o criterios de calidad:

- **No ambigua.** Un requerimiento no es ambiguo si, y solo si, tiene una única interpretación. Todos los lectores deben llegar a la misma comprensión del requerimiento.
- **Consistente.** Los requerimientos deben ser consistentes en relación con todos los otros requerimientos; es decir, no deben contradecirse entre sí independientemente de su nivel de detalle. Además, el requerimiento debe ser formulado en una forma que permita la consistencia con sí mismo; es decir, el requerimiento no debe contradecirse.
- **Verificable.** Un requerimiento es verificable –puede ser probado- si existe algún proceso manual o automático que permita determinar si el software satisface dicho requerimiento.
- **Factible.** Debe ser posible implementar cada requerimiento considerando las limitaciones organizacionales, legales, técnicas o financieras.
- **Trazable.** Los requerimientos son trazables si se conoce el origen de cada uno y se facilita la referencia de cada requerimiento a otros requerimientos, a los componentes de diseño o a los componentes de la implementación. Esto puede hacerse por medio de los identificadores únicos de requerimientos.
- **Completa.** Una especificación de requerimientos es completa si: - - Incluye todos los requerimientos significativos del software (relacionados con la funcionalidad, ejecución, diseño, restricciones, atributos de calidad o interfaces externas). Existe una definición de respuesta a todas las posibles entradas, tanto válidas como inválidas, en cada uno de los escenarios.
- **Correcta.** Una especificación de requerimientos es correcta si y solo si, todo requerimiento que figura en ella refleja alguna necesidad real, esto asegura que el sistema implantado será el sistema deseado.
- **Modificable.** Durante el ciclo de vida del producto los requerimientos pueden ser modificados. Una especificación de requerimientos es modificable si cualquier cambio puede realizarse de manera fácil, completa y consistente.

Técnicas y herramientas para la obtención, análisis, priorización y validación de requerimientos.

1.2.1 obtención:

Técnicas de Recopilación de requerimientos de software

1. **Análisis de la documentación**
2. **Observación:**
 1. Pasiva
 2. Activa
3. **Entrevistas:**
 1. Formales
 2. Informales
 3. Con preguntas:
 - Abiertas (Información faltante)
 - Cerradas (confirmar y validar información)
4. **Encuestas o cuestionarios:**
 - Veloces para cuando son muchos usuarios
 - Puede complementarse con entrevistas de seguimiento con algunos usuarios.
 - Pueden tener preguntas abiertas y cerradas
5. **Mesas de trabajo (Workshops):**
 - Grupos de personas – sincrónica
 - Desarrollo Conjunto de Aplicaciones (JAD) – Reuniones cooperativas entre usuarios y analistas
 - Buenas prácticas para reuniones efectivas
6. **Tormenta de ideas**
7. **Historias del usuario (entrevista etnográfica)**



Observación

Observación

- Por medio de esta técnica el analista obtiene información de primera mano sobre la forma en que se efectúan las actividades.
- Este método permite observar la forma en que se llevan a cabo los procesos y, por otro, verificar que realmente se sigan todos los pasos especificados.
- Como sabemos, en muchos casos los procesos son una cosa en papel y otra muy diferente en la práctica.
- Los observadores experimentados saben qué buscar y cómo evaluar la relevancia de lo que observan.
- De ser posible, se observará el sistema en acción. No se plantearán preguntas. Tan sólo se observará y se **tomarán notas o dibujos**.
- Conviene asegurarse de que las personas observadas saben que no se les está evaluando. En caso contrario, harán su trabajo de manera más eficaz que lo normal.

Entrevistas

Es una conversación que se sostiene entre dos o más personas a fin de conseguir información importante sobre los requerimientos del desarrollo solicitado.

Se utilizan para obtener **información cualitativa** como opiniones, o descripciones subjetivas de actividades.

La entrevista se puede definir como un “**intento sistemático de recoger información de otra persona**” a través de una comunicación interpersonal que se lleva a cabo por medio de una **conversación estructurada**.

Es muy importante la forma en que se plantea la conversación y la relación que se establece en la entrevista.

Estos son algunos de los aspectos más importantes a tener en cuenta al realizar entrevistas:

- **Preparación.** Analizar **documentos disponibles**, de tal forma que la entrevista se enfoque en aquellos aspectos que están solamente en la mente del entrevistado
- **Entrevistar al personal adecuado.** Enfoque **top-down**, comenzando a entrevistar a directivos para que brinden un panorama general de hacia donde deberían ir las cosas, y terminando por hablar con los empleados que aportan detalles importantes de la operación.
- **Duración.** Una entrevista debería durar a lo sumo un par de horas.
- **Formato.** Se recomienda utilizar **preguntas abiertas**, donde los entrevistados puedan elaborar y dar detalles, más allá de simplemente responder “sí” o “no”.



Tipos de entrevistas

ETNOGRÁFICA

Se usa en el contexto del trabajo para entender los **requerimientos sociales y organizacionales**.

Un analista se sumerge por sí solo en el entorno laboral donde el sistema se utilizará. El trabajo diario se **observa** y se hacen notas de las **tareas reales** en las que los participantes están involucrados. La etnografía es especialmente efectiva para descubrir dos tipos de requerimientos:

- 1 Los requerimientos que se derivan **de la forma en la que la gente trabaja realmente** más que de la forma en la que las definiciones de los **procesos** establecen que debería trabajar.
- 2 Los requerimientos que se derivan de **la cooperación y conocimiento de las actividades de la gente**.

INTERPRETATIVA

Estructura de formulación y secuencia de las preguntas.

La introducción en la conversación de **hechos o situaciones más específicas** debe ser cuidadosa ya que “requiere que el investigador **se abstenga de formular preguntas directas o que pueden inducir dirección u orientación**”.

Se puede seguir una “**guía de la entrevista**” para asegurarse que los temas claves sean explorados.

EXPLORATORIA

EN PROFUNDIDAD

Tormenta de ideas (Brainstorming)

- Es una sesión de trabajo estructurada orientada para obtener la mayor cantidad de ideas posibles.
- Es recomendable limitarlas en el tiempo, utilizar ayudas visuales y designar un facilitador.
- Las reglas son importantes, por ejemplo los criterios para evaluar ideas y asignarles un puntaje, no permitir las críticas a las ideas y limitar el tiempo de discusión.
- En una primera fase, se deben identificar la mayor cantidad de ideas, para luego evaluarlas. Todas las ideas deben ser consideradas y deben evitar que una idea se descarte o critique antes de tener tiempo de desarrollarla.



Cuestionarios o Encuestas

- El uso de cuestionarios permite reunir información proveniente de un **grupo grande de personas**.
- El empleo de **formatos estandarizados** para las preguntas puede proporcionar datos más confiables que otras técnicas; por otra parte, su amplia distribución asegura el **anonimato** de los encuestados, situación que puede conducir a respuestas más honestas.
- Como la información se obtiene por observación indirecta, a través de los encuestados, es mejor comprobar la veracidad de los datos recabados.
- El inconveniente es que la respuesta puede ser limitada, ya que es posible que no tenga mucha importancia para los encuestados llenar el cuestionario. Es recomendable conseguir apoyo de la alta dirección para solicitar a las personas de la organización que contesten el cuestionario.
- La información se recoge de forma **estructurada** para poder compararlos y analizarlos. Es una entrevista estructurada, no se pueden cambiar las preguntas, ni repreguntar, ampliar o agregar nuevas.
- Al igual que con las entrevistas, se debe seleccionar a los encuestados. El analista debe asegurar que el conocimiento y experiencia de éstos califiquen para dar respuestas a las preguntas.
- Las preguntas son, en su mayoría, **cerradas**. Es decir, las opciones de respuesta se limitan a las previamente especificadas por el investigador. De ahí la conveniencia de complementar los datos de encuesta con la información reunida mediante otras estrategias de investigación, por ejemplo, el **estudio de casos**.

Historias de Usuarios

Las historias de usuario, son una aproximación simple al levantamiento de requerimientos de software, en la cual la conversación pasa a ser más importante que la formalización de requerimientos escritos.

Es recomendable que **sean escritas por el mismo cliente o interesado** (con apoyo del facilitador si es necesario), con énfasis en las **funcionalidades** que el sistema deberá realizar.

Al redactar una historia de usuario debe tenerse en cuenta describir el Rol, la funcionalidad y el resultado esperado de la aplicación en una frase corta.

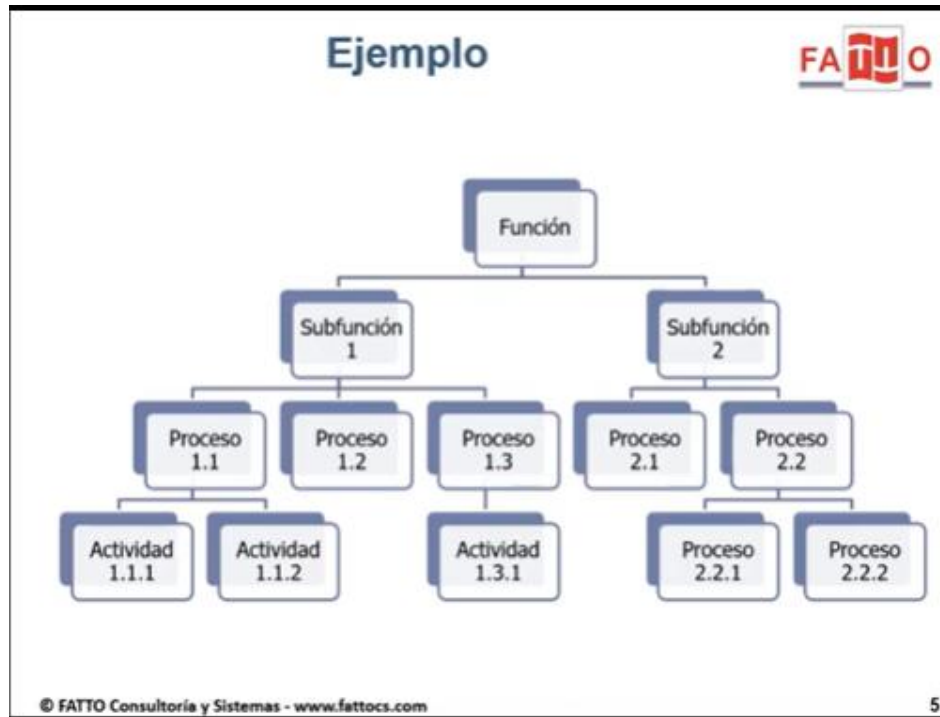
Las historias de usuario son una de las técnicas más difundidas para levantar requerimientos de software en metodologías ágiles.



1.2.2 Análisis de Requerimientos:

1.- Descomposición funcional

- Su objetivo es simplificar la complejidad, mejorar la claridad y facilitar el diseño, el análisis, la optimización y el desarrollo, especialmente en áreas como la ingeniería de sistemas y el desarrollo de software.
- Por lo general, la descomposición funcional se realiza para identificar y entender los componentes o partes que constituyen un todo (o función global).
- En este proceso, es vital identificar las interacciones entre componentes.
- Aplicado a la Ingeniería de requisitos, consiste en tomar los requerimientos de software, dividirlos en partes y analizarlos individualmente. De ser necesario, se pueden descomponer en más partes hasta lograr un nivel adecuado de detalle.
- En cierto sentido, el proceso es similar al de la elaboración de una estructura de desglose de trabajo de un proyecto.
- En Ingeniería de sistemas, la descomposición funcional consiste en definir un sistema en términos funcionales, para luego definir funciones de más bajo nivel y establecer las relaciones con estas funciones de alto nivel.
- La intención es dividir un sistema de tal forma que cada componente se pueda describir sin necesidad de referir a otro componente.
- De esta forma, cada parte del sistema tendrá funciones independientes, que pueden reusarse y reemplazarse.



2.- Modelado del Proceso:

- El modelado del proceso en software es una técnica que utiliza diagramas y representaciones gráficas para describir el ciclo de vida del desarrollo de software, incluyendo actividades, roles, productos y riesgos. Se emplea para estandarizar la forma en que se construye el software, mejorar la eficiencia y la calidad, y asegurar que todos los involucrados tengan una visión clara y compartida del proceso.
Comprende la elaboración de diagramas de flujo de procesos (Flujogramas) a partir de los requerimientos del software.
- Existen diversas herramientas de modelado de procesos, cada una de las cuales posee sus propios símbolos y reglas.
- Es muy útil para entender el trabajo realizado en múltiples pasos, tareas, roles y departamentos intervinientes.
- Los procesos son iniciados por eventos y pueden abarcar actividades automatizadas, manuales o combinación entre ambas.
- Su naturaleza visual ayuda a la comprensión y comunicación a terceros.
- Cuando los procesos son complejos, deben desglosarse en componentes (subprocesos).

- La relación entre los diagramas de flujo y la gerencia de proyectos es fundamental para el éxito.

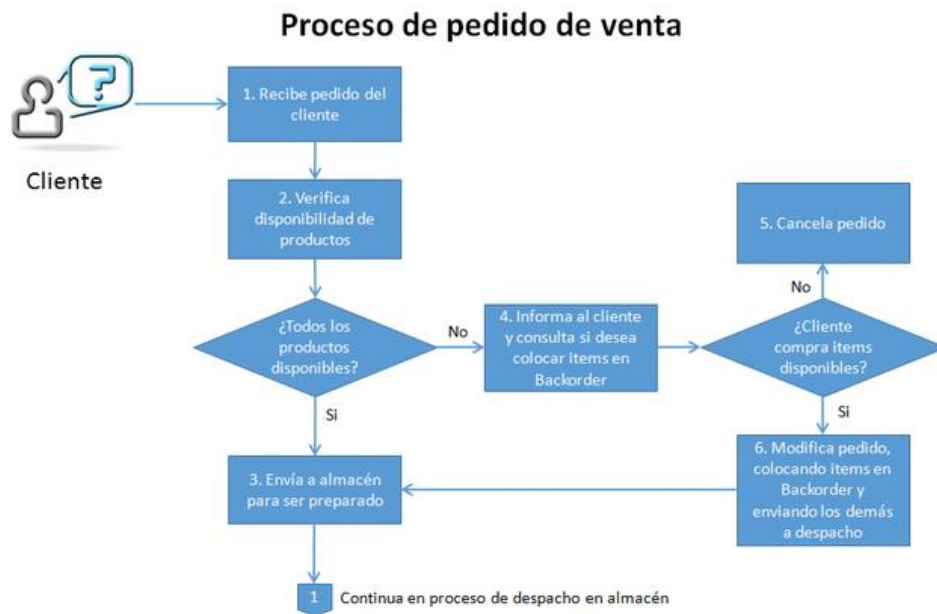


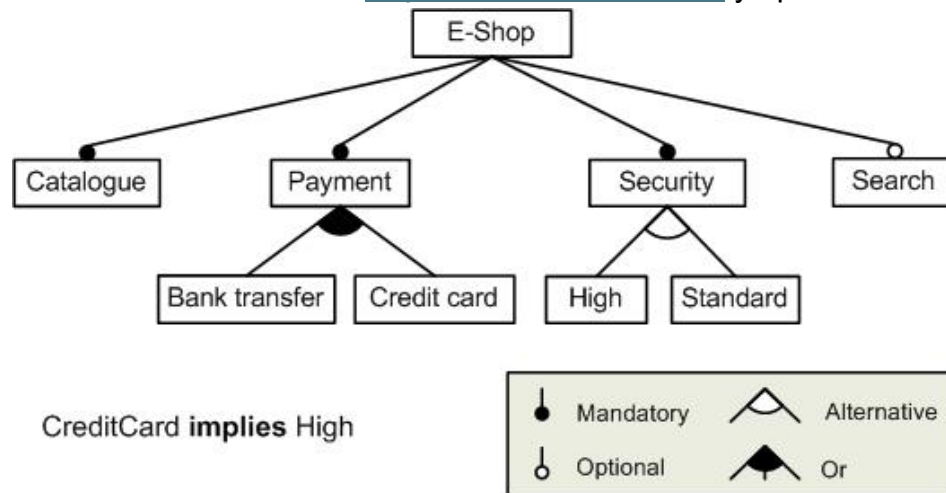
Imagen elaborada por www.pmoinformatica.com

3. Modelo de dominio

En Ingeniería de software, en análisis de dominio consiste en analizar sistemas o software relacionados en un dominio, con la finalidad de encontrar sus partes comunes y partes que los diferencian.

- Produce un modelo de contexto de negocio para todo el sistema.
- Un modelo de dominio comprende diagramas conceptuales que incluyen tanto el comportamiento de un sistema como sus datos.
- Un tipo de modelo de dominio son los diagramas de funcionalidades (Features Diagrams), que es una representación “compacta” del sistema o aplicación en términos de sus características.
- El análisis de dominio produce modelos orientados a objetos o modelos relacionales de datos, que pueden ser usados por los desarrolladores de

software como base de [arquitecturas de software](#) y aplicaciones.



4.CASOS DE USO

- Representación grafica en donde nos explica como es que el usuario interactúa con el sistema.
- Los casos de uso son (Óvalos) los requerimientos funcionales de nuestro sistema de softwares
- Actor (Stakeholder): No se refiere solo a un individuo (Usuario) sino que también entes externos al sistema.



Diagramas de Casos de Uso

- Los Diagramas de Casos de Uso especifican requisitos funcionales
 - Cada requisito funcional es representado por un caso de uso
 - Cada caso de uso es documentado mediante una Descripción Textual



técnicas y herramientas de documentación de requerimientos.

1.3.1 ¿Qué es un documento SRS?

Un documento de Especificación de requisitos de software (SRS) es una descripción detallada y estructurada de los requisitos funcionales y no funcionales de un sistema de software. Sirve como guía definitiva para desarrolladores, diseñadores y partes interesadas, y describe con precisión lo que el software debe hacer para satisfacer las necesidades de la empresa y de los usuarios. Al cubrir los aspectos técnicos y operativos, **una SRS garantiza que todas las partes involucradas compartan una comprensión unificada de los objetivos y el alcance del proyecto.**

La especificación de requisitos de software (SRS) es una descripción completa del comportamiento del sistema que se va a desarrollar. Incluye un conjunto de casos de uso que describe todas las interacciones que tendrán los usuarios con el software. Los casos de uso también son conocidos como requisitos funcionales. Además de los casos de uso, la ERS también contiene requisitos no funcionales (complementarios). Los requisitos no funcionales son requisitos que imponen restricciones en el diseño o la implementación, como, por ejemplo, restricciones en el diseño o estándares de calidad.



FORMATO ESPECIFICACIÓN REQUERIMIENTOS

INTRODUCCIÓN

Propósito del documento
Alcance
Personal Involucrado

Glosario

Negocio
Sistema
Técnicos

RFnn Requisito funcional
RNFnn Requisito no funciona
RQnn Requisito de calidad
RInn Requisito de interfaçe
n número consecutivo del requisito especificado

DESCRIPCIÓN DEL SISTEMA

Características de los implicados en el desarrollo

Descripción de los requerimientos funcionales y no funcionales del sistema propuesto

Código	Nombre	Fecha	Grado Necesidad
RF001	Registrar clientes	lunes, 28 de noviembre de 2016	Alto
Descripción	El sistema debe permitir a los clientes registrar su información personal		
Entradas	Fuente	Salida	Destino
			Regla de negocio

1.3.2 Herramientas para gestionar requerimientos

Estas herramientas permiten **controlar, rastrear, validar y mantener** los requerimientos durante todo el ciclo de vida del proyecto:

Puntos clave por herramienta

- **Jira Software:** gestiona requerimientos como tareas e historias de usuario en entornos ágiles (Scrum/Kanban). Ideal para colaboración y seguimiento visual.
- **JAMA Software:** orientada a industrias críticas, permite trazabilidad bidireccional, validación y cumplimiento de normas ISO/FDA.
- **Modern Requirements:** extensión para Azure DevOps que permite documentar, revisar y rastrear requerimientos dentro del ecosistema Microsoft.
- **Redmine:** plataforma gratuita y flexible que puede adaptarse para gestionar requerimientos mediante incidencias y plugins.
- **DRES:** herramienta académica para documentar y versionar requerimientos en proyectos universitarios.



Análisis de las opciones:

Opción	Descripción	Uso principal
A) Jira Software 	Herramienta de Atlassian usada en proyectos ágiles (Scrum, Kanban). Permite gestionar tareas, historias de usuario, versiones, incidencias y requerimientos .	Seguimiento de desarrollo de software, control de versiones y flujos ágiles.
B) JAMA Software	Orientada a la gestión de requisitos y trazabilidad, sobre todo en proyectos de ingeniería de sistemas o hardware/software complejos .	Gestión de requisitos y validaciones, más formal que ágil.
C) DRES	No corresponde a una herramienta ampliamente reconocida para gestión ágil ni control de versiones (podría referirse a sistemas académicos o internos, pero no es estándar).	No aplicable en este contexto.

Diseño de Sistemas de Software

Diseño Arquitectónico de Software (Información del libro Ing. de software)

- El diseño arquitectónico se interesa por entender cómo debe **organizarse** un sistema y **cómo tiene que diseñarse la estructura global de ese sistema**.

En palabras sencillas, el **Diseño Arquitectónico** de software es como hacer los **planos de un edificio** 🏠 antes de empezar a construir.

Se trata de tomar las **decisiones más importantes** y de **alto nivel** sobre la estructura del sistema.

¿Qué se define en esta etapa?

1. **Componentes Principales (Las "Salas"):** Decides cuáles son los grandes módulos o subsistemas de software (por ejemplo, el módulo de usuarios, el de inventario, el de pagos).
 2. **Relaciones (Las "Tuberías" y "Camino"):** Determinas cómo se comunican esos grandes componentes entre sí (si se hablan directamente, si usan una base de datos intermedia, si se envían mensajes, etc.).
 3. **Restricciones de Calidad (La "Robustez" del Edificio):** Te aseguras de que la estructura cumpla con los requisitos no funcionales, como que el sistema sea:
 - **Escalable** (que pueda crecer sin romperse).
 - **Seguro** (que proteja la información).
 - **Fácil de mantener** (que sea sencillo arreglar o actualizar una parte sin afectar al resto).
- El diseño arquitectónico es la primera etapa en el proceso de diseño del software. Es el enlace crucial entre el diseño y la ingeniería de requerimientos, ya que identifica los principales componentes estructurales en un sistema y la relación entre ellos.
 - **Las arquitecturas de software se diseñan en dos niveles de abstracción**, que en este texto se llaman arquitectura en pequeño y arquitectura en grande:
 1. La arquitectura en **pequeño** se interesa por la arquitectura de programas individuales. En este nivel, uno se preocupa por la forma en que el programa individual se separa en

componentes. Este capítulo se centra en arquitecturas de programa.

2. La arquitectura en **grande** se interesa por la arquitectura de **sistemas empresariales** complejos que incluyen otros sistemas, programas y componentes de programa. Tales sistemas empresariales se distribuyen a través de diferentes computadoras, que diferentes compañías administran y poseen. En los capítulos 18 y 19 se cubren las arquitecturas grandes; en ellos se estudiarán las arquitecturas de los sistemas distribuidos.

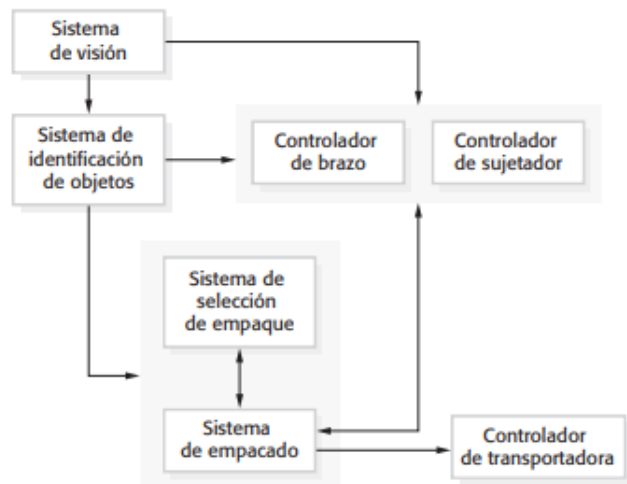


Figura 6.1 Arquitectura de un sistema de control para un robot empacador

Bass y sus colaboradores (2003) analizan tres ventajas de diseñar y documentar de manera explícita la arquitectura de software:

1. *Comunicación con los participantes* La arquitectura es una presentación de alto nivel del sistema, que puede usarse como un enfoque para la discusión de un amplio número de participantes.
2. *Análisis del sistema* En una etapa temprana en el desarrollo del sistema, aclarar la arquitectura del sistema requiere cierto análisis. Las decisiones de diseño arquitectónico tienen un efecto profundo sobre si el sistema puede o no cubrir requerimientos críticos como rendimiento, fiabilidad y mantenibilidad.
3. *Reutilización a gran escala* Un modelo de una arquitectura de sistema es una descripción corta y manejable de cómo se organiza un sistema y cómo interoperan sus componentes. Por lo general, la arquitectura del sistema es la misma para sistemas con requerimientos similares y, por lo tanto, puede soportar reutilización de software a gran escala. Como se explica en el capítulo 16, es posible desarrollar arquitecturas de línea de productos donde la misma arquitectura se reutilice mediante una amplia gama de sistemas relacionados.

-
- Las arquitecturas de sistemas se **modelan con frecuencia usando diagramas de bloques simples**, como en la figura 6.1. Cada recuadro en el diagrama representa un **componente**(Hace una tarea específica y bien definida). Los recuadros dentro de recuadros indican que el componente se dividió en **subcomponentes**(piezas más pequeñas que trabajan juntas dentro de un mismo bloque funcional (el Componente)). Las flechas significan que los datos y/o señales de control pasan de un componente a otro en la dirección de las flechas. Hay varios ejemplos de este tipo de modelo arquitectónico en el catálogo de arquitectura de software de Booch (Booch, 2009).

4.2. Patrón Arquitectónico

Buschmann et al. (1996) define *patrón* como una regla que consta de tres partes, la cual expresa una relación entre un contexto, un problema y una solución. En líneas generales, un patrón sigue el siguiente esquema:

- ✓ *Contexto*. Es una situación de diseño en la que aparece un problema de diseño
- ✓ *Problema*. Es un conjunto de fuerzas que aparecen repetidamente en el contexto
- ✓ *Solución*. Es una configuración que equilibra estas fuerzas. Ésta abarca:
 - Estructura con componentes y relaciones
 - Comportamiento a tiempo de ejecución: aspectos dinámicos de la solución, como la colaboración entre componentes, la comunicación entre ellos, etc.

Partiendo de esta definición, propone los *patrones arquitectónicos* como descripción de un problema particular y recurrente de diseño, que aparece en contextos de diseño específico, y presenta un esquema genérico demostrado con éxito para su solución. El esquema de solución se especifica mediante la descripción de los componentes que la constituyen, sus responsabilidades y desarrollos, así como también la forma como estos colaboran entre sí.

21

Patrón Arquitectónico	Descripción	Atributos asociados	Atributos en conflicto
<i>Layers</i>	Consiste en estructurar aplicaciones que pueden ser descompuestas en grupos de subtarear, las cuales se clasifican de acuerdo a un nivel particular de abstracción.	Reusabilidad Portabilidad Facilidad de Prueba	Desempeño Mantenibilidad
<i>Pipes and Filters</i>	Provee una estructura para los sistemas que procesan un flujo de datos. Cada paso de procesamiento está encapsulado en un componente filtro (<i>filter</i>). El dato pasa a través de conexiones (<i>pipes</i>), entre filtros adyacentes.	Reusabilidad Mantenibilidad	Desempeño
<i>Blackboard</i>	Aplica para problemas cuya solución utiliza estrategias no determinísticas. Varios subsistemas ensamblan su conocimiento para construir una posible solución parcial ó aproximada.	Modificabilidad Mantenibilidad Reusabilidad Integridad	Desempeño Facilidad de Prueba
<i>Broker</i>	Puede ser usado para estructurar sistemas de software distribuido con componentes desacoplados que interactúan por invocaciones a servicios remotos. Un componente <i>broker</i> es responsable de coordinar la comunicación, como el reenvío de solicitudes, así como también la transmisión de resultados y excepciones.	Modificabilidad Portabilidad Reusabilidad Escalabilidad Interoperabilidad	Desempeño
<i>Model-View-Controller</i>	Divide una aplicación interactiva en tres componentes. El modelo (<i>model</i>) contiene la información central y los datos. Las vistas (<i>view</i>) despliegan información al usuario. Los controladores (<i>controllers</i>) capturan la entrada del usuario. Las vistas y los controladores constituyen la interfaz del usuario.	Funcionalidad Mantenibilidad	Desempeño Portabilidad

Patrón Arquitectónico	Descripción	Atributos asociados	Atributos en conflicto
<i>Presentation-Abstraction-Control</i>	Define una estructura para sistemas de software interactivos de agentes de cooperación organizados de forma jerárquica. Cada agente es responsable de un aspecto específico de la funcionalidad de la aplicación y consiste de tres componentes: presentación, abstracción y control.	Modificabilidad Escalabilidad Integrabilidad	Desempeño Mantenibilidad
<i>Microkernel</i>	Aplica para sistemas de software que deben estar en capacidad de adaptar los requerimientos de cambio del sistema. Separa un núcleo funcional mínimo del resto de la funcionalidad y de partes específicas pertenecientes al cliente.	Portabilidad Escalabilidad Confiabilidad Disponibilidad	Desempeño
<i>Reflection</i>	Provee un mecanismo para sistemas cuya estructura y comportamiento cambia dinámicamente. Soporta la modificación de aspectos fundamentales como estructuras tipo y mecanismos de llamadas a funciones.	Modificabilidad	Desempeño

Tabla 10 (b). Patrones arquitectónicos y atributos de calidad (Continuación)

PATRONES DE DISEÑO



4.3. Patrón de Diseño

Un *patrón de diseño* provee un esquema para refinar los subsistemas o componentes de un sistema de software, o las relaciones entre ellos. Describe la estructura comúnmente recurrente de los componentes en comunicación, que resuelve un problema general de diseño en un contexto particular (Buschman et al., 1996).

Son menores en escala que los patrones arquitectónicos, y tienden a ser independientes de los lenguajes y paradigmas de programación. Su aplicación no tiene efectos en la estructura fundamental del sistema, pero sí sobre la de un subsistema (Buschman et al., 1996), debido a que especifica a un mayor nivel de detalle, sin llegar a la implementación, el comportamiento de los componentes del subsistema. La tabla 12 presenta algunos patrones de diseño, junto a los atributos de calidad que propician y los atributos que entran en conflicto con la aplicación del patrón, según Buschmann et al. (1996).

Patrón de Diseño	Descripción	Atributos asociados	Atributos en conflicto
<i>Whole-Part</i>	Ayuda a constituir una colección de objetos que juntos conforman una unidad semántica.	Reusabilidad Modificabilidad	Desempeño
<i>Master-Slave</i>	Un componente maestro (<i>master</i>) distribuye el trabajo a los componentes esclavos (<i>slaves</i>). El componente maestro calcula un resultado final a partir de los resultados arrojados por los componentes esclavos.	Escalabilidad Desempeño	Portabilidad
<i>Proxy</i>	Los clientes asociados a un componente se comunican con un representante de éste, en lugar del componente en sí mismo.	Desempeño Reusabilidad	Desempeño
<i>Command Procesor</i>	Separa las solicitudes de un servicio de su ejecución. Maneja las solicitudes como objetos separados, programa sus ejecuciones y provee servicios adicionales como el almacenamiento de los objetos solicitados, para permitir que el usuario pueda deshacer alguna solicitud.	Funcionalidad Modificabilidad Facilidad de Prueba	Desempeño
<i>View Handler</i>	Ayuda a manejar todas las vistas que provee un sistema de software. Permite a los clientes abrir, manipular y eliminar vistas. También coordina dependencias entre vistas y organiza su actualización.	Escalabilidad Modificabilidad	Desempeño
<i>Forwarder- Receiver</i>	Provee una comunicación transparente entre procesos de un sistema de software con un modelo de interacción punto a punto (<i>peer to peer</i>).	Mantenibilidad Modificabilidad Desempeño	Configurabilidad
<i>Client- Dispatcher- Server</i>	Introduce una capa intermedia entre clientes y servidores, es el componente despachador (<i>dispatcher</i>). Provee una ubicación transparente por medio de un nombre de servicio, y esconde los detalles del establecimiento de una conexión de comunicación entre clientes y servidores.	Configurabilidad Portabilidad Escalabilidad Disponibilidad	Desempeño Modificabilidad
<i>Publisher- Subscriber</i>	Ayuda a mantener sincronizados los componentes en cooperación. Para ello, habilita una vía de propagación de cambios: un editor (<i>publisher</i>) notifica a los suscriptores (<i>suscribers</i>) sobre los cambios en su estado.	Escalabilidad	Desempeño

Tabla 12. Patrones de diseño y atributos de calidad.

La figura 1.1 presenta la relación de abstracción existente entre los conceptos de estilo arquitectónico, patrón arquitectónico y patrón de diseño. En ella se representa el planteamiento de Buschmann et al. (1996), que propone el desarrollo de arquitecturas de software como un sistema de patrones, y distintos niveles de abstracción.

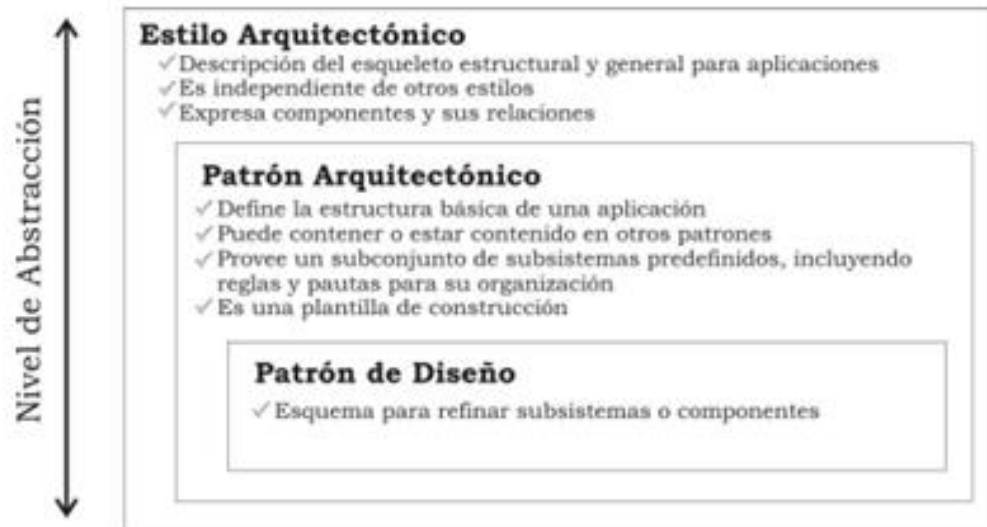


Figura 1.1. Relación de abstracción entre estilos y patrones

Los estilos y patrones ayudan al arquitecto a definir la composición y el comportamiento del sistema de software, y una combinación adecuada de ellos permite alcanzar los requerimientos de calidad.

6.2 Vistas arquitectónicas

En la introducción a este capítulo se explicó que los modelos arquitectónicos de un sistema de software sirven para enfocar la discusión sobre los requerimientos o el diseño del software. De manera alternativa, pueden emplearse para documentar un diseño, de modo que se usen como base en el diseño y la implementación más detallados, así como en la evolución futura del sistema. En esta sección se estudian dos temas que son relevantes para ambos:

1. ¿Qué vistas o perspectivas son útiles al diseñar y documentar una arquitectura del sistema?
2. ¿Qué notaciones deben usarse para describir modelos arquitectónicos?

Es imposible representar toda la información relevante sobre la arquitectura de un sistema en un solo modelo arquitectónico, ya que cada uno presenta únicamente una vista o perspectiva del sistema. Ésta puede mostrar cómo un sistema se descompone en módulos, cómo interactúan los procesos de tiempo de operación o las diferentes formas en que los componentes del sistema se distribuyen a través de una red. Todo ello es útil en diferentes momentos de manera que, para el diseño y la documentación, por lo general se necesita presentar múltiples vistas de la arquitectura de software.

Existen diferentes opiniones relativas a qué vistas se requieren. Krutchen (1995), en su bien conocido modelo de vista 4+1 de la arquitectura de software, sugiere que deben existir cuatro vistas arquitectónicas fundamentales, que se relacionan usando casos de uso o escenarios. Las vistas que él sugiere son:

1. Una vista lógica, que indique las abstracciones clave en el sistema como objetos o clases de objeto. En este tipo de vista se tienen que relacionar los requerimientos del sistema con entidades.



2. Una vista de proceso, que muestre cómo, en el tiempo de operación, el sistema está compuesto de procesos en interacción. Esta vista es útil para hacer juicios acerca de las características no funcionales del sistema, como el rendimiento y la disponibilidad.
3. Una vista de desarrollo, que muestre cómo el software está descompuesto para su desarrollo, esto es, indica la descomposición del software en elementos que se implementen mediante un solo desarrollador o equipo de desarrollo. Esta vista es útil para administradores y programadores de software.
4. Una vista física, que exponga el hardware del sistema y cómo los componentes de software se distribuyen a través de los procesadores en el sistema. Esta vista es útil para los ingenieros de sistemas que planean una implementación de sistema.

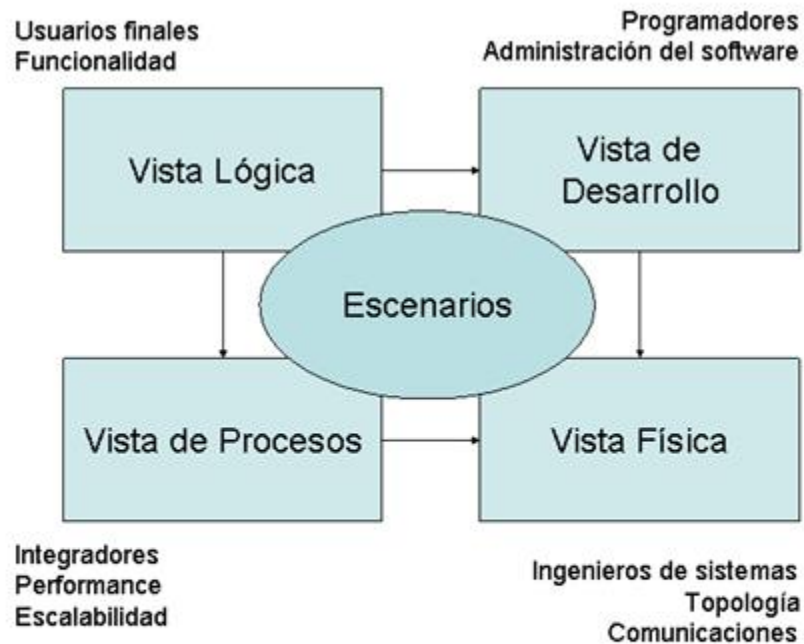
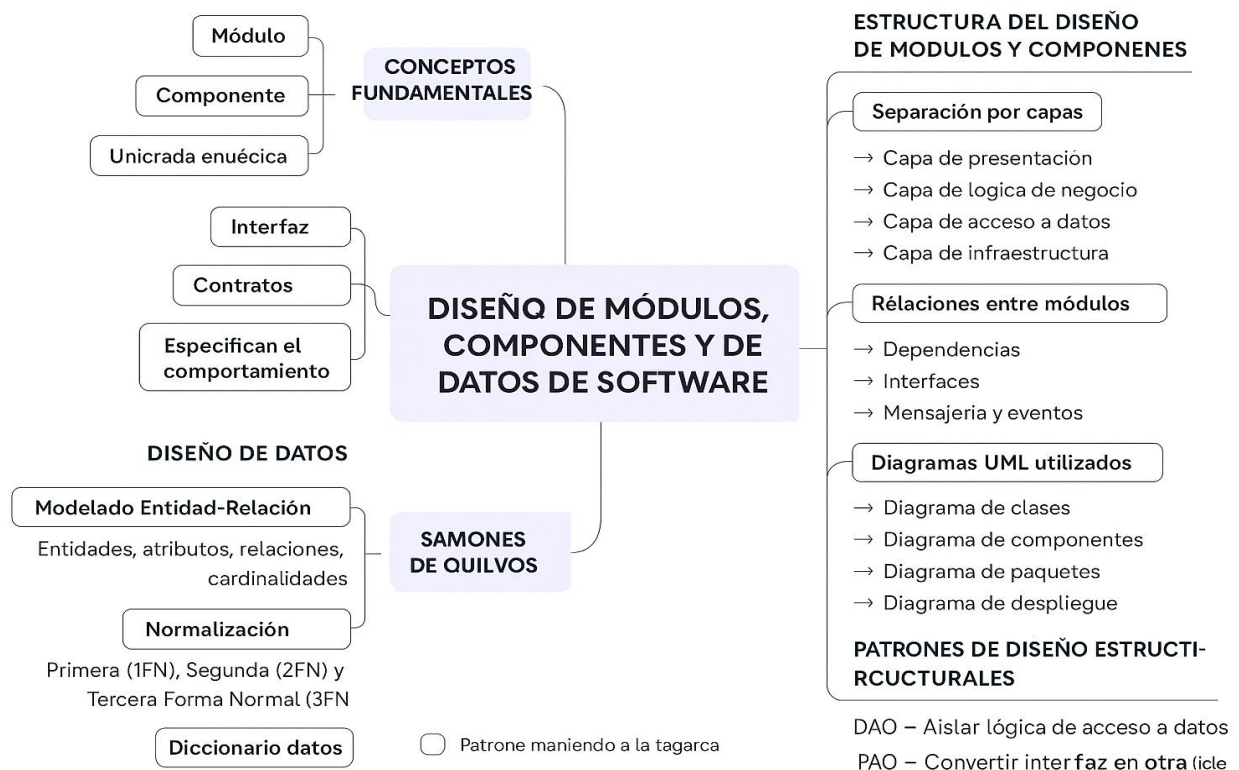


Figure 1: Modelo de “4+1” vistas

Diseño de módulos, componentes y de datos de software.



propósito es dividir el sistema en partes más pequeñas y manejables

(Ejemplos: módulo de autenticación, módulo de reportes, módulo de pagos.)

¿QUÉ ES UN COMPONENTE?

R= Un componente de software es una **unidad modular con interfaces explícitas**, diseñada para ofrecer servicios o funcionalidades específicas dentro de un sistema.

¿QUÉ ES INTERFAZ DE SOFTWARE?

Punto de **comunicación** entre módulos o componentes.

Define los métodos disponibles, tipos de datos aceptados y contratos de servicio.

¿QUE ES UN CONTRATO?

Especifican **cómo debe comportarse** un servicio o componente.

¿QUÉ ES DISEÑO MODULAR?

El **diseño modular** es un enfoque **arquitectónico que organiza un sistema complejo en módulos independientes**, cada uno encargado de una función o conjunto de funciones relacionadas. Estos módulos interactúan entre sí mediante **interfaces bien definidas**, lo que permite construir sistemas más escalables, mantenibles y reutilizables.

- **Módulo:** unidad de software que agrupa funciones relacionadas; se caracteriza por alta cohesión y bajo acoplamiento.
- **Componente:** módulo o conjunto de módulos que pueden desplegarse o reutilizarse como una unidad; tiene interfaces claras.
- **Datos de software:** todas las estructuras que el sistema usa para almacenar, organizar y manipular información — modelos de datos, entidades, relaciones, estructuras físicas.
- **Cohesión:** grado en que todas las funciones de un módulo están fuertemente vinculadas entre sí.
- **Acoplamiento:** grado en que un módulo depende de otros; menor acoplamiento es mejor.
- **Encapsulamiento:** ocultar detalles internos y exponer solo lo necesario mediante interfaces.
- **Reutilización:** usar componentes existentes para ahorrar coste, tiempo y mejorar calidad.
- **Interfaces de componentes:** puntos de interacción definidos — entradas, salidas, protocolos.
- **Modelado de datos:** diseño lógico, diseño físico, normalización, persistencia.
- **Dependencias de datos:** cómo los módulos acceden, modifican, consultan los datos.

Estructura de diseño de Módulos y Componentes

SEPARACION POR CAPAS

La arquitectura por capas **divide el sistema en niveles funcionales independientes**, donde cada capa tiene una responsabilidad específica. Esto **facilita el diseño modular**, mejora la mantenibilidad y permite escalar el sistema

con claridad. (SEPARACION DE PREOCUPACIONES DANDO ENFASIS A CADA CAPA)

1. CAPA DE PRESENTACION (UI)

>> Interacción con el usuario/ Muestra información al usuario y recoge sus acciones/ PUNTO DE CONTACTO DIRECTO CON EL USUARIO/ RECIBE LAS ACCIONES DEL USUARIO Y MUESTRA LOS DATOS DE LA LOGICA DE NEGOCIO.

>>EJEMPLO: Pantalla de login.

2. CAPA DE LOGICA DE NEGOCIO

>>Procesar reglas del sistema, validar datos, coordinar operaciones / El núcleo de la aplicación / Coordinar y validar datos conforme las reglas internas del sistemas.

3. CAPA DE ACCESO A DATOS

>> Interactuar con la base de datos / Provee los métodos para conectar, consultar, actualizar y eliminar la información persistente

4. CAPA DE INFRAESTRUCTURA O SERVICIOS

>> Proveer servicios técnicos transversales.

>>Esta capa contiene **funcionalidades transversales que dan soporte a las demás capas.** No maneja lógica de negocio directamente, sino que **provee servicios técnicos esenciales para que el sistema funcione correctamente.**

RELACIONES ENTRE MODULOS

En un sistema modular, los módulos **no funcionan aislados: se relacionan entre sí** para cumplir los requerimientos del sistema.

Estas relaciones pueden ser de distintos **tipos**:

1. **Dependencia**: Un módulo **depende** de otro cuando **necesita usar sus funciones, servicios o datos.**

2.**Interfaces**: Son **contratos formales** que definen cómo un módulo puede comunicarse con otro.

>> Las interfaces permiten que los módulos se comuniquen **sin conocer su implementación interna.**

3. Mensajería y Eventos;

>> Es una forma de **comunicación entre módulos o componentes** donde **no se llama directamente a otro módulo**, sino que se **envía un mensaje o se lanza un evento**, y otros módulos **deciden si lo escuchan o no**.

>> **Flexibilidad**: cada módulo decide qué eventos escuchar.

DIAGRAMAS UML UTILIZADOS;

✿ Diagramas UML utilizados en diseño modular

Diagrama UML	¿Qué representa?	¿Para qué se usa?	Ejemplo típico
Diagrama de clases	Clases, atributos, métodos y relaciones (herencia, asociación, composición)	Modelar la estructura interna del software y sus reglas de negocio	Alumno, Curso, Inscripción con relaciones
Diagrama de componentes	Componentes físicos o lógicos y sus interfaces	Visualizar cómo se organizan los componentes y cómo se comunican	AuthService conectado a UserRepository
Diagrama de paquetes	Agrupaciones lógicas de clases o módulos (paquetes)	Organizar el sistema en capas o dominios funcionales	Paquete Seguridad con subpaquetes Login, Roles
Diagrama de despliegue	Nodos físicos (servidores, contenedores) y artefactos que se ejecutan en ellos	Mostrar cómo se distribuye el sistema en infraestructura real	Servidor web con Frontend, servidor de app con API

🧠 ¿Cómo se relacionan con módulos y componentes?

Clases: se agrupan en módulos según su funcionalidad.

Componentes: encapsulan clases y exponen interfaces.

Paquetes: organizan módulos por capas o dominios.

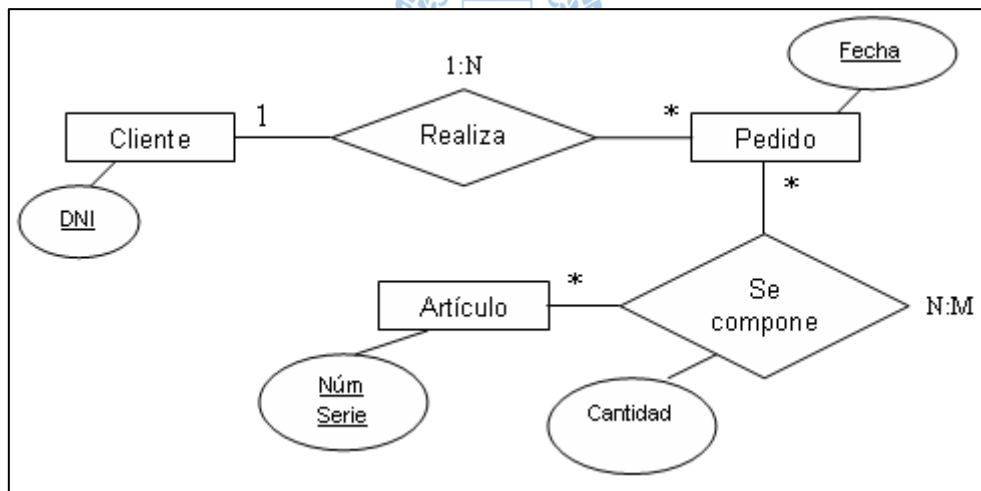
Despliegue: muestra dónde se ejecutan los componentes en la infraestructura.

DISEÑO DE DATOS

>> *MODELO ENTIDAD- RELACION (E-R)*

Es la herramienta conceptual inicial.

- Entidades: Objetos o conceptos importantes (ej., Usuario, Pedido).
- Atributos: Características de las entidades (ej., para Usuario: nombre, email, id).
- Relaciones: Asociaciones entre entidades (ej., Un Usuario realiza varios Pedidos).
- Cardinalidades: Definen cuántas instancias de una entidad se relacionan con cuántas instancias de otra (ej., 1 a N, N a M).



>> **La Normalización:** es el proceso de **organizar las tablas** en una base de datos para **eliminar la información repetida innecesariamente (redundancia)** y asegurar que los datos estén **almacenados lógicamente** y de forma correcta. Se hace siguiendo reglas llamadas Formas Normales (FN).

- **1FN:** Los atributos son atómicos (no se dividen) y no hay grupos repetitivos.
- **2FN:** Ya está en 1FN y todos los atributos no clave dependen completamente de la **clave primaria completa**.
- **3FN:** Ya está en 2FN y no hay dependencias transitivas (ningún atributo no clave depende de otro atributo no clave).

>>Diccionario de Datos

Es un catálogo centralizado que contiene **metadatos**, es decir, información sobre los datos en la base.

- Describe la **estructura** de la base de datos (tablas, campos, relaciones).
- Detalla el **formato** y las **restricciones** de cada campo (ej., **Campo: precio, Tipo: decimal(10,2), restricción: no nulo**).

Importancia: Es vital para la documentación, el mantenimiento y la comunicación entre desarrolladores y analistas.



Tabla hospital_usuario

Clave primaria	Nombre de campo	Tipo de datos	Valor nulo	único	condición	Descripción
PK	Id_usuario	Int	NO	SI	NO	Clave primaria del usuario que trabaja en el hospital
	Error: Reference source not found	VarChar (30)	NO	NO	NO	nombre del usuario
	apellidos	VarChar (40)	NO	NO	NO	apellido del usuario
	direccion	VarChar (50)	NO	NO	NO	Dirección del usuario
	Teléfono	bigint	NO	NO	NO	Teléfono del usuario
	password	Varchar(15)	NO	NO	NO	Contraseña del usuario

Tabla user_trabajo

Clave primaria	Nombre de campo	Tipo de datos	Valor nulo	único	condición	Descripción
PK	Error: Reference source not founduser t	Int	NO	SI	NO	Clave primaria de la tabla
Fk	Id_usuario	Int	NO	NO	NO	Clave foránea del usuario del hospital
Fk	Id_depto	Int	NO	NO	NO	Clave foránea del departamento

>>Esquema

Un esquema es como el plano que describe la estructura de lo que vamos a construir. Hay dos niveles de planos:

Esquema Lógico	Esquema Físico
Tablas y Relaciones	Índices y Tipos de Datos
Se centra en la organización de la información.	Se centra en el rendimiento y el almacenamiento .
"Necesito una tabla de Clientes y una de Productos."	" <i>Nombre</i> será un <code>VARCHAR(100)</code> y tendrá un índice."

PRINCIPIOS DE DISEÑO (SOLID Y COMPLEMENTARIOS)

>> SOLID es un acrónimo que representa un conjunto de **cinco principios de diseño** en la programación orientada a objetos (POO).

□ S → Principio de Responsabilidad Única (Single Responsibility Principle)

>> Una clase, módulo o función **debe tener solo una razón para cambiar**. Esto significa que solo maneja una única tarea o área de la funcionalidad del sistema.

>> Tiene cohesión alta, es decir, **la clase está altamente enfocada en una cosa**.

□ O → Principio Abierto/Cerrado (Open/Closed Principle)

>> Una entidad de software (clase, módulo, etc.) debe estar abierta a la **extensión** (pueden agregar nuevas funcionalidades) pero **cerrada** a la modificación (no debes cambiar el código existente).

>> Se logra mediante polimorfismo e interfaces.

□ L → Principio de Sustitución de Liskov (Liskov Substitution Principle)

>> Un objeto de una clase **subtipo** (clase hija) debe poder **reemplazar** a un objeto de su **supertipo** (clase padre) sin alterar la corrección del programa. **En otras palabras, la subclase no debe romper el comportamiento esperado de la superclase.**

□ **I → Principio de Segregación de Interfaces (Interface Segregation Principle)**

>> Es mejor tener **muchas interfaces específicas** (pequeñas y con pocos métodos) que una **única interfaz grande** (que obliga a implementar métodos que no se usan).

>> Evita que las clases dependan de métodos que **no utilizan**.

>> En lugar de una interfaz IAdministradorTareas con métodos imprimir(), guardar(), enviarCorreo() y configurarDB(), debes crear interfaces más pequeñas: IImprimible, IGuardable, IEnvable y IConfigurable. Una clase solo implementa la interfaz con las funcionalidades que realmente necesita.

□ **D → Principio de Inversión de Dependencias (Dependency Inversion Principle)**

>> Los módulos de alto nivel no deben depender los módulos de bajo nivel. Ambos deben depender de abstracciones.

- Otros principios relacionados:

→ **Encapsulamiento**: Consiste en **agrupar los datos** (atributos) y los **métodos** (funciones) que operan sobre esos datos, dentro de una misma entidad (clase u objeto). Al mismo tiempo, **protege los datos internos** del acceso o modificación directa e indebida desde fuera del objeto.

→ **Separación de Responsabilidades (Separation of Concerns - SoC)**: Se trata de **dividir** un sistema de software complejo en **partes distintas** (módulos, capas, componentes), donde cada parte se enfoca en una **responsabilidad específica**.

→ **Control de Acceso Basado en Roles (Role-Based Access Control - RBAC)**: En lugar de asignar permisos a usuarios individuales, se asignan permisos a **roles**, y luego los usuarios reciben uno o más de esos roles. Un **rol** agrupa un conjunto de **permisos** o **operaciones** que un tipo de usuario puede realizar.

Patrones de Diseño Relevantes

→ Creacionales

Los patrones creacionales se centran en los mecanismos de **creación de objetos**, tratando de crear objetos de una manera adecuada a la situación. Su objetivo es incrementar la flexibilidad y la reutilización del código.

I. Singleton (Instancia Única)

→ Garantiza que **SOLO EXISTA UNA INSTANCIA** de una **clase en todo el sistema** y proporciona un punto de acceso global a ella .

II. Factory Method / Abstract Factory (Fabricas)

→ factory method : Define una **interfaz para crear un objeto**, pero deja que las subclases decidan qué clase instanciar. Crea *familias de objetos*.

→ Abstract Factory : Crea una **familia completa de objetos relacionados o dependientes** sin especificar sus clases concretas. Crea *familias de fábricas*.

III. Builder (Constructor)

→ Permite **construir objetos complejos** paso a paso y con diferentes representaciones, utilizando la misma lógica de construcción.

IV. Prototype (Prototipo)

→ Permite **clonar (copiar) objetos existentes** sin hacer que el código dependa de sus clases concretas.

→ Requiere que las clases a clonar implementen una interfaz de clonación (Cloneable).

→ ESTRUCTURALES

Patrón	Uso	Ejemplo
Adapter	Compatibilizar interfaces incompatibles.	Exportar historiales a sistemas externos.
Bridge	Separar abstracción e implementación.	Generar reportes en distintos formatos.

Patrón	Uso	Ejemplo
Composite	Estructuras jerárquicas (tareas, sub-tareas).	Gestión de proyectos.
Decorator	Agregar funciones dinámicamente.	Promociones acumulables.
Facade	Simplificar subsistemas complejos.	API única para módulos internos.
Proxy	Controlar acceso a recursos.	Proxy de seguridad o cache.
Flyweight	Reutilizar objetos livianos repetidos.	Productos con variantes similares.

→COMPORTAMENTALES

Patrón	Uso	Ejemplo
Strategy	Intercambiar algoritmos fácilmente.	Calcular precios según cliente o promoción.
Observer	Notificar automáticamente cambios.	Notificaciones en tiempo real o alertas.
Command	Encapsular acciones (undo/redo).	Control de versiones, deshacer cambios.
State	Cambiar comportamiento según estado.	Objetos "Activo", "Suspendido".
Memento	Guardar/restaurar estados previos.	Reanudar lección donde se dejó.
Chain of Responsibility	Procesar solicitudes secuenciales.	Validaciones múltiples o niveles de soporte.
Mediator	Coordinar objetos sin acoplamiento directo.	Torre de control con aeronaves.
Template Method	Definir estructura común con pasos personalizables.	Flujo de cotización de seguros.

Arquitecturas y Estilos Relacionado

Arquitectura	Definición	Característica Distintiva
Monolítica	Modelo tradicional donde todas las funcionalidades (interfaz de usuario, lógica de negocio y capa de datos) están unificadas dentro de un único artefacto de despliegue y se ejecutan como un solo proceso o servicio.	Unidad única de código y despliegue; fuerte acoplamiento entre componentes.
Cliente–Servidor	Patrón arquitectónico que distribuye la carga de trabajo entre dos entidades principales: el Cliente (solicitante de servicios) y el Servidor (proveedor centralizado de servicios y recursos).	Distribución de tareas entre el consumidor (cliente) y el gestor de recursos (servidor).
En Capas (N-Tier)	Estilo de arquitectura que segmenta la aplicación en componentes lógicos y físicos organizados en capas horizontales (ej. Presentación, Lógica de Negocio, Acceso a Datos), donde la comunicación es restringida y unidireccional entre ellas.	Separación estricta de responsabilidades ; cada capa tiene un rol específico y aislado.
Orientada a Servicios (SOA)	Colección de servicios interconectados que se comunican entre sí para soportar procesos de negocio. Los servicios son modulares, reutilizables y se definen con estándares de interfaz para lograr la interoperabilidad.	Énfasis en la reutilización de funcionalidades de negocio a través de servicios con acoplamiento flexible.

Arquitectura	Definición	Característica Distintiva
Microservicios	Enfoque que estructura una aplicación como un conjunto de servicios pequeños y autónomos , modelados alrededor de capacidades de negocio. Cada servicio es desplegable de forma independiente y puede utilizar su propia tecnología y almacenamiento.	Independencia total en desarrollo, despliegue y escalabilidad de cada servicio funcional.
Orientada a Eventos (EDA)	Patrón de arquitectura donde los componentes se comunican a través de la producción, detección, consumo y reacción a eventos . Los servicios operan de forma asíncrona y están fuertemente desacoplados.	Comunicación asíncrona basada en notificaciones de cambio de estado ("eventos").
Batch Processing	Método de procesamiento automático de datos donde las transacciones o datos de entrada se agrupan en lotes y se ejecutan de forma secuencial en un equipo sin interacción continua del usuario durante la ejecución.	Procesamiento masivo y diferido de conjuntos de datos, priorizando el rendimiento sobre la inmediatez.

Estructura de módulos, diseño de componentes, interfaz y contrato, estructuración de modelos de datos que respaldan módulos y componentes.
 principios solid , patrones de diseño(estructurales).

Visión Orientada a Objetos

- Un componente contiene un conjunto de clases que colaboran.
- Cada clase dentro de un componente se elabora por completo para que incluya todos los atributos y operaciones relevantes para su implantación.
- También deben definirse todas las interfaces que permiten que las clases se comuniquen y colaboren.
- Para lograr esto, se elaboran clases de análisis y clases de infraestructura.



Módulos

- Se conoce como módulo a una estructura o bloque de piezas que, en una construcción, se ubican a fin de hacerla más sencilla. Estos tienen tres funciones importantes:
 1. Coordina la invocación de los componentes del dominio del problema.
 2. Implanta una necesidad que requiere el cliente.
 3. Es responsable de las funciones que dan apoyo al procesamiento requerido en el dominio del problema.

- También llamados módulos, estos provienen del modelo de análisis y contiene tres funciones importantes:

- 1) **Componentes de control:** coordina la invocación de todos los demás componentes.
- 2) **Componentes del dominio del problema:** implanta una función completa o parcial que requiere el cliente
- 3) **Componente de infraestructura:** responsable de las funciones que dan apoyo al procesamiento requerido.

Principios básicos del diseño

- Hay cuatro principios básicos que son aplicables al diseño en el nivel de componentes y que han sido ampliamente aceptados para la aplicación de la ingeniería de software orientada a objetos: La motivación para aplicar estos principios es crear diseños que sean más factibles de cambiar, así como reducir la propagación de efectos colaterales cuando se hagan cambios. Estos principios pueden usarse como guía cuando se desarrolle cada componente del software.

Principios básicos en el diseño

Existen cuatro principios básicos que se aplican al diseño de nivel de componentes:

- **Principio Abierto – Cerrado (PAC):** Un modulo debe ser abierto para la extensión pero cerrado para la modificación.
- **Principio de sustitución de Liskov (PSL):** una clase de base debe funcionar bien si una clase derivada de la clase base pasa al componente.
- **Principio de Inversión de la Dependencia (PID):** entre mas dependa un componente de otros componentes concretos, mas difícil será ampliarlo.
- **Principio de segregación de la interfaz (PSI):** el PSI sugiere que debe crear una interfaz especializada que atienda a cada categoría principal de clientes.

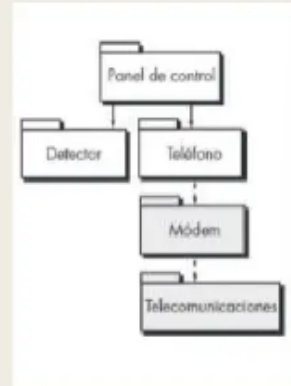


Cohesión

La cohesión implica que un componente o clase sólo contiene atributos y operaciones que se relacionan de cerca uno con el otro y con la clase o componente en sí.

Existen 3 categorías:

1. Funcional. Lo tienen sobre todo las operaciones; este nivel de cohesión ocurre cuando un componente realiza un cálculo y luego devuelve el resultado.
2. De capa. Lo tienen los paquetes, componentes y clases; este tipo de cohesión ocurre cuando una capa más alta accede a los servicios de otra más baja, pero ésta no tiene acceso a las superiores.
3. De comunicación. Todas las operaciones que acceden a los mismos datos se definen dentro de una clase.



Acoplamiento

- Es la medición cualitativa del grado en el que las clases se conectan una con otra. Conforme las clases (y componentes) se hacen más interdependientes, el acoplamiento crece. Un objetivo importante del diseño en el nivel de componente es mantener el acoplamiento tan bajo como sea posible.

Reutilización de Componentes

- En las últimas dos décadas, la comunidad de la ingeniería de software ha puesto el énfasis en la necesidad de elaborar sistemas que utilicen componentes ya existentes.
- En esencia, a medida que avanza el trabajo de diseño se dispone de un catálogo de diseño probado o de componentes en el nivel de código.
- Como estos componentes fueron contruidos teniendo en mente lo reutilizable, se dispone totalmente de la descripción de su interfaz, de las funciones que realizan y de la comunicación y colaboración que requieren.



Desarrollo de Sistemas de Software

3.1 Lenguajes de desarrollo de software

El propósito general es comprender los tipos, características y diferencias de los lenguajes de programación, así como sus paradigmas, entornos de ejecución, mecanismos de tipado, concurrencia, memoria, seguridad y criterios de selección.

3.1.2 Conceptos Fundamentales

>> Lenguaje de Programación: Es un sistema formal de instrucciones, símbolos y reglas que se utiliza para escribir el código fuente de un programa. Permite a los desarrolladores especificar los algoritmos que la computadora debe ejecutar.

>>Paradigma: Es un estilo o modelo mental que dicta la forma en que el programador concibe, estructura y organiza los programas y soluciones. / Es una forma de ver y entender el mundo. Es como un conjunto de ideas, creencias o modelos que usamos para interpretar lo que pasa a nuestro alrededor.

>> Sintaxis: Conjunto de reglas gramaticales que definen cómo deben combinarse las palabras reservadas, operadores y símbolos para formar instrucciones válidas.

>>Semántica: Es el **significado** de las instrucciones o estructuras de código. Si bien la sintaxis define si el código está bien escrito, la semántica define lo que realmente **hace** ese código. Un código puede ser sintácticamente correcto, pero semánticamente incorrecto si no hace lo que se espera.

-- **Ejemplo:** El operador `x++` es sintácticamente correcto y su semántica es clara: **incrementa el valor de la variable x en una unidad.**

>>Compilación/Interpretación: Se refieren a los dos métodos principales para ejecutar el código fuente escrito por el desarrollador. Es el proceso de traducir el código humano a lenguaje de máquina (código binario) que el hardware puede entender.

- **Compilación:** Todo el código fuente se traduce a lenguaje de máquina (un archivo ejecutable) antes de la ejecución (ej. C, C++).
- **Interpretación:** El código fuente se traduce e inspecciona línea por línea durante la ejecución (ej. Python, JavaScript).

3.1.3 Clasificación de Lenguajes

Propósito General (GPL - General-Purpose Languages): Estos lenguajes están diseñados para ser versátiles y se pueden usar para crear casi cualquier tipo de aplicación. No están limitados a un dominio o plataforma específica, lo que los hace ideales para la mayoría de los desarrollos de software.

- **Java:** Fuerte en aplicaciones empresariales a gran escala, sistemas Android y servidores. Es conocido por su filosofía "escribe una vez, ejecuta en cualquier lugar" (Write Once, Run Anywhere - WORA).
- **C#:** Muy utilizado en el ecosistema de Microsoft (aplicaciones de escritorio en Windows, desarrollo web con ASP.NET, juegos con Unity).
- **Python:** Muy utilizado en ciencia de datos, machine learning (TensorFlow, PyTorch), automatización de tareas y desarrollo web (Django, Flask).

Scripting: Los lenguajes de *scripting* a menudo se refieren a aquellos que son **interpretados** (no compilados antes de la ejecución) y se utilizan principalmente para automatizar tareas, manipular datos o proporcionar funcionalidad a una aplicación existente (como un navegador web o un sistema operativo).

- **Python:** Lenguaje de propósito general ampliamente utilizado en el desarrollo de scripts para automatización de sistemas, procesamiento de datos y ejecución de pruebas. Su sintaxis clara y su extensa biblioteca estándar lo convierten en una herramienta versátil en entornos académicos y profesionales.
- **JavaScript:** Inicialmente diseñado para incorporar interactividad en el frontend de páginas web, actualmente también se emplea en el desarrollo backend mediante entornos como Node.js. Es un componente esencial en el desarrollo web moderno, tanto en aplicaciones cliente como servidor.
- **Ruby:** Reconocido por su sintaxis legible y concisa, Ruby es especialmente popular en el desarrollo ágil de aplicaciones web gracias al framework Ruby on Rails. Este entorno permite construir aplicaciones funcionales en tiempos reducidos, lo que lo hace atractivo en contextos educativos y emprendedores.

Dominio Específico (DSL - Domain-Specific Languages): Son lenguajes especializados cuyo diseño y sintaxis están optimizados para resolver problemas en un área de aplicación muy concreta. Esto permite que el código sea más expresivo y cercano al lenguaje natural del dominio.

- **SQL (Structured Query Language):** Diseñado exclusivamente para gestionar y consultar bases de datos relacionales.
- **HTML (HyperText Markup Language):** Lenguaje de marcado, no de programación en sentido estricto, para estructurar el contenido de páginas web.
- **R:** Específico para el cálculo estadístico, análisis de datos y gráficos.
- **CSS (Cascading Style Sheets):** Utilizado para definir la presentación y el estilo de documentos escritos en un lenguaje de marcado (como HTML).



Bajo Nivel: Estos lenguajes están muy cerca de la arquitectura del hardware de la computadora. Permiten un control muy detallado sobre la memoria y el procesador, lo que resulta en programas extremadamente rápidos y eficientes, pero son difíciles de escribir y menos portables.

- **Lenguaje Máquina:** (Primera Generación): El único lenguaje que el procesador entiende directamente; es una secuencia de código binario (0s y 1s).
- **Lenguaje Ensamblador:** (Segunda Generación): Usa nemónicos (abreviaturas como MOV para mover, ADD para sumar) para representar las instrucciones del lenguaje máquina. Se usa en desarrollo de drivers, sistemas operativos y optimización de código crítico.

Alto Nivel: Estos lenguajes están diseñados para ser fáciles de entender, leer y escribir por los humanos. Abstraen (ocultan) los detalles de la

memoria, los registros del procesador y el código máquina, lo que permite al desarrollador concentrarse en la lógica del problema.

Java, Python, C#, JavaScript, Kotlin, Swift: Todos los ejemplos de propósito general y scripting mencionados anteriormente caen en la categoría de alto nivel.

3.1.4 Tipos de Tipado

El sistema de tipos de un lenguaje define cómo se clasifican los valores y cómo interactúan. Se clasifican principalmente según su comprobación (timing) y su coerción (strength).

Estático (Statically Typed): La verificación se hace por el **compilador** antes de que el programa se ejecute. El tipo de una variable es **fijo** y conocido durante todo el ciclo de vida del programa.

Mecanismo: El compilador exige que se declaren (o infieran) los tipos y verifica que todas las operaciones sean válidas. Por ejemplo, no te dejará sumar una variable de tipo Integer con una variable de tipo String.

Dinámico (Dynamically Typed): La verificación se realiza por el **intérprete** en el momento de la ejecución. El tipo está ligado al **valor**, no a la variable. Una variable puede contener un número y, más tarde, una cadena de texto.

Mecanismo: Cuando se ejecuta una línea de código, el sistema revisa los tipos de los valores en ese instante. Si la operación no es válida (ej., intentas llamar un método de lista en un número), el programa **colapsa** en ese punto.

Fuerte (Strongly Typed): El lenguaje **no realiza conversiones automáticas** entre tipos distintos a menos que exista un riesgo cero de perder información. Cualquier conversión potencialmente peligrosa (como de texto a número) debe ser explícita por el programador.

Mecanismo: El sistema prefiere **fallar** a adivinar. La operación `5 + "hola"` resultará en un **error** porque los tipos son incompatibles, obligando al programador a decidir cómo tratarlos.

Débil (Weakly Typed): El lenguaje realiza **conversiones implícitas y automáticas** ("coerción") entre tipos en tiempo de ejecución para intentar que una operación sea exitosa.

Mecanismo: El sistema asume lo que quieres. Por ejemplo, si sumas un número y un *string*, el lenguaje puede convertir el número a *string* para concatenar, o el *string* a número para sumar, dependiendo del operador.

3.1.5 Compilación, Interpretación y Ejecución:

Los métodos de traducción determinan cómo el código fuente escrito por el programador se convierte en instrucciones que la CPU de la computadora puede entender y ejecutar (código máquina).

Compilado (Compiled): El proceso compilado traduce el código fuente directamente a código máquina nativo del sistema operativo y la arquitectura de hardware (por ejemplo, x86 o ARM) antes de cualquier ejecución.

Interpretado (Interpreted): En este modelo, el código fuente se ejecuta directamente en tiempo real, línea por línea, por un programa llamado intérprete.

Mixto (Bytecode / Máquina Virtual - VM): Este modelo busca un equilibrio entre la velocidad de la compilación y la portabilidad de la interpretación.

Just-In-Time (JIT) Compilation: El JIT es una técnica de optimización que se aplica dentro de la ejecución de entornos Mixtos (VM) e Interpretados, combinando lo mejor de ambos.

3.1.6 Gestión de Memoria

La gestión de memoria se refiere a cómo un programa reserva y libera espacio en la memoria de acceso aleatorio (RAM) de la computadora.

Gestión de Memoria Manual: En la gestión manual, el programador es el responsable directo de solicitar (reservar) y devolver (liberar) la memoria al sistema operativo. Si bien ofrece un control total y potencialmente un alto rendimiento, también introduce el riesgo de errores graves

- **C/C++:** Funciones como `malloc()` (Memory Allocation) o `new` para reservar, y `free()` o `delete` para liberar.

Gestión de Memoria Automática (Garbage Collector - GC): La gestión automática, también conocida como Recolección de Basura (Garbage Collection), libera al programador de la tarea de liberar memoria. En su lugar, el sistema de tiempo de ejecución (runtime) supervisa la memoria y automáticamente recupera el espacio ocupado por objetos que ya no son accesibles o "usados" por el programa.

- Java, Python, C#, JavaScript, Go.

Punteros (Pointers): Un puntero es una variable que almacena una dirección de memoria. Permite una manipulación directa y de bajo nivel de la memoria.

- **C/C++:** Se usan para implementar estructuras de datos complejas (listas enlazadas, árboles) y para la gestión de memoria manual.

Referencias (References): Una referencia es esencialmente un **alias** o un nombre alternativo para una ubicación de memoria existente. A diferencia de un puntero, una referencia está más restringida y es más segura.

- **Java, Python, C#:** En estos lenguajes, la mayoría de las variables de objetos son en realidad referencias implícitas (se gestionan automáticamente).
- **C++:** También tiene referencias explícitas que actúan como alias seguros.

3.1.7 Concurrencia y Paralelismo: Conceptos Universitarios:

La programación concurrente y paralela son fundamentales para aprovechar el hardware moderno (múltiples núcleos de CPU) y mejorar la eficiencia y capacidad de respuesta de las aplicaciones.

Concurrencia (Concurrency): La concurrencia se centra en la gestión de muchas cosas a la vez. Es una propiedad de diseño de un programa que maneja múltiples tareas que progresan de forma independiente.

- **Propósito:** Mejorar la capacidad de respuesta y el rendimiento al solapar el tiempo de espera (I/O Bound) con el tiempo de procesamiento. Por ejemplo, mientras un hilo espera datos del disco duro, la CPU puede trabajar en otro hilo.

Paralelismo (Parallelism): El paralelismo se centra en la ejecución real de muchas cosas a la vez. Requiere hardware con múltiples unidades de procesamiento (CPU con varios núcleos, o GPU).

- **Propósito:** Acelerar el procesamiento de tareas intensivas en cómputo (CPU Bound) dividiendo el trabajo. El objetivo principal es reducir el tiempo total de ejecución.

Sincronización: La sincronización es el conjunto de técnicas utilizadas para coordinar la ejecución de múltiples tareas (hilos o procesos) y controlar su acceso a recursos compartidos para evitar condiciones de carrera (Race Conditions).

3.1.8 Ecosistema y Librerías

Frameworks (Marcos de Trabajo): Un *framework* es una estructura de soporte que proporciona las bases y un esqueleto predefinido para construir aplicaciones

Propósito: Los *frameworks* existen para evitar que el desarrollador tenga que escribir el código repetitivo y de infraestructura (como la configuración de servidores, la seguridad, o la conexión a bases de datos). Promueven un principio clave: Inversión de Control (IoC), donde el *framework* llama al código del desarrollador, en lugar de al revés.

Gestores de Paquetes (Package Managers): Un gestor de paquetes es una herramienta de software que automatiza la instalación, actualización, configuración y eliminación de dependencias (bibliotecas de terceros) de un proyecto.

- **Propósito Ampliado:** Solucionan el problema de la Gestión de Dependencias, asegurando que el proyecto use las versiones correctas de todas las bibliotecas externas que necesita, resolviendo a menudo conflictos de versiones automáticamente.

Entorno de Ejecución (Runtime Environment): El entorno de ejecución es el software que proporciona las bibliotecas, los servicios y el contexto necesario para que el código (ya sea interpretado, Bytecode o compilado) pueda ejecutarse correctamente en la máquina.

Propósito Ampliado: Actúa como una capa de abstracción entre el código del programa y el hardware subyacente del sistema operativo. Esto es clave para la portabilidad.

3.1.9 Seguridad y Robustez

Manejo de Excepciones (Exception Handling): El manejo de excepciones es un mecanismo formal para capturar y responder a errores o eventos anormales (excepciones) que ocurren durante la ejecución del programa, sin que este se detenga por completo. Permite separar la lógica normal del programa de la lógica de manejo de errores. Cuando ocurre una condición inesperada (ej., división por cero, acceso a un archivo que no existe), el sistema lanza una "excepción". El bloque try encierra el código que puede fallar, y el bloque catch contiene el código de recuperación o limpieza.

Chequeo de Tipos (Type Checking): El chequeo de tipos es el proceso de verificar si el uso de los valores en el programa es coherente con sus tipos de datos (como se explicó previamente). El objetivo es asegurar que las operaciones solo se realicen en datos compatibles. Por ejemplo, si intentas usar un operador de suma (+) en dos objetos que no están definidos para sumarse, el chequeo de tipos lo detectará. Este chequeo puede ser estático (en compilación) o dinámico (en ejecución).

Control de Acceso (Access Control): El control de acceso son las reglas que determinan qué partes del código (clases, funciones, métodos) pueden acceder a los datos internos (atributos o campos) de otras partes del código. En la Programación Orientada a Objetos (POO). Se implementa mediante modificadores de acceso (access modifiers) que aplican el principio de Encapsulamiento.

3.1.10 Criterios de Selección de Lenguaje

Elegir el lenguaje de programación adecuado es una decisión de arquitectura fundamental que afecta el costo, el tiempo de desarrollo, el rendimiento y el mantenimiento a largo plazo de un proyecto.

Productividad (Velocidad de Desarrollo): Este criterio mide la eficiencia del desarrollador y la rapidez con la que se pueden traducir ideas en código funcional.

Rendimiento (Velocidad y Consumo de Recursos): Este criterio evalúa qué tan rápido y con qué eficiencia de recursos (memoria y CPU) se ejecuta el código en producción.

Tipo de Proyecto (Dominio de Aplicación): La naturaleza del proyecto a menudo dicta el ecosistema y las herramientas disponibles, restringiendo la selección.

Curva de Aprendizaje: Este criterio se centra en el capital humano disponible: la habilidad del equipo actual y la facilidad para contratar nuevos talentos.

Compatibilidad : Este criterio es vital si el nuevo sistema debe interactuar con sistemas existentes o debe operar en múltiples entornos.

Java (Evolución Rápida y Funcional): Java ha pasado de un ciclo de lanzamiento lento a un modelo rápido de seis meses, lo que ha impulsado su modernización y adaptabilidad, manteniendo su dominio en el ecosistema empresarial. Introducción de Lambdas y la API Stream en Java 8. Esto permitió a los desarrolladores adoptar el estilo de programación funcional, escribiendo código más conciso para colecciones y operaciones de datos.

Python (Tipado y Concurrencia): Python, conocido por su alta productividad y uso en datos, se ha enfocado en madurar su ecosistema, resolviendo problemas históricos relacionados con el rendimiento y la flexibilidad del tipado. La migración forzada a Python 3 resolvió problemas fundamentales de codificación de *strings* y preparó al lenguaje para el futuro.

C# (.NET Core y Rendimiento): C# y el ecosistema .NET experimentaron una reestructuración radical con la llegada de .NET Core (ahora simplemente .NET), enfocándose en el código abierto y el alto rendimiento. La transición a .NET Core rompió con el requisito histórico de Windows, permitiendo que las aplicaciones C# se ejecuten nativamente y de manera eficiente en Linux y macOS, transformando a C# en una opción *backend* moderna y portable.

JavaScript / Node.js (Modularidad y Promesas): JavaScript ha pasado de ser un lenguaje de *scripting* de navegador a ser el lenguaje universal, impulsado por la estandarización y la creación de un robusto entorno de ejecución *server-side*. La introducción de la sintaxis `import/export` estandarizó cómo se comparte el código, resolviendo el caos histórico de los módulos. Esto fue clave para el desarrollo a gran escala.



3.2 Paradigmas de Programación.

El propósito general es comprender los modelos fundamentales de programación(paradigmas), sus principios, ventajas, desventajas y aplicaciones. El objetivo es poder **elegir el enfoque adecuado** según el tipo de problema o sistema a desarrollar / Proporcionar un **modelo conceptual y un conjunto de reglas** que guían al programador sobre **cómo debe estructurar y organizar la lógica** del código para resolver un problema.

3.3 Principales Paradigmas de Programación.

>> **Paradigma Imperativo / Estructurado**

Es el modelo de programación más antiguo y básico. **Su foco principal es la modificación del estado del programa** mediante una secuencia de órdenes explícitas. / Que se enfoca en los pasos/

Características Clave

- **Control de Flujo:** Utiliza estructuras explícitas para determinar el orden de ejecución, como:
 - **Condicionales:** if/else (para bifurcaciones).
 - **Bucles:** while, for (para repeticiones).
- **Variables:** Utiliza variables para almacenar datos y, crucialmente, para **modificar su valor** a lo largo del tiempo (cambiar el estado).
- **Procedimientos/Funciones:** Organiza el código en bloques (procedimientos o funciones) que se llaman para realizar tareas específicas, mejorando el orden (de ahí el término "estructurado").

Ejemplo de Examen (Concepto): Si ves un algoritmo que depende de **múltiples bucles y variables globales** que se modifican en serie, su enfoque es claramente **Imperativo**, porque se basa en una secuencia de pasos que alteran el estado.

■ Paradigma Imperativo

◆ En qué se basa:

En decirle paso a paso al computador **cómo** debe hacer algo.

◆ Características:

- Usa **variables globales o locales** para almacenar estados.
- El flujo se controla con **condicionales** (`if` , `switch`), **bucles** (`for` , `while`) y **funciones**.
- Los datos y las operaciones están **separados** (los datos son estructuras, las funciones los manipulan).
- La idea es "dar órdenes": haz esto, luego esto, después aquello.

◆ Ejemplo (C / Python imperativo):

>>Paradigma Orientado a Objetos (OOP)

El POO cambia el enfoque de las instrucciones secuenciales (como en el Imperativo) a la **interacción entre entidades**, que son los objetos.

- **Enfoque Central:** Organizar el software a través de **objetos** que son instancias de **clases**. Cada objeto agrupa:
 - **Datos (Atributos):** Las propiedades o características que describen al objeto (ej., el color de un coche).
 - **Comportamiento (Métodos):** Las acciones que el objeto puede realizar (ej., el método arrancar() de un coche).

LOS 4 PILARES FUNDAMENTALES DEL PARADOGMA ORIENTADO A OBJETOS

1. **Encapsulamiento:** Protege los datos internos de un objeto directo y no autorizado desde fuera.
Tiene como **propósito** asegurar la integridad de los datos y ocultar la complejidad de la implementación.
2. **Herencia:** Permitir que una clase(subclase o clase hija) adquiera los atributos y métodos de otra clase(super clase).
Tiene como **propósito** reutilizar el código.

3. **Polimorfismo:** Es la capacidad de un método de actuar de forma diferente dependiendo del objeto que lo vaya a usar. Se llama “**Muchas formas**”.
4. **Abstracción:** Mostrar solo la información esencial de un objeto y ocultar los detalles internos irrelevantes o complejos. Se enfoca en *qué* hace el objeto, no en *cómo* lo hace.

vs Abstracción vs. Encapsulamiento

Concepto	Enfoque Principal	Objetivo	¿Cómo se logra?
Abstracción	Se enfoca en el diseño y la interfaz (lo que se muestra).	Simplificar la complejidad para el usuario.	Ocultando la complejidad interna y mostrando solo las características esenciales (<i>qué hace</i>).
Encapsulamiento	Se enfoca en la implementación y la protección (lo que se guarda).	Proteger la integridad de los datos internos del objeto.	Agrupando datos y métodos, y restringiendo el acceso directo a los datos (usando <code>private</code> , <code>protected</code>).



>> **Paradigma Funcional:** Es un modelo de programación que se basa en los conceptos de las **funciones matemáticas**. Los programas se construyen mediante **funciones puras**: **Son inmutables quiere decir que los datos (variables) no cambian después de ser creados.**

>> **Paradigma Declarativo/Lógico:** El enfoque declarativo **se opone directamente** al enfoque **imperativo**.

Imperativo se enfoca en los pasos (*el cómo*), y el **Declarativo** se enfoca en el resultado (*el qué*).

Ejemplo de examen:

“Consulta en SQL que define qué datos obtener sin indicar cómo.”

✓ Paradigma declarativo.

Característica	Paradigma Imperativo	Paradigma Declarativo
Enfoque Principal	CÓMO se debe lograr la meta.	QUÉ se desea obtener como resultado.
Rol del Código	Un conjunto de instrucciones que el computador debe ejecutar en orden para modificar el estado.	Una descripción de la lógica o las condiciones que debe cumplir el resultado.
Control de Ejecución	Alto control. El programador define explícitamente el flujo con <code>if</code> , <code>for</code> , <code>while</code> , y los cambios de variables.	Bajo control. El sistema (el motor de inferencia o el optimizador) decide la mejor secuencia para lograr el resultado.
Analogía	Dar una receta de cocina paso a paso.	Pedir un plato específico en un restaurante.

>>Paradigma Reactivo / Basado en Eventos:

Se enfoca en cómo los sistemas responden, se comunican y **se adaptan a un flujo constante de eventos** (mensajes o cambios de estado).

Concepto y Enfoque

- **Enfoque Central:** El programa no espera instrucciones activas, sino que **reacciona de forma asíncrona** a los eventos a medida que ocurren.
- **Eventos:** **Un evento es una señal de que algo significativo ha sucedido** (ej. un clic de ratón, un dato de sensor, un pedido realizado, o el cambio de una variable).
- **Flujo de Datos:** En este modelo, los datos fluyen a través de "canales" o "transmisiones" (observables), y los módulos de la aplicación (observadores) se **suscriben** a esos canales para recibir y procesar los eventos que les interesan.

Ejemplo de Examen (Concepto): Cuando se necesita que **varios módulos** sean **notificados automáticamente** por un cambio **sin que el emisor los conozca directamente**, se aplica el **Paradigma Reactivo** a través del patrón **Observer**.

>> *Paradigma Concurrente / Paralelo*: Se centra en la **ejecución de múltiples tareas al mismo tiempo** (o en apariencia simultánea) para maximizar la eficiencia y utilizar todo el potencial de las máquinas con múltiples procesadores o núcleos (CPUs multinúcleo).

■ 6. Paradigma Concurrente / Paralelo

- **Enfoque**: múltiples tareas o hilos ejecutándose simultáneamente.
- **Objetivo**: aprovechar CPUs multinúcleo.
- **Lenguajes**: Go, Java (Threads), Python (async), Rust (channels).
- **Ventajas**: alto rendimiento.
- **Desventajas**: problemas de sincronización y depuración.

■ Ejemplo de examen:

“Varios procesos actualizan una base de datos al mismo tiempo.”

✓ Paradigma concurrente.

Criterios para Seleccionar un Paradigma



◆ 4. Criterios para Seleccionar un Paradigma

Criterio	Paradigma sugerido
Mantenibilidad y modularidad	Orientado a objetos
Procesamiento concurrente	Funcional o Concurrente
Sistemas expertos o reglas	Declarativo / Lógico
Respuestas en tiempo real	Reactivo / Eventos
Simplicidad en scripts	Scripting
Aplicaciones mixtas y flexibles	Multiparadigma

Paradigma	Foco	Ventajas	Ejemplo
Imperativo	Flujo de instrucciones	Control detallado	C, Pascal
OO	Objetos y clases	Reutilización y encapsulación	Java, C#
Funcional	Funciones puras	Concurrencia y confiabilidad	Haskell, Scala
Declarativo	Qué se desea lograr	Menos código	SQL, Prolog
Reactivo	Eventos y observadores	Tiempo real, desacoplamiento	Node.js, RxJS
Concurrente	Ejecución simultánea	Escalabilidad	Go, Rust

3.3 Entornos de Desarrollo de Software.

Comprender qué es un **entorno de desarrollo (IDE)**, sus componentes principales, tipos y herramientas complementarias que facilitan el ciclo de vida del software.



3.3.1 Conceptos centrales

>> Entorno de Desarrollo de Software (IDE):

Es una **aplicación de software integral** que proporciona a los programadores todo lo que necesitan para escribir y gestionar su código en un solo lugar. / Es el lugar donde el programador **escribe y prueba el código**. Es como el “taller” del desarrollador.

¿Qué incluye?

- Editor de texto para escribir código.
- Compilador o intérprete para ejecutarlo.
- Depurador para encontrar errores.
- Simuladores o emuladores (en algunos casos).

Ejemplo: Visual Studio, IntelliJ IDEA o Eclipse.

>> Entorno de Ejecución: Es el lugar donde el software **realmente se pone en marcha**. Puede ser tu computadora, un servidor o la nube. / **Es el espacio,**

sistema operativo o máquina virtual que proporciona las bibliotecas, recursos y servicios necesarios para que el código compilado o interpretado se ejecute correctamente.

Ejemplo: La Máquina Virtual de Java (**JVM**) es un entorno de ejecución, o un servidor Node.js que ejecuta código JavaScript.

>> Entorno de Integración Continua (CI): Es un sistema **automatizado** que se activa cada vez que un **desarrollador sube o actualiza código nuevo** al repositorio central. Su función es **compilar** ese código y **ejecutar** pruebas automáticas de inmediato. /

Es un sistema que **automatiza la revisión del código** cada vez que alguien lo modifica.

Tiene como objetivo: **Detectar errores**, fallo de integración o bugs lo **más rápido posible**.

>> Entorno Despliegue (CD): **Lugar** o proceso donde el **software se libera al usuario final**.

Se refiere tanto al **lugar (ambiente)** donde el software se instala (ej. servidores de producción) como al **proceso automatizado** mediante el cual el software supera las etapas finales de prueba y es liberado a los usuarios.

Tiene como objetivo asegurar que las nuevas versiones del software, una vez sean probadas, sean liberadas de manera confiable.

3.3.2 Tipos de Entorno en el ciclo de desarrollo

>>Desarrollo (Dev): Es el ambiente LOCAL e INDIVIDUAL en donde los programadores **escriben, prueban y corrigen el código localmente**.

Herramientas Clave: IDEs (VS Code, IntelliJ), **herramientas de control de versiones** (Git).

>>Pruebas (Test / QA): Es un entorno que **simula el comportamiento real del software** para verificar que todo funcione correctamente.

Simula el ambiente real para probar funciones y detectar errores.

Ejemplos: JUnit, Selenium, Postman.

>>Integración Continua (CI): **Automatiza** compilación, pruebas y empaquetado.

Es el **entorno automatizado** que se encarga de **compilar, validar y empaquetar** el código cada vez que se envía al repositorio central.

Tiene como función el verificar la integridad del código de todo el equipo de forma temprana y **continua**, asegurando que **todos los módulos trabajen bien juntos (integración)**.

>>Preproducción (Staging): **Ambiente casi idéntico al de producción para pruebas finales.**

Es un entorno **muy parecido al real**, donde se hacen las últimas pruebas antes de liberar el software.

Ejemplo:

Probar una app en **AWS EC2 Staging** antes de subirla a la tienda.

>> Producción (Prod): **Es el ambiente real y activo** donde el software está disponible para los clientes y usuarios finales.

Ejemplo: El servidor principal de una página web, el *backend* que soporta una aplicación móvil, o la aplicación en una tienda de apps.

3.3.3 Componentes de un Entorno de Desarrollo (IDE).

>> Editor de Código: Es el área principal donde el desarrollador **escribe y modifica** el código fuente.

>>Compilador / Intérprete: Se encarga de la **traducción** del código fuente.

Compilador: **Convierte** todo el código a la vez en un archivo ejecutable **binario** o *bytecode* (ej. C++, Java).

Intérprete: **ejecuta el código línea por línea sin crear un archivo ejecutable intermedio** (ej. Python, JavaScript).

>>Depurador: Es la herramienta que permite **detectar y corregir errores (bugs)** en el código.

>>Control de Versiones: Un sistema que **registra todos los cambios** realizados en el código fuente a lo largo del tiempo.

>> Gestor de Dependencias: Automatiza el proceso de **descargar, instalar y**

configurar librerías o paquetes externos (código escrito por terceros) que el proyecto necesita para funcionar.

Ejemplos: npm (Node Package Manager), pip (Python), o Maven (Java).

>>**Terminal Integrada:** **Una ventana de comandos** (como la consola o *prompt* de Windows/Linux) que está directamente incorporada dentro de la interfaz del IDE.

3.3.4 Ejemplos de Entornos por Lenguaje.

Lenguaje	Entorno de desarrollo (IDE) / Editor	Características destacadas
Java	IntelliJ IDEA, Eclipse, NetBeans	Compilador integrado, refactorización.
Python	PyCharm, VS Code	Ejecución interactiva, depuración visual.
C / C++	Code::Blocks, Dev-C++, Visual Studio	Compilador GCC o MSVC, manejo de memoria.
C# / .NET	Visual Studio	Integración con Azure, GUI con WinForms o WPF.
JavaScript / Web	VS Code, WebStorm	Integración con Node.js, frameworks (React, Angular).
PHP	PhpStorm, VS Code	Servidor local integrado (XAMPP).
Mobile (Android)	Android Studio	Emulador, Gradle, depuración remota.

3.3.5 Herramientas complementarias del entorno

Tipo de herramienta	Función	Ejemplo
Control de versiones	Registro histórico de cambios, colaboración.	Git, GitHub, GitLab
Compilación / Build	Automatiza tareas (compilar, empaquetar).	Maven, Gradle, Make
Pruebas automáticas	Ejecuta pruebas unitarias o funcionales.	JUnit, PyTest, Jest
Integración continua (CI)	Automatiza pruebas e integración.	Jenkins, Travis CI
Despliegue continuo (CD)	Publica nuevas versiones automáticamente.	Docker, Kubernetes, AWS CodePipeline

Tipo de herramienta	Función	Ejemplo
Contenedores / Virtualización	Aisla entornos y dependencias.	Docker, Vagrant, VirtualBox
Monitoreo / Logging	Observa rendimiento y fallos.	Grafana, ELK Stack

3.4 Gestión de Datos

La gestión de datos es un pilar arquitectónico que abarca el conjunto de disciplinas requeridas para diseñar, implementar y garantizar la persistencia, integridad y seguridad de la información a lo largo del ciclo de vida de una aplicación.

3.4.1 Conceptos Fundamentales

Dato: Es la unidad más pequeña de información. Es un hecho, una cifra o un texto crudo sin contexto.

Información: Datos organizados y procesados que ya tienen significado y utilidad

Base de Datos (BD): Un archivo digital gigante y estructurado donde se guarda toda la información.

DBMS (Sistema de Gestión de Bases de Datos): El programa que te permite gestionar, actualizar consultar la BD.

SQL: Es el lenguaje que usas para interactuar con el DBMS

3.4. 3 modelos de Datos

jerárquico: Este fue uno de los primeros modelos de bases de datos. Los datos están estructurados en un formato de árbol (estructura padre-hijo), donde cada registro (nodo) puede tener múltiples hijos, pero cada hijo solo puede tener un único padre. Esta estructura define rutas de acceso muy claras y predecibles.

Red: Surge como una evolución del modelo jerárquico para superar su rigidez. Permite que un registro (llamado "miembro" o "hijo") pueda tener múltiples propietarios (padres). Esto permite representar relaciones más complejas y de muchos a muchos de manera más natural que el modelo jerárquico.

Relacional (RDBMS): Es el modelo más dominante y utilizado en la actualidad. Se basa en el concepto matemático de relación, donde los datos se organizan en

tablas bidimensionales (relaciones) compuestas por filas (tuplas) y columnas (atributos). Las relaciones entre tablas se establecen mediante el uso de claves primarias y foráneas.

Objeto-Relacional: Es una extensión del modelo relacional que intenta integrar características del paradigma de la programación orientada a objetos (POO). Permite que las tablas (relaciones) almacenen objetos complejos (como tipos de datos definidos por el usuario, herencia, funciones y métodos) además de los tipos de datos simples tradicionales.

NoSQL: No es un modelo único, sino una categoría de bases de datos que se desarrollaron para abordar las limitaciones de escalabilidad y flexibilidad del modelo relacional, especialmente en entornos de grandes datos (Big Data) y aplicaciones web de alta concurrencia. El término significa "Not only SQL" (No solo SQL). Tienden a favorecer la disponibilidad y la tolerancia a particiones sobre la consistencia estricta (modelo BASE vs. ACID).

3.4.4 Etapas de la gestión de Datos

Etapas de la Gestión de Datos



- **Análisis de requerimientos de datos:** Identificar qué **información** (entidades, atributos, reglas) necesita el sistema para operar.
- **Modelado conceptual:** Crear un diagrama de alto nivel como el Diagrama E-R) que representa las entidades y sus relaciones.
- **Modelado lógico:** Transformar el modelo conceptual en un esquema detallado para un **modelo específico** (generalmente relacional: tablas, claves foráneas), aplicando **normalización**.
- **Modelado físico:** Definir la **estructura de almacenamiento real** optimizada para un SGBD específico, incluyendo índices, particiones y tipos de datos concretos.
- **Implementación:** creación de la base de datos y su esquema en el servidor utilizando comandos DDL.
- **Población y prueba:** Insertar datos iniciales y validar que la base de datos cumpla con todas las reglas de integridad y requerimientos de rendimiento.

- **Mantenimiento y respaldo:** Asegurar la disponibilidad continua mediante copias de seguridad, recuperación, ajuste de rendimiento y gestión de seguridad.

3.4.5 Modelado Conceptual y Lógico

Entidades

Representan **objetos o conceptos** del mundo real que son **distinguibles** y de los cuales se desea almacenar información. Ejemplos incluyen una **Persona**, un **Producto** o un **Departamento**.

Atributos

Son las **propiedades o características específicas** que describen a una entidad. Cada atributo toma un **valor** dentro de un dominio definido. Por ejemplo, la entidad **Persona** puede tener atributos como *nombre*, *fecha de nacimiento* y *dirección*.



Relaciones

Definen las **asociaciones o vínculos lógicos** que existen y se establecen entre dos o más entidades. Estas asociaciones son significativas para el contexto del negocio modelado. Por ejemplo, un **Empleado trabaja en un Departamento**.

Cardinalidad

Especifica el **número mínimo y máximo de instancias** de una entidad que pueden estar relacionadas con una instancia de otra entidad a través de una relación.

- **1:1 (Uno a Uno):** Una instancia de la Entidad A se relaciona con, como máximo, una instancia de la Entidad B, y viceversa.
- **1:N (Uno a Muchos):** Una instancia de la Entidad A se relaciona con cero, una o muchas instancias de la Entidad B, pero una instancia de la Entidad B se relaciona con, como máximo, una instancia de la Entidad A.

- **N:M (Muchos a Muchos):** Una instancia de la Entidad A se relaciona con cero, una o muchas instancias de la Entidad B, y viceversa.

Normalización:

Es un proceso sistemático utilizado para organizar las columnas y tablas de una base de datos relacional. Su objetivo principal es reducir la redundancia de datos e mejorar la integridad de los datos (evitar anomalías de inserción, actualización y borrado).

Formas Normales Básicas

1FN (Primera Forma Normal): Se cumple si todos los atributos son atómicos (contienen un solo valor indivisible) y no existen grupos repetidos dentro de las filas de la tabla.

2FN (Segunda Forma Normal): Se cumple si está en 1FN y todo atributo no clave depende completamente de la clave primaria completa. Es decir, se eliminan las dependencias parciales de la clave.

3FN (Tercera Forma Normal): Se cumple si está en 2FN y no existen dependencias transitivas. Esto significa que ningún atributo no clave debe depender funcionalmente de otro atributo no clave.

3.4.6 Lenguajes de Gestión de Datos (SQL)

DDL (Data Definition Language): Lenguaje de Definición de Datos

Se utiliza para crear y modificar los objetos que almacenarán los datos, es decir, la estructura y el esquema.

Comando	Función Específica	Alcance
CREATE	Construye un nuevo objeto (tabla, base de datos, vista, índice).	Crea la existencia de la estructura.

Comando	Función Específica	Alcance
ALTER	Modifica la estructura de un objeto existente.	Añade, elimina o cambia las columnas de una tabla.
DROP	Elimina completamente un objeto de la base de datos.	Borra la estructura y todos los datos de la tabla.
TRUNCATE	Elimina todos los registros de una tabla de forma rápida.	Borra solo los datos, manteniendo la estructura de la tabla.

DML (Data Manipulation Language): Lenguaje de Manipulación de Datos

Se encarga de gestionar los datos (registros o filas) que están almacenados dentro de los objetos definidos por DDL.

Comando	Función Específica	Alcance
INSERT	Añade nuevas filas o registros a una tabla existente.	Introduce datos por primera vez.
UPDATE	Modifica los valores de los datos existentes en una o varias filas.	Cambia datos que ya estaban almacenados.
DELETE	Elimina filas o registros específicos de una tabla.	Borra filas que cumplen una condición.

DQL (Data Query Language): Lenguaje de Consulta de Datos

Su propósito fundamental es recuperar y visualizar información de la base de datos sin modificarla.

Comando	Función Específica	Alcance
SELECT	Recupera datos de una o más tablas basándose en criterios.	Muestra los datos. Es el comando más usado en SQL.

DCL (Data Control Language): Lenguaje de Control de Datos

Se utiliza para gestionar los permisos y el acceso de los usuarios a los objetos de la base de datos.

Comando	Función Específica	Alcance
GRANT	Otorga un permiso específico a un usuario o rol.	Permite que un usuario pueda hacer algo (ej. SELECT , INSERT).
REVOKE	Elimina o revoca un permiso previamente concedido.	Retira la capacidad de un usuario para hacer algo.

TCL (Transaction Control Language): Lenguaje de Control de Transacciones

Administra grupos de comandos DML para asegurar que los cambios sean permanentes y consistentes (Principio ACID).

Comando	Función Específica	Alcance
COMMIT	Hace que los cambios realizados en una transacción sean permanentes en la base de datos.	Guarda el trabajo de forma definitiva.
ROLLBACK	Deshace todos los cambios realizados en la transacción desde el último COMMIT o SAVEPOINT .	Revierte la base de datos a un estado previo seguro.
SAVEPOINT	Establece un punto de reversión dentro de una transacción.	Permite hacer un ROLLBACK a un punto intermedio específico.

3.4.7 Transacciones y Concurrency

Transacción

Es una secuencia lógica de una o más operaciones sobre la base de datos (generalmente DML: **INSERT**, **UPDATE**, **DELETE**) que deben ejecutarse completamente o no ejecutarse en absoluto. Su propósito principal es mantener la integridad de los datos. Si alguna operación dentro de la secuencia falla, toda la transacción se cancela.

Propiedades ACID

Son el conjunto de reglas que un sistema de gestión de bases de datos (DBMS) debe garantizar para que una transacción sea considerada fiable y segura:

- **Atomicidad:** Garantiza que la transacción se trata como una unidad indivisible. O se ejecutan todas sus operaciones (éxito), o ninguna de ellas (fallo).
- **Consistencia:** Asegura que una transacción lleva a la base de datos de un estado válido a otro estado válido, manteniendo todas las reglas y restricciones (como claves primarias, foráneas, y *checks*).
- **Aislamiento:** Determina cómo y cuándo los cambios realizados por una transacción son visibles para otras transacciones simultáneas. Cada transacción debe ejecutarse como si fuera la única en el sistema.
- **Durabilidad:** Una vez que una transacción ha sido confirmada (COMMIT), sus cambios son permanentes y deben sobrevivir a fallos del sistema (como un apagón).

Bloqueos (Locks)

Son mecanismos utilizados para sincronizar el acceso a los datos cuando múltiples transacciones intentan leer o modificar la misma información al mismo tiempo. Al aplicar un bloqueo a un recurso (fila, tabla, etc.), se evita que otras transacciones interfieran, manteniendo el **aislamiento** de la transacción en curso.

Commit / Rollback

Son los comandos de **TCL** que controlan el final de una transacción:

- **COMMIT:** Es la acción de confirmar permanentemente todos los cambios realizados por la transacción en la base de datos. Una vez ejecutado, los cambios son duraderos y visibles para otras transacciones.
- **ROLLBACK:** Es la acción de revertir o deshacer todos los cambios que se han realizado desde el inicio de la transacción (o desde el último SAVEPOINT). Esto se hace si la transacción falla o si se detecta un error.

3.4.8 Integridad de Datos y Seguridad de los Datos

La integridad de datos asegura la precisión y consistencia de los datos a lo largo de su ciclo de vida y es gestionada mediante restricciones (constraints).

Tipo de Integridad	Descripción Extendida	Función Esencial	Ejemplo Específico
Integridad de Entidad	Garantiza que cada fila de una tabla pueda identificarse de forma única . Esto se logra asegurando que el atributo (o conjunto de atributos) designado como clave primaria (PRIMARY KEY) no contenga valores duplicados y no sea nulo (NOT NULL).	Identificación única de cada registro.	La columna ID_Empleado debe ser PRIMARY KEY y no puede repetirse.
Integridad Referencial	Mantiene la coherencia de las relaciones entre tablas. Requiere que cualquier valor en una clave foránea (FOREIGN KEY) coincida con un valor en la clave primaria de la tabla a la que hace referencia, o bien, sea nulo.	Previene "registros huérfanos" (datos inconsistentes).	Un ID_Departamento en la tabla Empleados debe existir en la tabla Departamentos.
Integridad de Dominio	Limita y restringe el conjunto de valores válidos que pueden ser almacenados en una columna específica. Se define mediante tipos de datos, longitudes y restricciones adicionales como CHECK o NOT NULL.	Asegura que los datos ingresados sean lógicos y válidos .	La columna Email debe ser un string de menos de 100 caracteres y debe tener un formato válido.

Seguridad de Datos

La seguridad de datos se centra en la protección de la información contra accesos no autorizados y fallos del sistema.

Usuarios, Roles y Privilegios

- **Usuarios:** Son las identidades que interactúan con la base de datos (personas o aplicaciones).
- **Roles:** Son colecciones de privilegios predefinidos. Simplifican la administración al asignar un conjunto de permisos a un grupo de usuarios a la vez (ej: Rol Administrador, Rol Lector).
- **Privilegios:** Son los permisos específicos concedidos a usuarios o roles, controlados mediante los comandos GRANT (otorgar) y REVOKE (revocar) de DCL (Data Control Language). Esto asegura que solo las personas autorizadas puedan realizar ciertas acciones (SELECT, INSERT, DELETE, etc.).

Copias de Seguridad (Backups) y Recuperación

- **Backups:** Son copias de la base de datos tomadas periódicamente. Son esenciales para la Durabilidad (propiedad D de ACID). Permiten restaurar la base de datos a un estado anterior en **caso de fallos de hardware**, corrupción de datos, o errores humanos.
- **Recuperación (Recovery):** Es el proceso de usar la copia de seguridad y los registros de transacciones para devolver la base de datos a su último estado consistente después de un fallo.

Cifrado de Datos Sensibles

- **Cifrado:** Proceso de codificación de datos para que solo las partes autorizadas puedan descifrarlos.
 - **AES (Advanced Encryption Standard):** Es un algoritmo de cifrado simétrico muy utilizado para proteger datos en reposo (almacenados) y en tránsito (enviados). Es excelente para datos sensibles como números de tarjetas de crédito.
 - **SHA (Secure Hash Algorithm):** Es una familia de funciones *hash* que se usan principalmente para la integridad de datos y, crucialmente, para el almacenamiento seguro de contraseñas. Una *función hash* crea una huella digital única e irreversible de los datos originales.

3.4.9 Gestión de Datos en Aplicaciones (Conceptos)

ORM (Object-Relational Mapping): Un traductor que convierte objetos del código de programación (ej. Python, Java) en filas y tablas de la base de datos (y viceversa).

Consultas Parametrizadas: Un método para escribir consultas SQL que separa el comando de los datos, utilizando marcadores de posición para prevenir la inyección SQL.

APIs de Datos: Interfaces que permiten a otras aplicaciones solicitar y manipular datos a través de la red (ej. REST, GraphQL).

Persistencia: La capacidad de los datos de almacenarse de forma permanente y sobrevivir después de que el programa que los creó se haya cerrado.

Migraciones: Archivos de código que permiten hacer cambios controlados y rastreables en el esquema (estructura) de la base de datos a lo largo del tiempo.



3.4.10 Técnicas de Integración de Datos

ETL (Extract, Transform, Load): Es la técnica tradicional y más común para la integración de datos, especialmente utilizada para poblar Almacenes de Datos (Data Warehouses).

- **Extract (Extracción):** Consiste en leer y copiar datos desde diversas fuentes (bases de datos, archivos, APIs).
- **Transform (Transformación):** Es el paso central donde los datos se limpian, validan, normalizan, agregan y reestructuran para que cumplan con los requisitos del destino. Se resuelven problemas de calidad y consistencia.
- **Load (Carga):** Los datos transformados se escriben o cargan en el sistema de destino final, que suele ser un repositorio optimizado para análisis (como un *data warehouse*).

Integración en Tiempo Real (Streaming): Esta técnica se enfoca en el movimiento y procesamiento de datos continuo e inmediato tan pronto como se generan, en lugar de esperar a un lote programado.

- **Propósito:** Permite tomar decisiones o realizar acciones instantáneas basadas en eventos de datos a medida que ocurren.
- **Mecanismos:** Utiliza plataformas de *message queuing* o *event streaming*:
 - **Kafka:** Un *broker* de mensajes distribuido y escalable, usado para manejar *feeds* de datos de alto rendimiento.
 - **RabbitMQ:** Un *message broker* popular para aplicaciones que necesitan comunicar sistemas de forma asíncrona.
 - **APIs (Webhooks, Push APIs):** Utilizadas para notificar a otros sistemas inmediatamente cuando un evento ocurre.

Federación de Datos: La federación de datos (o virtualización de datos) es una técnica que permite a los usuarios y aplicaciones acceder a datos de múltiples fuentes heterogéneas (bases de datos, archivos, hojas de cálculo) sin la necesidad de mover o copiar físicamente los datos a un repositorio central.

3.4.11 Tipos de Bases de Datos NoSQL

Las bases de datos **NoSQL** (Not Only SQL) surgieron para manejar grandes volúmenes de datos y esquemas flexibles que las bases de datos relacionales tradicionales no manejan eficientemente.

Tipo de NoSQL	Estructura de Datos	Uso Principal	Ejemplo Clave
Documentos	Almacena datos en estructuras semiestructuradas similares a documentos anidados (JSON o BSON). Cada documento es independiente y puede tener su propio esquema, ofreciendo gran flexibilidad .	Gestión de contenido, catálogos de productos, perfiles de usuario y datos con esquemas cambiantes.	MongoDB

Tipo de NoSQL	Estructura de Datos	Uso Principal	Ejemplo Clave
Clave-Valor	Es el modelo más simple, que almacena datos como una gran tabla hash o diccionario. Cada elemento se accede mediante una clave única que apunta directamente a un valor.	Caché de datos, gestión de sesiones de usuario, almacenamiento rápido de datos temporales.	Redis , Amazon DynamoDB
Columnar (Wide-Column)	Organiza los datos por columnas en lugar de filas. Una "fila" puede tener una gran cantidad de columnas, pero solo se almacenan juntas aquellas que realmente tienen datos, lo que es eficiente para consultas analíticas.	Almacenamiento de series de tiempo, análisis de Big Data , alta disponibilidad para <i>writes</i> .	Cassandra , HBase
Grafos	Utiliza nodos (entidades) y aristas (relaciones) para almacenar los datos. Las aristas son tan importantes como los nodos y pueden tener propiedades.	Modelado de redes sociales , sistemas de recomendación, detección de fraude, gestión de identidades y accesos.	Neo4j

Big Data: Tecnologías como Hadoop, Spark, Hive permiten manejar grandes volúmenes de información distribuidos.

3.4.12 Administración y Mantenimiento

Monitoreo: El monitoreo implica la supervisión continua de la base de datos para detectar problemas de rendimiento y salud general antes de que se conviertan en fallas graves.

- **Rendimiento (*Performance*):** Seguimiento de métricas clave como la utilización de CPU, memoria y disco, así como la tasa de aciertos del *buffer cache* para asegurar que los recursos sean suficientes.
- **Tiempos de Consulta:** Medición del tiempo que tarda cada consulta en completarse. Esto ayuda a identificar las consultas lentas (*slow queries*) que consumen más recursos y necesitan optimización.
- **Índices:** Seguimiento de la utilización de los índices. Monitorear si los índices se están usando efectivamente o si hay índices redundantes que solo ralentizan las operaciones de escritura.

Backup y Restore

Conjunto de procedimientos esenciales para la Durabilidad (D de ACID) de la base de datos, asegurando la capacidad de recuperación ante fallos o pérdidas de datos.

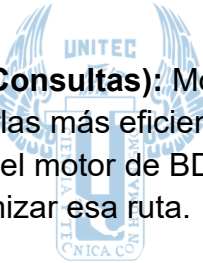


- **Copias Completas (Full Backups):** Una copia de todos los datos y el esquema de la base de datos en un momento dado. Es la copia más grande y el punto de partida para cualquier restauración.
- **Copias Incrementales:** Solo copian los datos que han cambiado desde el último backup (completo o incremental). Son rápidos de hacer, pero la restauración requiere aplicar la copia completa y todas las copias incrementales sucesivas.
- **Copias Diferenciales:** Copian los datos que han cambiado desde el último backup completo. Son de tamaño intermedio. La restauración solo requiere el último backup completo y el último diferencial.

Optimización (Tuning)

Proceso continuo de ajuste de la base de datos y las consultas para mejorar el rendimiento y reducir el uso de recursos.

- **Índices (Indexing):** Creación y ajuste de estructuras especiales que aceleran la búsqueda y recuperación de datos. Son cruciales para las consultas SELECT, pero un exceso puede penalizar las operaciones de escritura (INSERT, UPDATE, DELETE).
- **Particiones (Partitioning):** Técnica que divide una tabla grande en tablas más pequeñas y manejables (particiones), generalmente basándose en un criterio (ej. por fecha o por rango de valores). Esto mejora el rendimiento de las consultas al permitir que el motor solo escanee las particiones relevantes.
- **Query Tuning (Ajuste de Consultas):** Modificación del código SQL de las consultas lentas para hacerlas más eficientes. Implica revisar el plan de ejecución (la ruta que elige el motor de BD para ejecutar la consulta) y reescribir el SQL para optimizar esa ruta.



Auditoría

Proceso de registro sistemático y detallado de las actividades de los usuarios y del sistema dentro de la base de datos.

- **Propósito:** La auditoría proporciona un rastro de quién hizo qué, cuándo y dónde. Esto es vital para la seguridad, la conformidad regulatoria (ej. GDPR, HIPAA) y para investigar actividades sospechosas o fallos de seguridad.
- **Registro de Actividades:** Típicamente registra inicios de sesión fallidos, cambios de permisos (GRANT, REVOKE), y operaciones DML críticas (INSERT, UPDATE, DELETE) en tablas sensibles.

3.5 Plataformas de Desarrollo.

El objetivo de este subtema es comprender que es una **plataforma de desarrollo, los tipos existentes, sus características técnicas, y cómo estas influyen en el desarrollo, despliegue y mantenimiento** del software.

3.5.1 Concepto central

>>Plataforma de Desarrollo: Es el **ecosistema completo** que rodea el proceso de creación de software. Proporciona un entorno estandarizado y coherente. Por ejemplo, la plataforma **.NET** incluye los lenguajes (C#), los *frameworks* (ASP.NET), las librerías base y el entorno de ejecución (CLR).

>>Framework: Es **una estructura de código semi-completa y reutilizable** que define cómo debe estar organizada una aplicación. Un *framework* impone ciertas reglas y patrones de diseño (como el MVC).

En lugar de comenzar desde cero, los desarrolladores construyen sobre esta base estandarizada, centrándose en la lógica específica del proyecto, ya que el framework se encarga de tareas comunes como la gestión de bases de datos, seguridad y ruteo. Funciona como un esqueleto o andamio que impone un flujo de trabajo, lo que permite crear software de manera más eficiente y organizada.

Ejemplos: Django (Python) o Spring (Java) te dicen dónde deben ir tus archivos de modelo, vista y controlador.

>>SDK (Software Development Kit): Es un **paquete de herramientas** completo que los desarrolladores necesitan para construir aplicaciones para una plataforma específica (un sistema operativo, una consola de juegos o un dispositivo).

Función: Proporciona todo lo necesario para interactuar con la plataforma objetivo, incluyendo librerías, *samples* de código, documentación y, a menudo, un emulador o simulador.

>>API (Application Programming Interface): Es un **conjunto de funciones o servicios** que una plataforma ofrece para que otras aplicaciones puedan comunicarse con ella.

Permite que sistemas diferentes se conecten.

Facilita el acceso a datos o funcionalidades externas.



Relación entre ellos

- Una **plataforma** incluye **frameworks**, **SDKs** y **APIs**.
- El **framework** te da la estructura.
- El **SDK** te da las herramientas.
- La **API** te permite comunicarte con otros servicios.

◆ 2. Concepto central

Concepto	Definición
Plataforma de desarrollo	Conjunto de herramientas, lenguajes, frameworks y entornos de ejecución que permiten crear, probar, desplegar y mantener software .
Framework	Estructura o marco predefinido que facilita el desarrollo en una plataforma (ej. Spring, Django).
SDK (Software Development Kit)	Conjunto de herramientas, librerías y documentación necesarias para desarrollar sobre una plataforma específica.
API (Application Programming Interface)	Conjunto de funciones o servicios que una plataforma ofrece a las aplicaciones externas.

Gestión de proyectos de Software

Pruebas y Aseguramiento de la Calidad del Software.

El propósito general de esta unidad es comprender y aplicar los procesos, técnicas y herramientas para **garantizar la calidad del software** mediante pruebas sistemáticas, métricas y buenas practicas a lo largo del ciclo de vida del desarrollo.

Subtema 4.1 — Gestión de tiempos, costos, recursos humanos y riesgos.

4.1.1 Conceptos clave

>> **Proyecto de Software**

Un esfuerzo temporal emprendido para crear un producto, servicio o resultado único relacionado con el desarrollo de software. Los proyectos tienen un inicio y un final definidos, y deben generar algo que antes no existía.

Temporalidad: Tiene una duración limitada, a diferencia de las operaciones continuas. Resultado Único: El producto de software (una app, un módulo, un sistema) debe ser particular.

>> **Gestión de Proyectos**

La aplicación de conocimientos, habilidades, herramientas y técnicas a las actividades del proyecto para cumplir con los requisitos definidos. Implica equilibrar las demandas competitivas de las partes interesadas.

Se organiza en **cinco grupos de procesos** principales: **Iniciación, Planificación, Ejecución, Monitoreo y Control, y Cierre.**

>> **PMBOK (Project Management Body of Knowledge)**

Es un **marco internacional de referencia** y una colección de **estándares y directrices** en gestión de proyectos, desarrollado y mantenido por el Project Management Institute (**PMI**).

Contiene lo siguiente:

◆ **5 Grupos de Procesos**

1. **Inicio** Define el proyecto y obtiene aprobación para comenzarlo.
2. **Planificación** Establece objetivos, cronograma, presupuesto y cómo se ejecutará el proyecto.

3. **Ejecución** Se hacen las tareas del plan, se asignan recursos y se entregan resultados.
4. **Monitoreo y control** Se supervisa el avance, se comparan resultados y se corrigen desviaciones.
5. **Cierre** Se finaliza el proyecto, se entregan productos y se documentan lecciones aprendidas.

◆ 10 Áreas de Conocimiento del PMBOK

Estas áreas agrupan los temas clave que deben gestionarse en todo proyecto.

1. Gestión del Alcance

- Define qué se va a hacer y qué no.
- Incluye la recopilación de requisitos, la creación de la EDT (estructura de desglose del trabajo) y el control de cambios en el alcance.

2. Gestión del Tiempo (Cronograma)

- Estima la duración de tareas, define el orden de ejecución y crea el cronograma.
- Se usa para asegurar que el proyecto se complete a tiempo.

3. Gestión de Costos

- Estima, asigna y controla el presupuesto.
- Incluye análisis de valor ganado y control de desviaciones financieras.

4. Gestión de la Calidad

- Asegura que el producto o servicio cumpla con los requisitos y estándares definidos.
- Incluye planificación, aseguramiento y control de calidad.

5. Gestión de Recursos

- Administra al equipo humano, materiales, tecnología y otros recursos necesarios.
- Incluye planificación de recursos, asignación de roles y liderazgo.

6. Gestión de las Comunicaciones

- Define cómo se transmitirá la información entre los miembros del proyecto y los interesados.

- Incluye planificación, distribución, monitoreo y control de la comunicación.

7. Gestión de los Riesgos

- Identifica eventos que podrían afectar el proyecto.
- Evalúa su probabilidad e impacto, y define respuestas preventivas o correctivas.

8. Gestión de las Adquisiciones

- Administra la compra de bienes o servicios externos.
- Incluye selección de proveedores, contratos, seguimiento y cierre de adquisiciones.

9. Gestión de los Interesados

- Identifica a las personas o grupos que pueden afectar o ser afectados por el proyecto.
- Se analiza su influencia, se gestiona su participación y se busca su compromiso.

10. Gestión de la Integración

- Coordina todos los elementos del proyecto para que funcionen como un sistema coherente.
- Incluye la elaboración del acta de constitución, el plan del proyecto, la dirección y el control de cambios.



>>Triple Restricción

La triple restricción, a veces denominada el "triángulo de hierro", es el modelo central en la gestión de proyectos. **Un cambio en cualquiera de los vértices impacta en los otros dos.**

- **Tiempo (Schedule):** El tiempo necesario para completar el proyecto.
- **Costo (Budget):** Los recursos financieros necesarios.
- **Alcance (Scope):** El trabajo que debe hacerse para entregar el producto.

Ejemplo: Si un cliente exige **más Alcance** (añadir una función), el gestor debe aumentar el **Costo** (más personal) o el **Tiempo** (más duración) para mantener la calidad.

>>Riesgo

Es un **evento o condición incierta** que, si ocurre, puede tener un **impacto positivo o negativo** en al menos uno de los objetivos del proyecto: **tiempo, costo, alcance o calidad**.

Tipos de riesgo

Tipo	Descripción	Ejemplo práctico
Amenaza	Riesgo negativo que puede perjudicar el proyecto.	El proveedor podría retrasar la entrega → afecta el cronograma.
Oportunidad	Riesgo positivo que puede beneficiar el proyecto.	Un nuevo software podría acelerar el desarrollo → mejora tiempos.

4.1.2 Gestión del tiempo (Project Time Management)

El objetivo central es **asegurar que el proyecto se complete en el tiempo planificado**. Implica definir, secuenciar, estimar y controlar el trabajo.

Etapas Clave y Herramientas

1. Definir las Actividades (¿Qué se hará?)

- **Enfoque:** Tomar los entregables de alto nivel definidos en el alcance y descomponerlos en **paquetes de trabajo y tareas concretas** que pueden ser asignadas y estimadas.
- **Herramienta Fundamental: WBS (Work Breakdown Structure)**
 - Es una **estructura jerárquica** que organiza y define el alcance total del proyecto. Cada nivel descendente representa una definición más detallada del trabajo. El nivel más bajo son los paquetes de trabajo.

2. Secuenciar Actividades (¿En qué orden?)

- **Enfoque:** Identificar y documentar las **dependencias lógicas** entre las actividades.
 - **Dependencia Obligatoria:** Inherente al trabajo (ej., no puedes probar el software antes de codificarlo).
 - **Dependencia Discrecional:** Basada en las mejores prácticas (ej., realizar una revisión de diseño).
- **Herramientas: Diagrama de Red y Diagrama de Gantt**

- **Diagrama de Red:** Muestra las tareas y sus dependencias en flujo. Es la base para calcular la Ruta Crítica.
- **Diagrama de Gantt:** Es una vista gráfica de barras que muestra el calendario del proyecto: cuándo empieza y termina cada actividad.

3. Estimar la Duración (¿Cuánto tardará?)

- **Enfoque:** Desarrollar una aproximación del número de períodos de trabajo necesarios para completar cada actividad.
- **Técnicas de Estimación:**
 - **Análoga:** Usar la duración de proyectos pasados similares. (Rápida, pero menos precisa).
 - **Paramétrica:** Usar una relación estadística entre datos históricos y variables del proyecto (ej., si tardamos 2 horas por página, un documento de 10 páginas tardará 20 horas).
 - **Estimación de Tres Puntos (PERT):**

$$T_E = \frac{O + 4M + P}{6}$$

Se usa para reducir la incertidumbre al promediar una estimación Optimista (O), la Más Probable (M), y la Pesimista (P).

4. Desarrollar el Cronograma (¿Cuál es la fecha final?)

- **Enfoque:** Analizar secuencias, duraciones, requisitos de recursos y restricciones para crear el **modelo del cronograma**.
- **Resultado:** El cronograma finalizado y la **Ruta Crítica** identificada.

5. Controlar el Cronograma (¿Estamos a tiempo?)

- **Enfoque:** Monitorear el estado del proyecto para actualizar el progreso y gestionar los cambios a la línea base del cronograma.
- **Herramienta: Gestión del Valor Ganado (Earned Value Management - EVM)**
 - El SPI (Schedule Performance Index) es una métrica clave de EVM:

$$SPI = \frac{EV \text{ (Valor Ganado)}}{PV \text{ (Valor Planificado)}}$$

- **Interpretación:** Si $SPI < 1$, el trabajo realizado (EV) es menor que el trabajo que debería haberse hecho (PV). El proyecto está **retrasado**

1. Ejemplo: Duración Esperada con PERT ⌚

El método **PERT (Program Evaluation and Review Technique)** se usa cuando la duración de una actividad es incierta. Requiere tres estimaciones para calcular una duración promedio ponderada.

La fórmula es:

$$T_E = \frac{O + 4M + P}{6}$$

Donde:



- T_E = Duración Esperada
- O = Duración **Optimista** (Si todo va perfecto)
- M = Duración **Más Probable** (El escenario más realista)
- P = Duración **Pesimista** (Si todo sale mal)

Scenario: Desarrollo de un Módulo de Pago

Imagina que necesitas estimar cuánto tardará la tarea "Integrar pasarela de pago X".

Estimación	Valor (días)	Justificación
Optimista (\$O\$)	4 días	El desarrollador ya ha trabajado con esta pasarela antes.
Más Probable (\$M\$)	6 días	Es el tiempo normal, asumiendo algunas pruebas y ajustes.

Estimación	Valor (días)	Justificación
Pesimista (\$P\$)	16 días	Puede haber problemas de documentación o fallos en el <i>sandbox</i> de pruebas de la pasarela.

Cálculo:

$$T_E = \frac{4 + 4(6) + 16}{6}$$

$$T_E = \frac{4 + 24 + 16}{6}$$

$$T_E = \frac{44}{6}$$

$$T_E \approx 7.33 \text{ días}$$

Conclusión: La duración más realista para planificar en el cronograma es de **7.33 días**. El PERT da más peso a la estimación **Más Probable** (multiplicándola por 4) para obtener una estimación más fiable que un simple promedio.

2. Ejemplo: Índice de Desempeño de Tiempo (SPI)

El **SPI (Schedule Performance Index)** es una métrica de la Gestión del Valor Ganado (EVM) que mide la eficiencia del cronograma.

La fórmula es:

$$SPI = \frac{EV \text{ (Valor Ganado)}}{PV \text{ (Valor Planificado)}}$$

S

Donde:

- **EV (Earned Value - Valor Ganado):** El valor del trabajo **realmente completado** hasta la fecha, expresado en unidades monetarias o de tiempo.
- **PV (Planned Value - Valor Planificado):** El valor del trabajo que **debería haberse completado** hasta la fecha, según el cronograma.

Scenario: Avance de la Fase de Diseño

Imagina un proyecto con un presupuesto total de \$100,000 y una duración de 10 semanas. Al final de la Semana 4, el estado es:

Métrica	Valor (\$)	Significado
Valor Planificado (PV)	\$40,000	Deberíamos haber completado el 40% del trabajo total (\$100,000 / 10 semanas * 4 semanas).
Valor Ganado (EV)	\$32,000	Solo hemos completado trabajo por un valor de \$32,000.

Cálculo del SPI:

$$SPI = \frac{32,000}{40,000}$$

$$SPI = 0.8$$

Conclusión:

Dado que el \$SPI\$ es **0.8 (menor que 1)**, el proyecto está **retrasado**.

- Por cada dólar de trabajo que debería haberse completado (PV), solo se ha completado \$0.80 de trabajo (EV).
- El equipo solo está avanzando al **80% de la velocidad planificada**.

El gestor de proyectos debe investigar las causas del retraso e implementar acciones correctivas (como reasignar recursos o acelerar tareas en la Ruta Crítica).

4.1.3 Gestion de Costos (Project Cost Management)

El objetivo es **planificar y controlar los recursos financieros del proyecto** para asegurar que se complete dentro del presupuesto aprobado.

Etapas:

1. **Estimación de costos:** identificar recursos (personal, software, hardware).
2. **Presupuestar:** asignar costos a cada tarea o paquete de trabajo.
3. **Controlar costos:** comparar costos planificados con reales (**CPI — Índice de desempeño del costo**).

Fórmulas básicas:

- **EV (Valor Ganado)** = Trabajo completado en valor monetario.
- **PV (Valor Planificado)** = Valor del trabajo que debió completarse.
- **AC (Costo Real)** = Lo que realmente se gastó.
- **CPI = EV / AC**
 - $CPI < 1 \rightarrow$ Sobrepresupuesto
 - $CPI > 1 \rightarrow$ Bajo presupuesto
- **EAC = BAC / CPI** $\rightarrow$ Estimado al concluir el proyecto.

Ejemplo de Cálculo y Análisis de Costos

Escenario de Control de Costos

Imagina que tu proyecto tiene un **Presupuesto al Concluir (BAC)** de **\$200,000**. Al final del primer mes, los datos son:

Métrica	Valor (\$)	Significado
Valor Ganado (EV)	\$60,000	El valor del trabajo que realmente hemos completado .
Costo Real (AC)	\$100,000	Lo que realmente hemos gastado para hacer ese trabajo.
Presupuesto al Concluir (BAC)	\$200,000	El presupuesto total original del proyecto.

Cálculo y Significado del CPI

1. Cálculo del Índice de Desempeño del Costo (CPI):

$$CPI = \frac{EV}{AC}$$

$$CPI = \frac{60,000}{100,000}$$

$$CPI = 0.6$$

Análisis:

- El CPI es \$0.6\$ (menor que 1).
- **Conclusión:** El proyecto está **sobrepresupuestado**. Por cada dólar que gastamos (AC), solo estamos obteniendo \$0.60 de trabajo completado (EV).

Respuesta al Ejemplo de Examen

"Si EV = 60, AC = 100, CPI = 0.6, ¿qué indica?"

✓ Indica que el proyecto está gastando más de lo planeado. El trabajo completado (\$60) tiene un costo real (\$100), lo que significa que el proyecto está siendo ineficiente en costos.

2. Cálculo del Estimado al Concluir (EAC)

Una vez que identificamos que estamos sobrepresupuestados (CPI < 1), necesitamos predecir cuánto costará el proyecto si mantenemos esta tasa de gasto ineficiente.

La fórmula que asume que el rendimiento actual (CPI) **continuará** es:

$$EAC = \frac{BAC}{CPI}$$

$$EAC = \frac{200,000}{0.6}$$

$$EAC \approx \$333,333.33$$

Análisis del EAC:

- El **Estimado al Concluir (EAC)** es $\approx \$333,333$.
- **Conclusión:** Si no se toman medidas correctivas, el proyecto superará su presupuesto original de \$200,000 por más de \$133,000.

4.1.4 Gestión de recursos humanos (Human Resource Management)

Objetivo:

Optimizar el uso del talento humano y promover un ambiente de colaboración y productividad.

Etapas:

1. **Planificación del personal:** definir roles, responsabilidades y estructura.
2. **Adquisición del equipo:** reclutar al personal adecuado.
3. **Desarrollo del equipo:** fomentar comunicación, liderazgo, motivación.
4. **Gestión del equipo:** resolver conflictos, evaluar desempeño.

Herramientas Esenciales

Matriz RACI

- **Significado:** Una herramienta de asignación de responsabilidades que aclara quién hace qué.
 - **Responsable:** La persona que hace el trabajo para completar la tarea. (Solo puede haber uno o varios)
 - **Accountable:** La persona **últimamente responsable** de que la tarea se complete correctamente. **Aprueba** el trabajo de los *Responsable*. (Solo puede haber uno).
 - **Consulted:** Personas a consultar antes de tomar decisiones o realizar acciones.
 - **Informed:** Personas que deben ser informadas después de que se toma una decisión o se completa la acción.

Organigrama del Proyecto

- **Uso:** Una representación gráfica de la estructura jerárquica y las relaciones de reporte entre los miembros del equipo del proyecto. Aclara quién le reporta a quién.
-

Modelos de Motivación (Comportamiento del Equipo)

Estos modelos ayudan al Gerente de Proyecto a entender y motivar al equipo:

- **Maslow (Jerarquía de Necesidades):** Postula que las necesidades humanas se satisfacen en un orden jerárquico (Fisiológicas → Seguridad → Sociales → Estima → Autorrealización). Un equipo solo puede ser motivado por la *estima* si sus necesidades básicas (salario, seguridad) están cubiertas.
- **Herzberg (Teoría de los Dos Factores):**
 - **Factores Higiénicos:** Los básicos (salario, condiciones de trabajo). Su ausencia causa insatisfacción, pero su presencia **no motiva**.
 - **Factores Motivadores:** Relacionados con el contenido del trabajo (reconocimiento, logro, responsabilidad). **Solo estos motivan** a un desempeño superior.
- **McGregor (Teoría X y Y):** Describe dos estilos de liderazgo basados en la percepción del Gerente sobre la naturaleza humana:
 - **Teoría X:** Asume que la gente es perezosa, le disgusta el trabajo y necesita ser **controlada y amenazada**.
 - **Teoría Y:** Asume que la gente encuentra satisfacción en el trabajo, es creativa y acepta la responsabilidad. Promueve la **participación y la autonomía**.

4.1.5 Gestión de riesgos en proyectos.

Anticiparse a los problemas antes de que ocurran. Es decir, identificar, analizar y controlar eventos inciertos que podrían afectar el proyecto en tiempo, costo, alcance calidad.

1. Identificación

- Se detectan posibles riesgos mediante:
 - **Lluvia de ideas**
 - **Análisis de causa raíz**
 - **Revisión de proyectos anteriores**
- Se responde: ¿Qué podría salir mal (o muy bien)?

✦ Ejemplo: “¿Qué pasa si el proveedor no entrega a tiempo?”

2. Análisis cualitativo

- Se clasifica cada riesgo según:
 - **Probabilidad** de que ocurra.
 - **Impacto** que tendría si ocurre.
- Se usa una **matriz de riesgos** para priorizar.

✦ Ejemplo: Riesgo con alta probabilidad y alto impacto → prioridad alta.

3. Análisis cuantitativo

- Se estima el **impacto numérico** del riesgo:
 - ¿Cuánto costaría?
 - ¿Cuánto tiempo se perdería?
- Se aplican técnicas como:
 - **Análisis de Monte Carlo**
 - **Árboles de decisión**

✦ Ejemplo: Si el riesgo ocurre, se perderían \$10,000 y 2 semanas.

4. Planificación de respuesta

Se definen estrategias para cada riesgo:

Estrategia	Acción	Ejemplo práctico
Evitar	Eliminar la causa del riesgo.	Cambiar de proveedor para evitar retrasos.
Transferir	Pasar el riesgo a otro (seguro, contrato).	Contratar un seguro contra fallas técnicas.
Mitigar	Reducir la probabilidad o el impacto.	Añadir pruebas extra para evitar errores en producción.
Aceptar	Asumir el riesgo si es bajo o inevitable.	Continuar sin cambios si el impacto es mínimo.

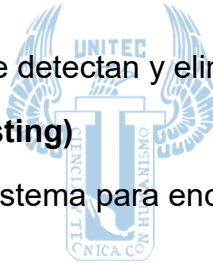
Área	Enlace con las demás
Tiempo	Si se retrasa una actividad, se incrementa el costo.
Costo	Afecta la cantidad de recursos disponibles.
Recursos humanos	Afecta tiempos y riesgos según desempeño.
Riesgos	Impactan tiempo, costo o disponibilidad de personal.

Calidad del Software

Asegurar que los productos, procesos y resultados del proyecto de software cumplan con los requisitos de calidad, las normas internacionales y las buenas prácticas de ingeniería.

4.2.2 Conceptos Clave

- **Calidad del software**
 - Grado en que el software cumple con los requisitos funcionales, no funcionales y expectativas del usuario.
- **Gestión de la calidad**
 - Conjunto de actividades para planificar, asegurar, controlar y mejorar la calidad del proyecto.
- **Aseguramiento de la calidad (QA)**
 - Actividades preventivas que garantizan que los procesos sigan estándares.
- **Control de calidad (QC)**
 - Actividades correctivas que detectan y eliminan defectos.
- **Pruebas de software (Testing)**
 - Ejecución controlada del sistema para encontrar defectos.
- **Mejora continua (PHVA)**
 - Ciclo de perfeccionamiento constante basado en Planear–Hacer–Verificar–Actuar.



4.2.3 Fases de la Gestión de Calidad

Planificación de la Calidad

Objetivo: Definir los estándares y métricas del proyecto.

Actividades clave:

- Definición de políticas de calidad.
- Establecimiento de los requisitos de calidad del producto/proyecto.
- Selección y configuración de herramientas de medición.

Aseguramiento de la Calidad (QA)

Objetivo: Verificar que el equipo siga los procesos establecidos y mejores prácticas.

Actividades clave:

- Realización de auditorías de procesos.
- Ejecución de revisiones de código y documentación.
- Aseguramiento del cumplimiento de normas y estándares internos/externos.

Control de Calidad (QC)

Objetivo: Validar que el producto final cumpla con las especificaciones y requisitos definidos.

Actividades clave:

- Realización de pruebas unitarias.
- Ejecución de pruebas de integración.
- Implementación de pruebas de sistema.
- Coordinación de pruebas de aceptación (por parte del usuario/cliente).



4.2.4 Normas y Modelos de Calidad más importante

Norma / Modelo	Descripción	Enfoque
ISO 9001:2015	Norma de gestión de calidad organizacional.	Procesos de gestión y mejora continua.
ISO/IEC 25000 (SQuaRE)	Evalúa la calidad del producto de software mediante atributos medibles.	Producto final.

ISO/IEC 12207	Define el ciclo de vida del software y sus procesos.	Proceso de desarrollo.
CMMI (Capability Maturity Model Integration)	Evalúa la madurez de los procesos organizacionales.	Nivel de mejora continua.
IEEE 829 / 29119	Estandariza la documentación y ejecución de pruebas.	Testing y QA.
MAAGTICSI (México)	Marco normativo mexicano para calidad y seguridad en TIC.	Sector público.

4.2.5 Atributos de calidad del software (según ISO/IEC 25010)

Funcionalidad: El grado en que el software proporciona funciones que satisfacen las necesidades explícitas e implícitas cuando se usa bajo condiciones específicas. Esto incluye la idoneidad funcional (la totalidad de las funciones), la exactitud de los resultados o efectos proporcionados, y la interoperabilidad con otros sistemas cuando sea necesario.

Eficiencia del Desempeño: El desempeño relativo a la cantidad de recursos utilizados bajo condiciones específicas. Esto se evalúa mediante la conducta temporal (tiempos de respuesta y procesamiento), la utilización de recursos (memoria, CPU, red), y la capacidad máxima del sistema para manejar transacciones o datos. Un software eficiente utiliza recursos y tiempo de forma óptima para su función.

Fiabilidad: La capacidad del sistema para mantener su nivel de desempeño bajo condiciones especificadas y durante un periodo de tiempo determinado. Sus subcaracterísticas clave incluyen la madurez (ausencia de fallos debidos a defectos), la tolerancia a fallos (capacidad de operar a pesar de la presencia de fallos), y la capacidad de recuperación (habilidad para restablecer un estado operativo).

Usabilidad: La capacidad del software para ser entendido, aprendido, operado y ser atractivo para el usuario. Esto implica la capacidad de reconocimiento de la

idoneidad del software, la capacidad de aprendizaje (facilidad para que un usuario alcance un nivel de competencia), la operatividad (facilidad para usar features y controles) y la protección contra errores del usuario.

Mantenibilidad: El grado en que el sistema puede ser modificado de manera efectiva y eficiente sin introducir errores, incluyendo correcciones, mejoras o adaptaciones a cambios en el entorno. Se compone de la modularidad (estructura bien definida), la reutilización (partes que pueden usarse en otros sistemas), la analizabilidad (facilidad para diagnosticar defectos), y la modificabilidad (facilidad para implementar cambios).

Seguridad: La capacidad de proteger la información y los datos para que personas o sistemas no autorizados no puedan acceder, modificar, inspeccionar, o usar la información. Esto incluye la confidencialidad (protección contra la divulgación no autorizada), la integridad (protección contra la modificación no autorizada), la no repudiación (prueba de acciones), la autenticidad (verificación de identidad) y la trazabilidad (registro de acciones).

Compatibilidad: El grado en que el software puede intercambiar información y llevar a cabo sus funciones requeridas mientras comparte el mismo entorno de hardware o software con otros productos. Incluye la coexistencia (capacidad de compartir recursos comunes) y la interoperabilidad (capacidad de interactuar con uno o más sistemas específicos).

Portabilidad: La capacidad del software para ser transferido de manera efectiva y eficiente de un entorno de hardware, software o uso a otro. Sus subcaracterísticas son la adaptabilidad (facilidad para ajustarse a diferentes entornos), la capacidad de instalación (facilidad para instalarse y desinstalarse) y la capacidad de reemplazo (facilidad para que el software reemplace a otro en un entorno dado).

4.2.6 Estrategias y niveles de pruebas de software

1. Pruebas Unitarias (Unit Testing): Se centran en validar la funcionalidad más pequeña y aislada del código (una función, un método, o una clase específica). Su objetivo es asegurar que cada módulo individual opera correctamente y según su diseño, antes de que se integre con otras partes del sistema. Son la primera línea de defensa contra los errores.

2. Pruebas de Integración (Integration Testing): Verifican cómo diferentes módulos, componentes o subsistemas interactúan y se comunican entre sí. El foco no es la funcionalidad interna del módulo (eso ya lo cubren las unitarias), sino la interfaz y el flujo de datos entre ellos. Pueden ser pruebas de "arriba hacia abajo" (top-down) o de "abajo hacia arriba" (bottom-up).

3. Pruebas de Sistema (System Testing): Validan el sistema completo e integrado en un entorno que simula el de producción, comprobando que cumpla con los requisitos funcionales y no funcionales especificados (como rendimiento, seguridad y usabilidad). En este nivel se examina el comportamiento del sistema de principio a fin, incluyendo su interacción con el sistema operativo, hardware y otros sistemas externos.

4. Pruebas de Aceptación (Acceptance Testing - UAT): Es la fase final donde se valida si el sistema está listo para su lanzamiento y si satisface las necesidades del cliente o usuario final desde una perspectiva de negocio. Se verifica si el producto cumple con los criterios de aceptación contractuales. Las Pruebas de Aceptación del Usuario (UAT) son el tipo más común en este nivel.

4.2.7 Ciclo de mejora continua de la calidad (PHVA)

Fase	Acción en Proyectos de Software (Extendida)
Planear (Plan) 🌀	Definir el Plan de Gestión de Calidad y el Plan de Pruebas. Esto incluye establecer los estándares de codificación, las métricas de calidad aceptables (por ejemplo, cobertura de código o densidad de defectos), los criterios de entrada y salida de cada fase, y las herramientas de QA a utilizar.
Hacer (Do) 🏗️	Implementar los procesos definidos y ejecutar las tareas planificadas. En el contexto de QA, esto significa ejecutar el

	desarrollo según los estándares, realizar revisiones de código, y llevar a cabo los diferentes niveles de pruebas (unitarias, integración, sistema) documentando los resultados y los defectos encontrados.
Verificar (Check) 🤔	Analizar los resultados de las actividades de "Hacer" comparándolos con lo definido en "Planear". Se detectan desviaciones entre el rendimiento real y los estándares (por ejemplo, si las métricas de defectos superan el umbral o si un proceso no se siguió correctamente). Esto incluye generar informes de <i>bugs</i> y métricas de progreso.
Actuar (Act) ✅	Corregir las desviaciones encontradas: Se priorizan y resuelven los defectos (<i>bugs</i>) detectados. Se documentan las lecciones aprendidas y se ajustan/mejoran los procesos de desarrollo y QA para el siguiente ciclo o <i>sprint</i> , estandarizando el cambio positivo para evitar que el mismo error se repita.

4.2.8 Herramientas Clave para la Gestión de Calidad (QA)

Categoría	Herramientas Comunes	Función Extendida
Automatización de Pruebas	Selenium, JUnit, PyTest, Postman	Ejecutan y verifican automáticamente los casos de prueba (scripts) para reducir el tiempo de ciclo y aumentar la cobertura. Permiten probar la interfaz de usuario (UI), las API y las unidades de código, asegurando la regresión constante.
Gestión de Pruebas	TestLink, Jira, Zephyr	Centralizan la planificación, el diseño de casos de prueba, la trazabilidad de requisitos, la ejecución de <i>tests</i> y el registro de resultados. Son esenciales para la organización del proceso de Control de Calidad (QC).

Integración Continua (CI/CD)	Jenkins, GitHub Actions,	Permiten la compilación, el testeo y el despliegue automatizado y continuo del código. Cada cambio integrado al repositorio dispara una batería de <i>tests</i> automatizados para detectar errores tempranamente.
Revisión de Código	SonarQube, GitLab CI	Analizan el código fuente sin ejecutarlo (análisis estático) para detectar <i>bugs</i> , vulnerabilidades de seguridad, violaciones de estándares de codificación y <i>code smells</i> (malas prácticas de programación).
Documentación de Calidad	Confluence, Notion	Sirven como repositorio central para la documentación del proyecto, incluyendo planes de calidad, estrategias de prueba, registros de resultados, métricas de calidad (<i>dashboards</i>), y el manual de procedimientos de QA.

4.2.9 Métricas Calidad del Software

Métrica	Qué mide	Ejemplo / Fórmula
Densidad de defectos	Errores por tamaño del código.	Defectos / KLOC (mil líneas).
Tasa de fallos	Errores durante operación.	Fallos / horas de uso.
Cobertura de pruebas	Porcentaje de código probado.	$(\text{Líneas probadas} / \text{totales}) \times 100$
Disponibilidad	Tiempo que el sistema está activo.	$(\text{Tiempo activo} / \text{Total}) \times 100$
Satisfacción del usuario	Opinión del cliente.	Encuestas o NPS.

4.2.10 Roles involucrados

Rol	Responsabilidad
Gerente de proyecto	Asegurar que se cumplan los estándares de calidad.
Líder de QA	Diseñar el plan de calidad y coordinar pruebas.
Equipo de desarrollo	Aplicar buenas prácticas de codificación.
Usuario / Cliente	Validar que el producto cumpla sus necesidades.

4.2.11 Documentos de gestión de la calidad

1. **Plan de calidad del software (SQA Plan)**
 - Define políticas, estándares, métricas y responsabilidades.
2. **Informe de revisión y auditoría**
 - Registra resultados de QA.
3. **Registro de defectos**
 - Monitorea errores detectados y corregidos.
4. **Informe de métricas**
 - Mide rendimiento, fiabilidad y satisfacción.
5. **Lecciones aprendidas / Post-mortem**
 - Documenta oportunidades de mejora.



4.2.12 Enfoque DevOps en la calidad

Práctica DevOps	Funcionalidad
Integración Continua (CI)	Permite la detección temprana de errores y conflictos de integración. Al compilar y probar automáticamente el código tras cada cambio, se asegura que los defectos sean identificados y corregidos cuando son más fáciles y baratos de solucionar.
Entrega Continua (CD)	Asegura que el software esté siempre en un estado funcional y desplegable, reduciendo el riesgo en el

	proceso de liberación. Garantiza que las versiones listas para producción sean estables y que el despliegue sea un proceso robusto y repetible.
Monitoreo Continuo	Permite el seguimiento proactivo del rendimiento, la estabilidad y la tasa de errores de la aplicación en tiempo real en el entorno de producción. Los datos reales informan la calidad, permitiendo detectar problemas antes de que afecten gravemente a los usuarios.
Automatización de Pruebas y Despliegues	Reduce drásticamente los errores humanos y garantiza que las pruebas de calidad (funcionales, de regresión, etc.) se ejecuten de manera consistente y rápida. Esto facilita una mayor cobertura de pruebas y liberaciones más confiables.

Metodologías de Desarrollo de Software (completo)



Una **Metodología de Desarrollo de Software (MDS)** es un **marco de trabajo** utilizado para **estructurar, planificar y controlar el proceso de creación**, implementación y mantenimiento de sistemas de información.

4.3.1 Metodologías Tradicionales.

Se caracterizan por su estructura **rígida y secuencial**, donde se intenta **planificar el proyecto de principio a fin** antes de comenzar la codificación.

>>Modelo de Cascada

Es una **metodología de desarrollo tradicional** que **organiza el trabajo en fases secuenciales**. Cada fase debe **completarse completamente** antes de iniciar la siguiente.

Fases Clave y Su Naturaleza

El flujo lineal de las fases es estricto y unidireccional, como el agua que cae por una cascada:

1. **Análisis (Requisitos):** Se recopilan y documentan **todos los requisitos funcionales y no funcionales**. El entregable final es el *Documento de Especificación de Requisitos (ERS)*.
2. **Diseño:** Se define la **arquitectura del sistema**, el diseño de la base de datos y la interfaz de usuario.
3. **Implementación (Codificación):** Los desarrolladores **escriben el código** basándose en **el diseño**.
4. **Pruebas (Verificación):** Se comprueba que el software cumpla con los requisitos definidos en la Fase 1.
5. **Mantenimiento:** Tras el despliegue, se corrigen errores y se hacen adaptaciones evolutivas.

Ventajas: Claridad y Control

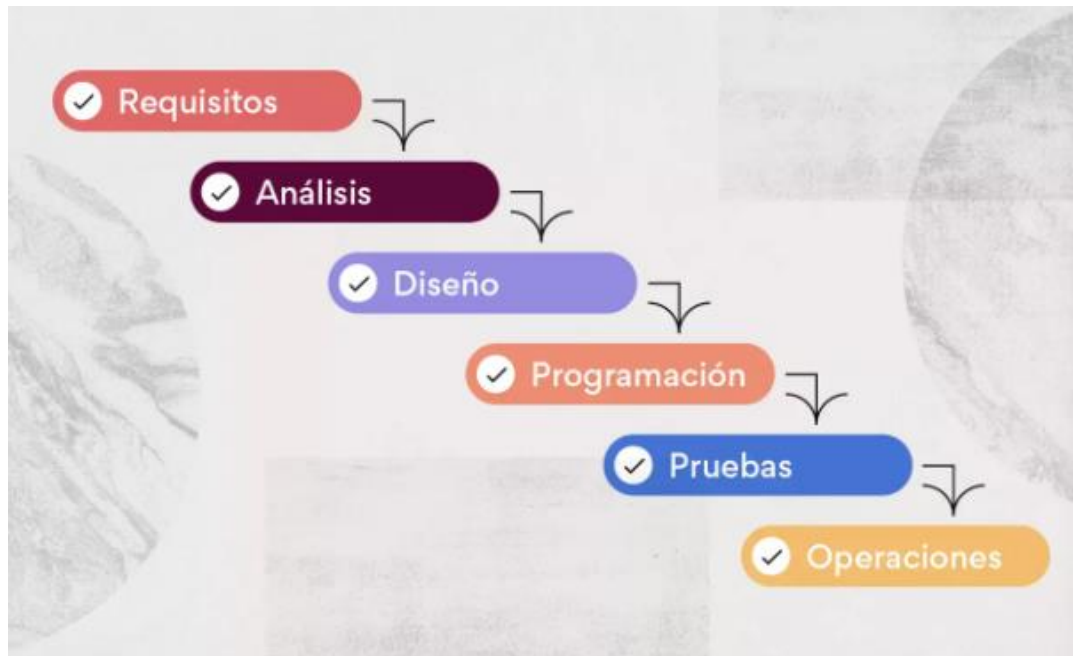
Al requerir la aprobación formal de una fase antes de pasar a la siguiente, **se logra una documentación exhaustiva** y una fácil medición del progreso. Es excelente para la gestión del riesgo y la auditoría.

Desventajas: **La detección de errores ocurre muy tarde** (principalmente en la fase de Pruebas). Un error de interpretación en la fase de Análisis puede ser catastróficamente **caro de corregir al final del ciclo**, lo que lleva a menudo a que el producto final no cumpla con las expectativas del usuario.

Aplicación Ideal

El Modelo en Cascada es más efectivo en proyectos donde:

- **Los requisitos son estables** y es poco probable que cambien (ej., sistemas legados, integraciones sencillas).
- **El dominio es familiar** y el riesgo técnico es bajo (ej., tu ejemplo de **Sistemas administrativos o bancarios estables** con requisitos legales fijos).
- **El contrato es fijo** y la documentación completa es una necesidad regulatoria o contractual.



>> *Modelo en V (Verificación y Validación)*

Imagina el proceso de desarrollo de software como un viaje. El **Modelo en V** propone que por cada paso que das hacia abajo (desarrollando y construyendo el sistema), hay un paso correspondiente hacia arriba (verificando y validando lo que construiste). No pruebas todo al final, sino que preparas y ejecutas pruebas para cada nivel de detalle.

Estructura de la "V"

El modelo se asemeja a la letra V, con las fases de desarrollo en el lado izquierdo y las fases de prueba correspondientes en el lado derecho:

- **Lado Izquierdo (Desarrollo y Diseño - Verificación):** Se enfoca en la descomposición de los requisitos y el diseño de la solución.
 - Requisitos de Usuario
 - Diseño del Sistema
 - Diseño de Arquitectura
 - Diseño del Módulo
- **Lado Derecho (Pruebas - Validación):** Se enfoca en ensamblar y verificar que el producto cumpla con los requisitos definidos en el lado izquierdo.

- Pruebas de Componente (Unidad) → Asociadas al Diseño del Módulo.
- Pruebas de Integración → Asociadas al Diseño de Arquitectura.
- Pruebas de Sistema → Asociadas al Diseño del Sistema.
- Pruebas de Aceptación → Asociadas a los Requisitos de Usuario.

La Clave: Verificación vs. Validación

Las dos mitades del modelo se distinguen por dos conceptos de calidad fundamentales:

Concepto	Definición	Pregunta Clave
Verificación (Lado Izquierdo)	Proceso de evaluar si el producto de una fase de desarrollo cumple con las condiciones impuestas al inicio de esa fase.	"¿Estamos construyendo el producto correctamente?"
Validación (Lado Derecho)	Proceso de evaluar si el software satisface la intención de uso original y los requisitos del cliente.	"¿Estamos construyendo el producto correcto?"



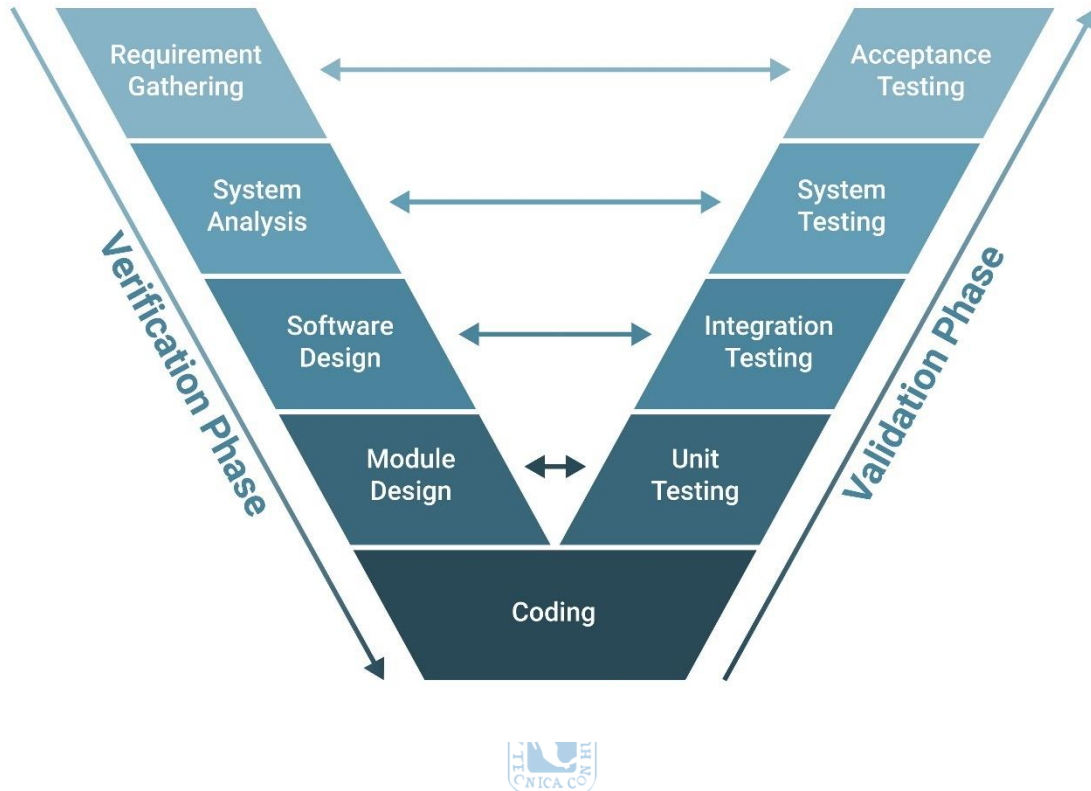
✓ Ventajas

- **Alta calidad:** se detectan errores desde etapas tempranas.
- **Enfoque estructurado en pruebas:** cada fase tiene su validación.
- **Trazabilidad clara:** se puede seguir el rastro desde requisitos hasta pruebas.

✗ Desventajas

- **Poca flexibilidad:** no se adapta bien a cambios durante el desarrollo.
- **Requiere requisitos bien definidos** desde el inicio.
- **No promueve entregas parciales** ni retroalimentación continua.

En resumen, el **Modelo en V** es un modelo de desarrollo disciplinado y en cascada que **hace hincapié en la verificación y validación en cada etapa**, pero su rigidez lo hace vulnerable en entornos donde el cambio es la única constante.



>> *Modelo Espiral*

Combina la **naturaleza secuencial** del Modelo en Cascada con la **entrega iterativa** del desarrollo de prototipos. **Su característica definitoria** es **la inclusión explícita del Análisis de Riesgos** en cada ciclo.

El modelo opera en ciclos repetitivos (espiras). En cada vuelta, el proyecto pasa por cuatro cuadrantes de actividad, y el radio de la espiral representa el **costo acumulado** del proyecto.

1. **Planificación:** Se definen los objetivos de la fase actual, alternativas de implementación y las restricciones.
2. **Análisis de Riesgos:** Es el cuadrante crucial. Se identifican, analizan y mitigan los riesgos potenciales (técnicos, de gestión, etc.). Si los riesgos son demasiado altos, el proyecto puede ser cancelado o redefinido.

3. **Ingeniería (Desarrollo):** Se desarrolla el producto, que puede variar desde un prototipo, una simulación o una versión operativa del software, dependiendo de la fase de la espiral.
4. **Evaluación (Revisión):** Se evalúan los resultados de la fase de ingeniería y se planea la próxima iteración de la espiral.

✓ Ventajas

- **Control de riesgos:** permite anticiparse y responder a problemas antes de que escalen.
- **Flexibilidad:** se adapta a cambios en requisitos o contexto.
- **Iteraciones controladas:** cada ciclo mejora el producto progresivamente.

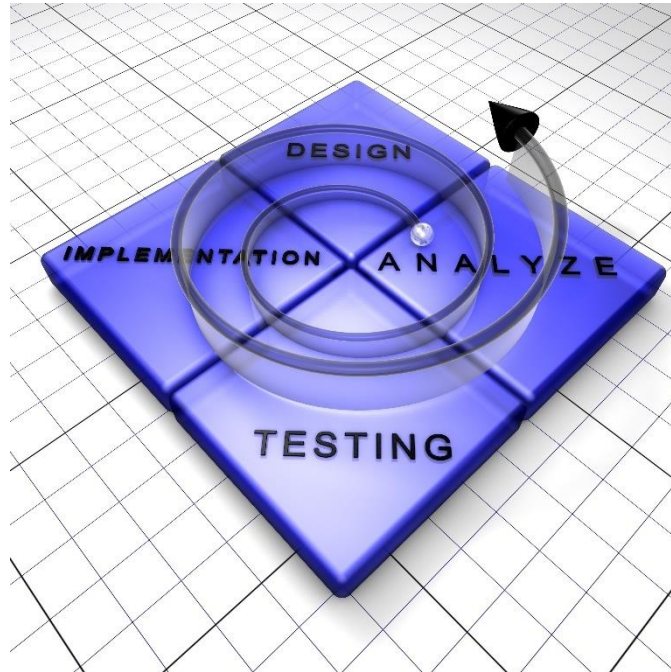
✗ Desventajas

- **Complejidad:** requiere alta planificación y experiencia en gestión de riesgos.
- **Costo elevado:** no es ideal para proyectos pequeños o con recursos limitados.
- **Difícil de aplicar sin participación activa del cliente.**



🧠 Tip para exámenes

“¿Qué ventaja ofrece el modelo espiral frente al modelo en cascada?” ✓ Permite **iterar y controlar riesgos** en cada fase, adaptándose mejor a proyectos complejos.



>>Modelo Incremental

Imagina que estás construyendo un automóvil. En lugar de ensamblar todas las piezas hasta el final para mostrar un coche completo, decides construir y entregar primero el chasis con las ruedas (algo funcional que se puede rodar), luego le añades el motor (ahora puede moverse), después la carrocería (protege los componentes) y finalmente el interior con la tapicería (lo hace cómodo).

Eso es el modelo incremental: **El software se divide en "trozos" o incrementos funcionales. Cada incremento es un producto usable que aporta una funcionalidad específica.** Al final, la suma de todos los incrementos forma el sistema completo.

📊 Características clave

- **Desarrollo por etapas:** se entrega software funcional en cada ciclo.
- **Retroalimentación continua:** el cliente puede probar y sugerir mejoras desde el inicio.
- **Menor riesgo:** los errores se detectan temprano.
- **Alta adaptabilidad:** se pueden ajustar los requisitos entre incrementos.

✅ Ventajas

- Entregas tempranas y funcionales.
- Permite validar con el cliente desde el principio.

- Facilita la planificación y el control del proyecto.
- Reduce el impacto de errores o cambios.

✗ Desventajas

- Requiere una buena **planificación modular**.
- Puede haber **retrabajo** si los primeros incrementos no se diseñan con visión global.
- La integración continua puede ser compleja si no se gestiona bien.

■ Ejemplo típico de aplicación

Desarrollo de una **aplicación web**:

- Incremento 1: login y registro.
- Incremento 2: perfil de usuario.
- Incremento 3: sistema de pagos.
- Incremento 4: panel de administración.

💡 Tip para exámenes

- “¿Qué diferencia al modelo incremental del modelo en cascada?”  El modelo incremental **entrega partes funcionales del software desde el inicio**, mientras que en cascada se entrega todo al final.

>>Modelo de Prototipos

Es una metodología que se basa en **crear versiones preliminares del software** (prototipos) para que el cliente pueda **visualizar, probar y retroalimentar** antes de construir el sistema final.

- ☐ **Recolección inicial de requisitos** Se identifican las necesidades básicas del cliente.
- ☐ **Diseño rápido del prototipo** Se construye una versión simplificada del sistema, enfocada en la interfaz y funciones clave.
- ☐ **Evaluación del prototipo por el cliente** El cliente prueba el prototipo y da retroalimentación.
- ☐ **Refinamiento del prototipo** Se ajusta el prototipo según los comentarios recibidos.

- ❑ **Desarrollo del sistema final** Una vez que el cliente aprueba el prototipo, se construye el software completo.
- ❑ **Pruebas y entrega** Se verifica el sistema final y se entrega al cliente.

✓ Ventajas

- **Alta participación del cliente:** mejora la satisfacción.
- **Reducción de errores:** se detectan desde etapas tempranas.
- **Mejor comprensión de requisitos:** el cliente ve lo que recibirá.

✗ Desventajas

- **Puede generar falsas expectativas** si el prototipo parece más completo de lo que es.
- **No siempre es útil para sistemas complejos** o con lógica interna muy detallada.
- **Incrementa el esfuerzo inicial** si se crean muchos prototipos.

Metodologías ágiles

Las metodologías **ágiles son enfoques de gestión de proyectos que priorizan la flexibilidad, la colaboración y la entrega continua de valor**. Se basan en ciclos iterativos cortos que permiten adaptarse rápidamente a los cambios y mejorar el producto mediante la retroalimentación constante del equipo y del cliente.

SCRUM:

Scrum es una metodología ágil para la gestión de proyectos que busca entregar valor de manera incremental y adaptable, centrada en la colaboración del equipo y la satisfacción del cliente.

Concepto

Scrum es un **marco de trabajo ágil** que permite gestionar proyectos complejos de forma flexible y eficiente. Su enfoque se centra en **incrementos de trabajo cortos y frecuentes**, llamados *sprints*, que permiten evaluar constantemente el progreso y ajustarlo según sea necesario. Esta metodología se utiliza comúnmente en desarrollo de software, pero también es aplicable a otros contextos donde la adaptación rápida es clave.

Características principales

- **Iterativa e incremental:** el trabajo se divide en ciclos cortos (*sprints*), generalmente de 2 a 4 semanas.
- **Roles definidos:** incluye Product Owner (define prioridades y requisitos), Scrum Master (facilita el proceso y elimina obstáculos) y el Equipo de Desarrollo (realiza las tareas).

- **Eventos estructurados:** reuniones como *Planning*, *Daily Scrum*, *Sprint Review* y *Sprint Retrospective* que promueven la transparencia y la mejora continua.
- **Priorización de valor:** se centra en entregar los elementos más importantes del producto primero.
- **Transparencia y comunicación:** el progreso y los impedimentos se visualizan constantemente mediante herramientas como tableros Scrum o burndown charts.

Ventajas

- **Flexibilidad:** permite adaptarse a cambios de requisitos durante el proyecto.
- **Entrega continua de valor:** cada sprint genera un producto funcional o un incremento tangible.
- **Mejora de la comunicación:** fomenta la colaboración y la retroalimentación frecuente entre los miembros del equipo y con el cliente.
- **Mayor enfoque en el cliente:** el Product Owner asegura que el producto cumpla con las necesidades reales.
- **Detección temprana de errores:** problemas y desviaciones se identifican rápidamente gracias a la revisión frecuente de cada sprint.

Desventajas

- **Alcance poco definido:** Cuando las metas del proyecto no están claras, los cambios continuos en los requerimientos pueden hacer que el proyecto se alargue o se desvíe del plan original. Esto afecta recursos y tiempos.
- **Dependencia del equipo:** El rendimiento depende del compromiso de todos los integrantes. La falta de colaboración, desinterés o ausencia de un miembro clave puede retrasar o comprometer entregas importantes.
- **Escalabilidad limitada:** Para proyectos grandes o con varios equipos, coordinar a todos puede volverse complicado y disminuir la efectividad de Scrum.
- **Sobrecarga de reuniones:** Las reuniones frecuentes (diarias, planeación, revisión y retrospectivas) pueden consumir mucho tiempo y afectar la productividad si no se organizan correctamente.
- **Dependencia del Scrum Máster:** Si la persona que ejerce este rol no tiene experiencia, puede haber problemas en la organización, resolución de obstáculos y motivación del equipo, impactando el proyecto.
- **Experiencia requerida en el equipo:** Equipos sin suficiente conocimiento en Scrum pueden tener dificultades para planificar sprints, estimar tareas y priorizar actividades, lo que retrasa el proyecto.

Extreme Programming XP:

Concepto

XP es una metodología ágil de desarrollo de software que enfatiza la **flexibilidad, la comunicación continua con el cliente y la entrega frecuente de versiones funcionales** del producto. Se centra en adaptarse rápidamente a los cambios de requerimientos y en fomentar una cultura de colaboración estrecha entre programadores, clientes y otros miembros del equipo.

Características

1. **Iteraciones cortas:** Se desarrollan ciclos breves de trabajo, generalmente de 1 a 3 semanas, para entregar funcionalidades completas.
2. **Programación en parejas (Pair Programming):** Dos desarrolladores trabajan juntos en una misma tarea, mejorando la calidad y compartiendo conocimiento.
3. **Desarrollo dirigido por pruebas (Test-Driven Development, TDD):** Las pruebas se escriben antes del código para asegurar que las funciones cumplen los requisitos.
4. **Integración continua:** El código se integra frecuentemente en la base de código principal para detectar errores de manera temprana.
5. **Retroalimentación constante:** Uso de reuniones frecuentes con clientes y entre el equipo para ajustar prioridades y resolver problemas.
6. **Simplicidad:** Se desarrolla la solución más simple que funcione, evitando complejidades innecesarias.
7. **Propiedad colectiva del código:** Todos los miembros del equipo pueden modificar cualquier parte del código, fomentando responsabilidad compartida.

Ventajas

- **Adaptabilidad a cambios:** Facilita responder rápidamente a cambios en los requisitos del cliente.
- **Mejora de la calidad del software:** Gracias a TDD, revisiones y pair programming, se reducen errores y defectos.
- **Entrega temprana y frecuente:** Permite obtener feedback del cliente de manera continua, mejorando la satisfacción.
- **Fomento de la colaboración del equipo:** La programación en parejas y la propiedad colectiva incentivan el aprendizaje compartido y el trabajo cooperativo.
- **Reducción de riesgos:** La integración continua y las pruebas frecuentes detectan problemas antes de que se magnifiquen.

Desventajas

- **Requiere alta disciplina del equipo:** El éxito depende de la participación constante y el compromiso de todos los implicados.
- **Escalabilidad limitada:** XP funciona mejor en proyectos medianos o pequeños; en proyectos muy grandes puede ser difícil de aplicar.
- **Dependencia del cliente:** Requiere un involucramiento activo del cliente durante todo el proceso, lo que no siempre es posible.

- **Curva de aprendizaje:** Adoptar prácticas como TDD o pair programming puede ser desafiante para equipos no familiarizados con metodologías ágiles.
- **Costo inicial:** La programación en parejas y pruebas constantes pueden aumentar el esfuerzo inicial comparado con métodos tradicionales.

Kanban:

Concepto:

Desde otra perspectiva, Kanban no solo es un tablero con tarjetas, sino una estrategia de gestión del trabajo orientada al flujo continuo. Se centra en entregar valor de manera constante, visualizar cuellos de botella, y equilibrar la demanda con la capacidad del equipo. En este sentido, Kanban es tanto un sistema para organizar tareas como un marco para la mejora de procesos en cualquier tipo de organización o proyecto. Su esencia radica en que las tareas avanzan gradualmente según la capacidad del equipo, evitando sobrecargas y fomentando la eficiencia operativa

Características principales:

- **Visualización del trabajo:** Permite ver todas las tareas y su estado de manera clara, facilitando la identificación de cuellos de botella
- **Flexibilidad y adaptabilidad:** Puede implementarse sin modificar radicalmente los procesos existentes y ajustarse a distintos sectores, desde tecnología hasta construcción
- **Límites de trabajo en curso (WIP):** Establece un máximo de tareas simultáneas para evitar sobrecarga y mantener la calidad
- **Flujo continuo:** Las tareas avanzan de columna a columna sin necesidad de cumplir plazos rígidos, permitiendo adaptaciones rápidas
- **Mejora continua:** Promueve la evolución constante del proceso mediante retroalimentación y análisis de métricas
- **Transparencia y colaboración:** Todos los miembros del equipo pueden ver el progreso y responsabilidades, favoreciendo comunicación y coordinación

Ventajas de la metodología Kanban

- **Mayor visibilidad y control:** Todas las tareas y prioridades son fácilmente observables
- **Incremento de eficiencia y productividad:** Al limitar el WIP, los equipos se enfocan en terminar tareas antes de iniciar nuevas
- **Facilita la mejora continua y la detección de problemas:** Los cuellos de botella se identifican rápidamente y se pueden corregir
- **Flexibilidad y adaptabilidad:** Funciona para proyectos pequeños y grandes, y puede combinarse con otras metodologías ágiles como Scrum (Scrumban)

- Fomenta la colaboración: La transparencia del tablero mejora la comunicación y la cohesión del equipo

Desventajas de la metodología Kanban:

- **Dependencia excesiva de la visualización;** Kanban se centra en la representación visual de tareas, lo que puede complicar el seguimiento en **equipos remotos o distribuidos** .
- **Dificultad con tareas grandes o indivisibles;** la metodología requiere **dividir tareas en pasos más pequeños**, lo que puede ser ineficiente para proyectos complejos o de gran tamaño .
- **Necesidad de competencias interdisciplinarias;** si el equipo no domina distintas áreas, Kanban puede no explotarse al máximo, reduciendo su efectividad .
- **Falta de planificación temporal detallada;** Kanban no proporciona **cronogramas a largo plazo ni estimaciones precisas**, lo que puede dificultar cumplir plazos estrictos .
- **Riesgo de confusión por exceso de información;** demasiadas tarjetas o etiquetas en el tablero pueden **generar desorganización** si los miembros no están entrenados adecuadamente .



Lean Software Development (LSD):

Concepto

Lean Software Development (LSD) se basa en los principios del sistema de producción de Toyota, aplicados al desarrollo de software . Su objetivo principal es optimizar los procesos de desarrollo y entregar valor al cliente de forma rápida y eficiente. La filosofía del LSD se centra en eliminar actividades que no aportan valor, fomentar la mejora continua, y adaptarse ágilmente a las necesidades del usuario

Características y Principios Fundamentales

- Los principios básicos del Lean Software Development incluyen
- amaliagomez.es
- Eliminar desperdicios: Suprimir todo lo que no genere valor para el cliente, como tareas redundantes, re-trabajo o código defectuoso.
- Amplificar el aprendizaje: Fomentar un ciclo continuo de aprendizaje mediante la experimentación y la retroalimentación para mejorar habilidades y procesos.
- Decidir lo más tarde posible: Posponer decisiones no críticas hasta contar con suficiente información, aumentando la precisión al tomar decisiones.
- Entregar tan rápido como sea posible: Ciclos de desarrollo cortos para obtener feedback temprano y mejorar el software iterativamente.
- Empoderar a los equipos: Dar autonomía a los equipos de desarrollo para tomar decisiones y resolver problemas de forma independiente.

- **Construir integridad intrínseca:** Garantizar un software de alta calidad, fácil de mantener y extensible para reducir errores futuros.
- **Visualizar todo el conjunto:** Mantener una visión global del flujo de trabajo y las dependencias para optimizar los procesos

Ventajas

- **Reducción de costes y tiempos:** Al eliminar desperdicios, se reduce el tiempo de desarrollo y los costos asociados.
- **Mayor satisfacción del cliente:** Entregas rápidas y funcionales permiten adaptar el producto a las necesidades reales del cliente.
- **Mejora de la calidad del software:** La integración de mejora continua y pruebas frecuentes disminuye errores y aumenta la estabilidad.
- **Incremento de la eficiencia del equipo:** Equipos empoderados y motivados trabajan de manera más productiva y colaborativa.
- **Flexibilidad y adaptabilidad:** Permite responder rápidamente a cambios en los requisitos del proyecto, manteniendo el valor entregado al cliente

Desventajas y Desafíos

- **Falta de apoyo de la dirección:** La metodología requiere respaldo de la alta gerencia para ser efectiva.
- **Necesidad de equipo capacitado:** Se requiere personal con habilidades técnicas e innovadoras, así como disposición para aprender continuamente.
- **Enfoque cultural más que en herramientas:** Priorizar software o herramientas sobre la cultura Lean puede reducir la implicación del equipo.
- **Escalabilidad en proyectos grandes:** Integrar Lean en proyectos complejos requiere planificación cuidadosa y coordinación entre múltiples equipos.

Feature-Driven Development (FDD):

Concepto

FDD es un enfoque ágil orientado a **features**, entendidas como pequeñas funcionalidades del sistema que son de valor para el cliente. La metodología busca combinar **prácticas de desarrollo tradicionales** con la **flexibilidad y rapidez de métodos ágiles**, enfatizando el diseño y construcción de cada funcionalidad mediante procesos claros y estructurados. FDD fue introducido por Jeff De Luca y Peter Coad en la década de 1990. Su objetivo principal es mantener un **estándar de calidad alto** y permitir la entrega continua de software funcional.

Características principales

- **Orientación a funcionalidades:** Se centra en desarrollar y entregar funcionalidades completas que sean útiles para el usuario final.
- **Modelo de dominio:** Se crea un modelo que representa el dominio del negocio, facilitando la comprensión del sistema antes del desarrollo de funcionalidades.
- **Desarrollo incremental:** Cada feature se desarrolla de forma independiente y se integra al sistema iterativamente.

- **Roles y responsabilidades claros:** Incluye roles específicos como el jefe de proyecto, el jefe de desarrollo, los diseñadores y los desarrolladores, con responsabilidades bien definidas.
- **Planificación basada en features:** Las actividades se planifican usando una lista priorizada de funcionalidades a entregar, lo que permite un seguimiento preciso del progreso.
- **Revisión frecuente:** Cada funcionalidad se revisa y verifica mediante pruebas para garantizar la calidad desde el inicio.

Ventajas

- **Entrega continua y medible:** Permite evaluar el progreso de manera tangible a través de las funcionalidades completadas.
- **Alta calidad del software:** Al enfocarse en pequeñas funcionalidades y revisarlas continuamente, se reduce la probabilidad de errores.
- **Escalabilidad:** Funciona bien para proyectos grandes, ya que permite estructurar y coordinar equipos utilizando roles claros.
- **Mejor gestión del riesgo:** Los problemas se detectan tempranamente al desarrollar y revisar cada feature.
- **Flexibilidad dentro de la estructura:** Combina la adaptabilidad ágil con procedimientos formales de ingeniería de software.

Desventajas

- **Dependencia del modelado inicial:** Un modelo de dominio incompleto o incorrecto puede afectar el desarrollo de todas las funcionalidades futuras.
- **Menos enfoque en comunicación directa con el cliente:** En comparación con otras metodologías ágiles como Scrum, FDD enfatiza la planificación y modelado más que la interacción continua con el cliente.
- **Implementación rígida de roles:** Equipos pequeños o menos estructurados pueden encontrar difícil cumplir con los roles asignados de manera estricta.

El Dynamic Systems Development Method

Concepto de DSDM

El Dynamic Systems Development Method es un marco de desarrollo ágil diseñado para entregar sistemas que cumplan con las necesidades de negocio dentro de los plazos y presupuestos definidos. Originalmente derivado del Desarrollo Rápido de Aplicaciones (RAD) en los años 90 en el Reino Unido, DSDM promueve la participación activa del usuario, un desarrollo iterativo e incremental y prioriza los requisitos críticos para maximizar el valor entregado al cliente.

Características principales

- **Iterativo e incremental:** El desarrollo se realiza en ciclos cortos, con entregas frecuentes de componentes funcionales para evaluación y ajuste continuo.

- Enfoque en el valor del negocio: Se priorizan las funcionalidades críticas mediante técnicas como MoSCoW (Debe, Debería, Podría, No tendrá) para asegurar que el proyecto entregue el mayor valor posible
- Participación continua del usuario: Los usuarios finales interactúan con los desarrolladores en cada iteración, verificando los prototipos y proporcionando retroalimentación
- Timeboxing: Se asigna un tiempo fijo para cada iteración o fase, controlando plazos sin comprometer la calidad
- Principios clave: Incluyen colaboración, entrega a tiempo, mantener la calidad, construir sobre bases sólidas, comunicación continua y demostración de control
- Fases del ciclo de vida: Pre-proyecto, ciclo de vida del proyecto (viabilidad, estudio de negocio, iteraciones funcionales, diseño y construcción, implementación) y post-proyecto (mantenimiento y mejoras)

Ventajas

- Adaptabilidad: Permite responder rápidamente a cambios en los requisitos y necesidades del cliente
- Transparencia y visibilidad: Entregas frecuentes permiten seguimiento continuo tanto para el equipo como para el cliente
- Colaboración y comunicación: Mejora la interacción entre usuarios, desarrolladores y stakeholders, promoviendo cohesión y empoderamiento de los equipos
- Reducción de riesgos: La priorización y entrega incremental permiten identificar problemas y corregirlos oportunamente

Aunque ágil y flexible, presenta algunas desventajas prácticas como alta dependencia de la interacción, costos en implementación, riesgo de limitar la creatividad y dificultad para proyectos que no sean de software.

Desventajas

- **Alta interacción requerida:** DSDM demanda cooperación constante de todos los participantes, lo que puede ser difícil de mantener en equipos dispersos geográficamente o con recursos limitados .
- **Costos de implementación y capacitación:** La formación del equipo en DSDM puede ser costosa y requerir tiempo adicional, especialmente en pequeñas y medianas empresas .
- **Minimiza la creatividad:** Al enfocarse en entregas rápidas y funcionales, puede limitar la exploración de soluciones innovadoras y riesgos creativos durante las fases iniciales .
- **Exige estructura y disciplina:** Funcionará mejor si los equipos cuentan con liderazgo sólido y roles definidos; sin ello, puede ser difícil coordinar iteraciones y entregables .
- **No adecuado para todos los proyectos:** DSDM está orientado principalmente a desarrollo de software; proyectos críticos de seguridad, componentes altamente reutilizables o no relacionados con TI pueden no beneficiarse de su enfoque iterativo .

Crystal Method:

Concepto

Crystal es una **familia de metodologías ágiles** desarrollada por Alistair Cockburn que se centra en la adaptabilidad a las necesidades del proyecto y del equipo. A diferencia de metodologías rígidas como Waterfall, Crystal reconoce que **cada proyecto es único**, y por eso ofrece distintos enfoques ("cristales") según el tamaño del equipo y la complejidad del trabajo, priorizando la interacción humana, la colaboración y la flexibilidad sobre los procesos estrictos .

Características

- **Centrado en las personas:** Reconoce que el talento, la comunicación y la cooperación del equipo son más importantes que los procesos formales .
- **Adaptabilidad:** Cada proyecto puede definir sus propias prácticas y herramientas según sus necesidades específicas .
- **Entrega frecuente:** Se promueve la entrega incremental de software funcional para recibir retroalimentación temprana y continua .
- **Enfoque ultraligero:** Minimiza la documentación innecesaria, enfocándose en el valor y la calidad del software .
- **Mejora continua:** Se incentiva la reflexión periódica y los ajustes iterativos en el proceso de desarrollo .
- **Clasificación por colores:** Define variantes de Crystal según el tamaño y criticidad del proyecto:
 - *Crystal Clear:* Equipos pequeños (hasta 8 personas), proyectos de baja criticidad.
 - *Crystal Yellow:* Equipos medianos (10–20 personas).
 - *Crystal Orange:* Equipos grandes (25–50 personas).
 - *Crystal Red:* Equipos muy grandes, alta criticidad, más estructurado .

Ventajas

- **Flexibilidad:** Permite adaptarse a cambios sin grandes costos adicionales .
- **Comunicación efectiva:** Fomenta la transparencia y la interacción directa entre miembros del equipo y stakeholders .
- **Realimentación continua:** La entrega frecuente y las iteraciones cortas permiten ajustes tempranos y mejora del producto .
- **Adaptabilidad a cualquier tamaño de equipo:** Las variantes facilitan la escalabilidad según el contexto .

Desventajas

- **Definición de alcance crítica:** Requiere claridad en los objetivos y responsabilidades del proyecto para evitar confusiones .
- **Dependencia de la experiencia del equipo:** Equipos sin experiencia en metodologías ágiles pueden sentirse desorientados .
- **No toda documentación está estandarizada:** La ausencia de procesos rígidos puede complicar la implementación en organizaciones grandes o muy tradicionales .

- **Limitaciones en sectores muy regulados:** Puede ser difícil aplicar Crystal en proyectos con estrictos requerimientos legales o de cumplimiento normativo .

El DevOps (Desarrollo y Operaciones) Metodología Ágil.

- **Dev:** Su meta es **cambiar** el sistema lo más rápido posible para entregar nuevas características.
- **Ops:** Su meta es **mantener** el sistema estable y confiable, por lo que ven los cambios como un riesgo.

DevOps surge para derribar este "muro de la confusión", fomentando una **cultura de colaboración y responsabilidad compartida** donde el objetivo único es entregar software de alta calidad de forma rápida y estable.

Los Tres Pilares Fundamentales de DevOps (La "Trinidad")

1. Cultura (La Parte Más Importante)

Es el cambio de mentalidad. Se promueve:

- **Responsabilidad Compartida:** Los desarrolladores son responsables de cómo su código se comporta en producción, y los de operaciones participan en las etapas de diseño para entender la aplicación.
- **Colaboración y Comunicación Continua:** Se eliminan los silos departamentales. Todos trabajan hacia el mismo objetivo.
- **Fracaso como Oportunidad de Aprendizaje:** En lugar de buscar culpables, se investigan las causas raíz de los incidentes para mejorar el sistema y los procesos.

2. Prácticas (Los Procesos que Automatizan la Cultura)

Estas son las metodologías que hacen tangible la colaboración:

- **Integración Continua (CI):** Los desarrolladores fusionan su código en un repositorio compartido varias veces al día. Cada fusión desencadena una compilación y un conjunto automatizado de pruebas para detectar errores de inmediato.
- **Entrega Continua (CD):** Automatiza el despliegue de cada cambio que pasa las pruebas de CI en un entorno de **pre-producción** (staging). Esto permite liberar cualquier versión en cualquier momento de forma segura.

- **Despliegue Continuo (La evolución de CD):** Va un paso más allá: cada cambio que pasa todas las etapas automatizadas se libera **automáticamente a producción**, sin necesidad de intervención manual. Esto permite múltiples despliegues al día.
- **Infraestructura como Código (IaC):** La infraestructura (servidores, redes, etc.) se define y gestiona usando código y técnicas de desarrollo (como control de versiones). Esto permite crear, replicar y destruir entornos de forma automática, consistente y reproducible. (Herramientas: Terraform, Ansible, CloudFormation).

3. Herramientas (La Capa Tecnológica que Habilita las Prácticas)

Forman la "tubería" o **Pipeline de CI/CD** que automatiza el flujo de trabajo:

- **Gestión de Código: Git** (GitHub, GitLab, Bitbucket).
- **Integración/Entrega Continua: Jenkins, GitLab CI, GitHub Actions, CircleCI.**
- **Contenedores: Docker** empaqueta la aplicación y sus dependencias, garantizando que funcione igual en cualquier entorno.
- **Orquestación de Contenedores: Kubernetes** automatiza el despliegue, escalado y gestión de aplicaciones contenerizadas.
- **Monitorización: Prometheus, Grafana, ELK Stack** para observar el comportamiento de la aplicación en producción.

¿Cuándo es Apropiado Adoptar DevOps?

Es ideal para organizaciones que:

- Buscan una **ventaja competitiva** a través de la agilidad y la innovación.
- Tienen aplicaciones basadas en **microservicios** o arquitecturas cloud-nativas.
- Necesitan una **alta escalabilidad y confiabilidad** en sus sistemas.
- Están dispuestas a **invertir en un cambio cultural** a largo plazo.

En resumen, **DevOps** es una filosofía cultural y operativa respaldada por prácticas y herramientas que automatizan los procesos entre desarrollo y operaciones, con el objetivo de construir, probar y liberar software de forma más rápida, frecuente y confiable. No es un destino, sino un viaje de mejora continua.

Metodologías Ágiles

Metodología	Tipo	Enfoque principal	Ventajas	Cuando usarla
Cascada	Tradicional	Secuencial	Claridad y control	Requisitos fijos
Modelo en V	Tradicional	Verificación y validación	Alta calidad	Sistemas críticos
Espiral	Tradicional	Iterativo con gestión de riesgos	Control y flexibilidad	Proyectos grandes
Prototipos	Tradicional	Validación temprana	Mejor comunicación	Requisitos inciertos
Scrum	Ágil	Iterativo e incremental	Entregas rápidas	Equipos pequeños
XP	Ágil	Calidad del código	Menos errores	Entornos cambiantes
Kanban	Ágil	Flujo continuo	Simplicidad visual	Soporte y mantenimiento
Lean	Ágil	Eliminación de desperdicios	Eficiencia	Procesos repetitivos
FDD	Ágil	Desarrollo por funcionalidades	Modularidad	Grandes equipos
DevOps	Ágil / Híbrido	Automatización e integración continua	Despliegues rápidos	Entornos en la nube
ScrumBan	Híbrido	Scrum + Kanban	Flujo adaptable	Proyectos cambiantes
SAFe / DAD	Híbrido	Escalado de agilidad	Coordinación de equipos	Grandes empresas

Cocomo – cálculo de costos de desarrollo de software

Cocomo es una manera de calcular el costo y personas requeridas para desarrollar un proyecto de software, necesita las variables del tiempo que hay de desarrollo, y un estimado de que tan complejo será el desarrollo del sistema.

Esta metodología tiene una tabla en la que pondera como es que podemos clasificar un proyecto:

Tipo	Ejemplo	a	b	c	d
Orgánico	Pequeño y conocido (sistema administrativo)	2.4	1.05	2.5	0.38
Semi-acoplado	Tamaño medio, moderada complejidad	3.0	1.12	2.5	0.35
Empotrado	Sistemas críticos o tiempo real	3.6	1.20	2.5	0.32

Estas son sus principales formulas:

fórmulas principales

Esfuerzo (persona-mes)

$$E = a \times (KLOC)^b$$

Duración (meses)

$$D = c \times (E)^d$$

Tamaño del equipo

$$\text{Personal promedio} = \frac{E}{D}$$

Ahora hay otros tipos de estimaciones económicas de un proyecto como la estimación heurística.

La estimación heurística, tiene como objetivo brindar una respuesta a un proyecto en una edad muy temprana, que sea muy novedoso y no se tenga conocimiento de todo el proyecto; por así decirlo es el método que uno dice cuando está en aprietos.



Este método tiene varios enfoques desde preguntarle a expertos cuanto es que han cobrado por el desarrollo de un sistema similar, hasta por experiencias previas, que tu has tenido y sabes mas o menos cuanto puede rondar, es un método cualitativo, necesitas saber el contexto del proyecto para saber mas o menos que respuesta dar.

Este método es un primer paso para después pasar a cualquier método cuantitativo que suelen ser más exactos.

ISO – Estandars de calidad.

ISO 12207 “Ciclo de vida del software”

Define los procesos, actividades y tareas para el desarrollo, operación, mantenimiento y retiro del software.

Es la base de todos los modelos de proceso modernos (RUP, CMMI, ISO 15504).

Se estructura en procesos primarios (desarrollo, mantenimiento), de soporte (verificación, validación, documentación, QA) y organizativos (gestión y mejora continua).

Aporta trazabilidad y consistencia en todas las fases del ciclo de vida.

Se dice que esta iso es la madre de la ingeniería de software.

ISO 15504 SPICE (Internacional)

Evalúa la madurez de los procesos de software y define un marco para su mejora continua.

Tiene 5 niveles de madurez:

1. Incompleto
2. Realizado
3. Establecido
4. Gestionado
5. Optimizado

Es el precursor directo del modelo CMMI.(que es un marco en el cual se evalua la madurez de un proyecto.)

ISO 25010 (SquaRe)

Funcionalidad: Cumple con el objetivo

Fiabilidad: rendimiento. (Es rápido, no falla, funciona sobre los parámetros esperados.)

Usabilidad: UX/IX

UX – Enfocada a la facilidad de uso de la herramienta

IX – que tan intuitiva y organizada es de que color, posición, etc.

Eficiencia: Rapidez y consume pocos recursos

Mantenibilidad: Plan de mantenimiento correctivo preventivo, predictivo y perfectivo.

Portabilidad: Que pueda usarse en diferentes dispositivos

Seguridad: Encriptación, CID, ley de protección de datos

Compatibilidad: Recepción o interacción con sistemas externos.

Modelo internacional de evaluación de calidad del software.

Define 8 atributos:

Atributo	Descripción
----------	-------------

Funcionalidad	Cumple el objetivo especificado.
---------------	----------------------------------

Fiabilidad	Es estable, no falla y responde según lo esperado.
------------	--

Usabilidad	Facilidad de uso (UX) e interfaz intuitiva (IX).
------------	--

Eficiencia	Rápido y con bajo consumo de recursos.
------------	--

Mantenibilidad	Soporte a mantenimiento correctivo, preventivo, adaptativo y perfectivo.
----------------	--

Portabilidad	Puede ejecutarse en distintos entornos.
--------------	---

Seguridad	Protege la información (encriptación, control de acceso, confidencialidad).
-----------	---

Compatibilidad	Se integra con otros sistemas o servicios.
----------------	--



ISO 29148

Validación de requerimientos funcionales y no funcionales. 3er fase de el ciclo de vida del software

Va enfocada a reducir ambigüedades.

Usuario – Requerimientos que vienen de una petición del usuario

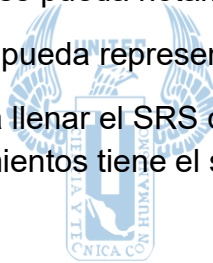
Sistema – Requerimientos mínimos que satisfacen las peticiones del cliente, no funcionales

Software – Requerimientos funcionales, procedimientos tareas y procesos.

Hay 5 atributos:

1. Es necesario?
2. Esta correcto?
3. Requerimiento completo, un principio y fin.
4. Que sea verificable, o que se pueda notar.
5. Que sea trazable o que se pueda representar en un diagrama.

Suele ser la base para empezar a llenar el SRS que es documento formal en donde se especifica que requerimientos tiene el sistema.



ISO 25002

(SQUARE)

Evaluación de calidad del sistema (KPIS)

Los KPIS son indicadores de información específica, por ejemplo métricas de flujo de clientes, número de descargas, etc.

Se crea una rubrica de evaluación, sobre las características que debería de tener el sistema. O que tan bien recibido es el sistema por el usuario. Generar KPIS para la evaluación de las características de un producto de software.

ISO 20000 Gestión de servicios

Se encarga de garantizar que cualquier servicio este siempre funcionando, ya sea infraestructura o software.

Entrega – servicios (ERP)

Gestión - Permisos

Continuidad – Siempre activo, siempre accesible, siempre activo.

Disponibilidad - desde cualquier lugar.

ISO 27001 Seguridad de la información

Protección de información.

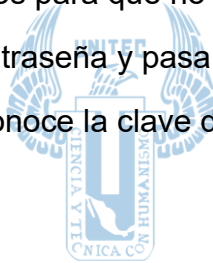
Gestión de accesos/usuario (Privilegio de la mínima información).

Encriptación: maquiación de datos para que no sean entendibles.

Público – el usuario define su contraseña y pasa por un proceso de transformación

Privado – solo el administrador conoce la clave de encriptación.

Leyes gubernamentales



ISO 37301 Compliance

No es exclusiva de TI, pero si se puede aplicar la misma

Mejores prácticas

Seguridad e higiene

Evitar infracciones

Minimizar riesgos.

Ley federal del trabajo

Compliance

es el conjunto de políticas, procedimientos y buenas prácticas que una organización implementa para asegurar que sus operaciones se ajustan a las leyes, regulaciones, normas y estándares éticos aplicables, con el objetivo de prevenir riesgos legales y reputacionales, y así proteger la viabilidad del negocio.

ISO/IEC 9001 – Gestión de calidad

Aplica a cualquier organización, y en software garantiza que los procesos de desarrollo estén documentados, controlados y mejorados continuamente.

Requiere auditorías, control documental y enfoque en satisfacción del cliente.

Ejemplo: una empresa de software que busca certificarse ISO 9001 debe tener procesos medibles de desarrollo, pruebas y mantenimiento.



Anexos

Consejos

El **EGEL Plus ISOFT** no solo evalúa conocimientos teóricos, sino tu capacidad para aplicarlos en escenarios reales (casos prácticos).

A continuación, presento los temas por área que debes dominar:

1. Iniciación de Proyectos

En esta fase, el examen se enfoca en la gestión y la viabilidad.

- **Metodologías:** Diferencias críticas entre Cascada (predicativo) y Ágiles (adaptativo). De Scrum, apréndete de memoria los roles (PO, SM, Dev Team) y los eventos (Sprint Planning, Daily, Review, Retrospective).
- **Estimación:** Entiende cuándo usar Puntos de Función (basado en funcionalidad) vs. COCOMO (basado en líneas de código).
- **Riesgos:** Identifica las 4 estrategias: Mitigar (reducir impacto), Evitar (cambiar el plan), Transferir (seguros/outourcing) y Aceptar (asumir el riesgo).
- **Factibilidad:** Técnica (¿podemos?), Económica (¿conviene?) y Operativa (¿se usará?).

2. Análisis y Diseño de Software

Es el área con más carga visual. Debes saber interpretar diagramas.

- **UML:** Los más preguntados son el **Diagrama de Clases** (relaciones de herencia, composición y agregación), el **Diagrama de Secuencia** (interacción temporal) y el de **Casos de Uso** (límites del sistema).
- **Patrones de Diseño:**
 - *Creacionales:* Singleton, Factory.
 - *Estructurales:* Facade, Adapter, Proxy.
 - *Comportamiento:* Observer, Strategy.
- **Arquitectura:** MVC (Modelo-Vista-Controlador) es básico. También estudia Microservicios vs. Monolito y Arquitectura en Capas.
- **Base de Datos:** Normalización (1FN, 2FN, 3FN) y propiedades ACID.

3. Desarrollo de Software

Aquí se evalúa la lógica de programación y estándares de calidad del código.

- **Estructuras de Datos:** Cuándo usar un Array, una Lista Ligada, una Pila (LIFO), una Cola (FIFO) o un Árbol Binario.
- **Complejidad Algorítmica:** Entender la notación Big O ($O(1)$, $O(n)$, $O(n^2)$).
- **SOLID:** Especialmente el principio de Responsabilidad Única y el de Abierto/Cerrado.
- **Herramientas:** Control de versiones (Git: merge, pull, push, branch) y pruebas unitarias (TDD).

4. Implantación y Mantenimiento

Se enfoca en lo que pasa después de "terminar" de programar.

- **Tipos de Pruebas:**
 - *Caja Negra:* Basadas en requerimientos (sin ver el código).
 - *Caja Blanca:* Basadas en la lógica interna (cobertura de rutas).
 - *Regresión:* Para asegurar que lo nuevo no rompió lo viejo.
- **Mantenimiento:** Correctivo (errores), Adaptativo (cambio de entorno), Perfectivo (mejoras) y Preventivo (evitar fallos futuros).
- **SQA (Software Quality Assurance):** Modelos como CMMI (niveles de madurez) y normas ISO (como la 25000 para calidad de producto).

Consejo Final para el Examen

Cuando leas una pregunta, identifica primero **en qué área estás**. Si la pregunta menciona "cliente" y "prioridad", piensa en **Iniciación**. Si menciona "herencia" o "objetos", piensa en **Diseño** o **Desarrollo**.

Para cerrar con éxito tu preparación, aquí tienes un set de consejos estratégicos diseñados específicamente para el ambiente y la estructura del examen EGEL Plus ISOFT.



Durante el Examen: Gestión del Tiempo

La Regla de los 2 Minutos: Si lees una pregunta (especialmente de código o cálculo de redes) y no sabes por dónde empezar, márcala y sigue adelante. El examen es largo y no quieres llegar al final (donde hay preguntas fáciles de Gestión) sin tiempo por haberte trabado en un diagrama de clases complejo.

Divide y Vencerás: El examen se divide en dos sesiones. No intentes repasar todo en el descanso. Úsalo para despejar la mente, hidratarte y comer algo ligero.

Cuidado con las "Negativas": Lee bien si la pregunta dice "¿Cuál de estos NO es..." o "Todos son correctos EXCEPTO...". Es el error más común por leer rápido.



Estrategias de Respuesta (Técnicas de Descarte)

Descarte de Extremos: En preguntas de cálculos (como puntos de función o PERT), si tienes opciones como: A) 5, B) 45, C) 50, D) 500; usualmente la A y la D son distractores extremos. Enfócate en las que tienen valores cercanos.

La Respuesta Más Completa: En preguntas teóricas de ética o gestión, la opción que suena más profesional, integral y apegada a estándares internacionales suele ser la correcta.

Contexto CENEVAL: El examen evalúa "el deber ser" según los libros. Aunque en tu trabajo real no usen diagramas de secuencia, para el examen, UML es la ley. Responde siempre basándote en la teoría estándar (IEEE, ISO, SWEBOK).



Consejos por Área Específica

Área 1 y 2 (Análisis y Diseño)

Visualiza el flujo: Si te dan un Caso de Uso, identifica rápidamente quién es el Actor y cuál es el Objetivo.

Relaciones UML: No confundas Agregación (rombo vacío, vida independiente) con Composición (rombo relleno, vida dependiente). Es una pregunta clásica.

Área 3 (Desarrollo)

Sé el compilador: En los ejercicios de código, fíjate primero en los puntos y coma y en las llaves. A veces la respuesta es "Error de sintaxis" y te ahorras resolver toda la lógica.

Cajas de Prueba: Recuerda: Caja Blanca = ves el código (lógica, rutas); Caja Negra = no ves el código (entradas y salidas).

Área 4 (Gestión)

Prioridades Scrum: El Product Owner siempre decide el "QUÉ" (negocio), el Equipo decide el "CÓMO" (técnico) y el Scrum Master cuida el proceso.

Riesgos: Si la pregunta menciona "eliminar la causa", la respuesta es Evitar. Si dice "reducir la probabilidad", es Mitigar.

Preparación Física y Mental

Material Permitido: Asegúrate de llevar tu pase de examen, identificación oficial, lápices del 2 o 2.5, goma y una calculadora básica (no programable).

Lectura Crítica: No asumas información que no está en el texto de la pregunta. Si el caso no dice que el cliente tiene prisa, no elijas una opción basada en "terminar rápido".

Confía en tu preparación: Has repasado las 75 preguntas clave y las fórmulas. El examen Plus busca evaluar tu criterio como profesional, no solo tu memoria.

Fórmulas Críticas para el Examen EGEL Plus

1. Estimación de Tiempo (PERT)

Se utiliza para obtener un promedio ponderado de la duración de una tarea cuando hay incertidumbre.

- **Fórmula:**

$$T_e = \frac{O + 4M + P}{6}$$

- **O (Optimista):** Tiempo si todo sale perfecto.
- **M (Más probable):** Tiempo normal.
- **P (Pesimista):** Tiempo si todo sale mal.

Ejemplo: Si una tarea tarda 2 días (O), 5 días (M) y 14 días (P), el tiempo esperado es:
 $(2 + 20 + 14) / 6 = 6$ días.

2. Gestión del Valor Ganado (EVM)

Estas fórmulas te dicen si tu proyecto va por buen camino en cuanto a **costo** y **tiempo**.

Métrica	Fórmula	Interpretación
Variación de Costo (CV)	$CV = EV - AC$	> 0 : Bajo presupuesto. < 0 : Sobre presupuesto.
Variación de Tiempo (SV)	$SV = EV - PV$	> 0 : Adelantado. < 0 : Retrasado.
Índice de Desempeño de Costo (CPI)	$CPI = \frac{EV}{AC}$	> 1 : Eficiente. < 1 : Ineficiente.
Índice de Desempeño de Tiempo (SPI)	$SPI = \frac{EV}{PV}$	> 1 : Rápido. < 1 : Lento.

- **EV (Value Ganado):** Valor del trabajo realmente realizado.
- **AC (Costo Real):** Dinero gastado hasta el momento.
- **PV (Valor Planeado):** Valor del trabajo que debería estar hecho según el plan.

3. Complejidad Ciclomática (Métrica de McCabe)

Sirve para medir qué tan complejo es un fragmento de código (Área 3). Indica el número de caminos independientes.

- **Fórmula basada en regiones:**

$$V(G) = P + 1$$

- **P:** Número de puntos de decisión (if, while, for, switch).

- **Fórmula basada en nodos y aristas:**

$$V(G) = E - N + 2$$

- **E:** Número de aristas (líneas que conectan).
 - **N:** Número de nodos (cajas de proceso).
-

4. Análisis de Puntos de Función (FP)

Se usa para medir el "tamaño" del software basado en funcionalidad.

- **Fórmula:**

$$UFP = \sum (\text{Componentes}) \times (\text{Peso})$$

- **Componentes a contar:** Entradas Externas (EI), Salidas Externas (EO), Consultas Externas (EQ), Archivos Lógicos Internos (ILF) y Archivos de Interfaz Externa (EIF).

Tip: En el examen, si te dan una tabla con estos valores, solo multiplica la cantidad de cada uno por su peso (Bajo, Medio o Alto) y súmalos todos.

5. Teorema de Brooks (Gestión)

Más que una fórmula, es una ley de gestión:

"Añadir personal a un proyecto de software retrasado, lo retrasa más".

- **Razón:** El esfuerzo de comunicación crece exponencialmente:

$$n(n - 1) / 2$$

, donde n es el número de personas.

PREGUNTAS PARA FORTALECER LOS CONOCIMIENTOS DEL MODULO 1

1. Un analista está entrevistando a los stakeholders para un nuevo sistema de inventarios. Un gerente menciona que "el sistema debe generar reportes mensuales de existencias". ¿A qué tipo de requerimiento pertenece esta afirmación?

- ☐ A) Requerimiento no funcional
- ☐ B) Requerimiento funcional
- ☐ C) Requerimiento de interfaz
- ☐ D) Requerimiento de dominio

2. Durante la fase de elicitación, el cliente indica que el sistema debe ser capaz de procesar 1,000 transacciones por segundo. ¿Cómo se clasifica este requerimiento?

- ☐ A) Disponibilidad
- ☐ B) Usabilidad
- ☐ C) Rendimiento
- ☐ D) Seguridad



3. En un Diagrama de Casos de Uso, ¿qué relación se utiliza para representar un comportamiento opcional que solo ocurre bajo ciertas condiciones?

- ☐ A) <<include>>
- ☐ B) <<extend>>
- ☐ C) Generalización
- ☐ D) Asociación simple

4. Se requiere modelar la lógica de un proceso de negocio complejo que incluye decisiones, ramificaciones y flujos paralelos. ¿Qué diagrama de UML es el más adecuado?

- ☐ A) Diagrama de Clases
- ☐ B) Diagrama de Actividades
- ☐ C) Diagrama de Componentes
- ☐ D) Diagrama de Despliegue

5. ¿Cuál es el objetivo principal de realizar un análisis de factibilidad técnica?

- ☐ A) Determinar si el proyecto generará ganancias económicas.
- ☐ B) Evaluar si la tecnología actual y el equipo pueden desarrollar el sistema.
- ☐ C) Identificar los riesgos legales del software.
- ☐ D) Definir el cronograma de actividades.

6. En el modelado orientado a objetos, ¿qué concepto permite que una clase hija herede atributos y métodos de una clase padre?

- ☐ A) Encapsulamiento
- ☐ B) Polimorfismo
- ☐ C) Herencia
- ☐ D) Abstracción

7. ¿Qué técnica de recolección de requerimientos es más efectiva cuando los usuarios finales tienen dificultades para expresar sus necesidades de forma verbal?

- ☐ A) Entrevistas
- ☐ B) Cuestionarios
- ☐ C) Prototipado
- ☐ D) Lluvia de ideas

8. Un requerimiento dice: "El sistema debe estar disponible el 99.9% del tiempo". ¿A qué atributo de calidad se refiere?

- ☐ A) Mantenibilidad
- ☐ B) Disponibilidad
- ☐ C) Escalabilidad
- ☐ D) Portabilidad

9. En un Diagrama de Clases, ¿qué símbolo representa una relación de composición (donde la parte no puede existir sin el todo)?

- ☐ A) Un rombo vacío
- ☐ B) Un rombo relleno
- ☐ C) Una flecha punteada
- ☐ D) Una línea sólida sin flecha

10. Al documentar un Caso de Uso, ¿qué sección describe el estado del sistema antes de que el caso de uso inicie?

- ☐ A) Postcondición
- ☐ B) Flujo alterno
- ☐ C) Precondición
- ☐ D) Escenario principal

11. ¿Qué herramienta se utiliza para rastrear un requerimiento desde su origen hasta su implementación y prueba?

- ☐ A) Diccionario de datos
- ☐ B) Matriz de trazabilidad de requerimientos
- ☐ C) Diagrama de Gantt
- ☐ D) Modelo Entidad-Relación



12. ¿Cuál es la principal diferencia entre un requerimiento funcional y uno no funcional?

- ☐ A) El funcional es obligatorio y el no funcional es opcional.
- ☐ B) El funcional describe "qué" hace el sistema y el no funcional "cómo" lo hace (restricciones).
- ☐ C) El funcional es para el programador y el no funcional para el cliente.
- ☐ D) No hay diferencia real en la práctica.

13. En UML, ¿qué diagrama describe la estructura estática de un sistema mostrando sus atributos y operaciones?

- ☐ A) Diagrama de Secuencia
- ☐ B) Diagrama de Clases
- ☐ C) Diagrama de Estados
- ☐ D) Diagrama de Comunicación



14. ¿Qué tipo de ambigüedad ocurre cuando un requerimiento puede interpretarse de varias formas por diferentes lectores?

- ☐ A) Ambigüedad técnica
- ☐ B) Ambigüedad semántica
- ☐ C) Inconsistencia
- ☐ D) Omisión

15. Usted está modelando el ciclo de vida de un objeto "Pedido" (Creado, Pagado, Enviado). ¿Qué diagrama de UML debe utilizar?

- ☐ A) Diagrama de Objetos
- ☐ B) Diagrama de Máquina de Estados
- ☐ C) Diagrama de Paquetes
- ☐ D) Diagrama de Robustez

16. ¿Qué técnica de validación de requerimientos consiste en una revisión formal por un grupo de expertos y stakeholders?

- ☐ A) Inspección
- ☐ B) Prototipado rápido
- ☐ C) Pruebas de unidad
- ☐ D) Simulación

17. En el estándar IEEE 830 para la especificación de requerimientos de software (SRS), ¿cuál de las siguientes es una característica de un buen requerimiento?

- ☐ A) Complejo
- ☐ B) Verificable
- ☐ C) Dependiente del código
- ☐ D) Ambiguo



18. ¿Qué tipo de actor en un diagrama de casos de uso representa un sistema externo que recibe datos del sistema que se está diseñando?

- ☐ A) Actor principal
- ☐ B) Actor secundario
- ☐ C) Actor pasivo
- ☐ D) Stakeholder interno

19. Un sistema debe ser capaz de ejecutarse tanto en Windows como en Linux. ¿A qué requerimiento no funcional corresponde?

- ☐ A) Interoperabilidad
- ☐ B) Portabilidad
- ☐ C) Fiabilidad
- ☐ D) Eficiencia

20. En un Diagrama de Secuencia, ¿qué representan las líneas verticales punteadas?

- ☐ A) Mensajes entre objetos
- ☐ B) La línea de vida de un objeto
- ☐ C) El tiempo de ejecución de un método
- ☐ D) La destrucción del objeto

21. ¿Cuál es el propósito del análisis de dominio?

- ☐ A) Encontrar errores en el código fuente.
- ☐ B) Entender el entorno y el contexto del negocio donde operará el software.
- ☐ C) Definir la base de datos física.
- ☐ D) Estimar el costo del hardware.

22. ¿Qué es una "Historia de Usuario" en metodologías ágiles?

- ☐ A) Un documento técnico de 50 páginas.
- ☐ B) Una descripción breve de una funcionalidad desde la perspectiva del usuario.
- ☐ C) Un diagrama de flujo de datos.
- ☐ D) Una instrucción de base de datos.

23. El principio de "Ocultamiento de información" está directamente relacionado con:

- ☐ A) Polimorfismo
- ☐ B) Encapsulamiento
- ☐ C) Herencia múltiple
- ☐ D) Cohesión temporal

24. Si un requerimiento dice: "Los colores de la interfaz deben ser agradables", ¿por qué se considera un mal requerimiento?

- ☐ A) Porque es muy largo.
- ☐ B) Porque no es funcional.
- ☐ C) Porque es subjetivo y no verificable.
- ☐ D) Porque no especifica el lenguaje de programación.

25. En el contexto de análisis, ¿qué significa que un requerimiento sea "Priorizado"?

- ☐ A) Que se implementará al final.
 - ☐ B) Que tiene un nivel de importancia asignado para decidir qué construir primero.
 - ☐ C) Que es el requerimiento más difícil.
 - ☐ D) Que ya fue aprobado por el departamento legal.
-

Respuestas (Clave para tu guía)

1-B, 2-C, 3-B, 4-B, 5-B, 6-C, 7-C, 8-B, 9-B, 10-C, 11-B, 12-B, 13-B, 14-B, 15-B, 16-A, 17-B, 18-B, 19-B, 20-B, 21-B, 22-B, 23-B, 24-C, 25-B.



PREGUNTAS PARA FORTALECER LOS CONOCIMIENTOS DEL MODULO 2

1. Un arquitecto de software decide separar la lógica de presentación de la lógica de negocio y el acceso a datos en un sistema web. ¿Qué patrón arquitectónico está aplicando?
 - ☐ A) Cliente-Servidor
 - ☐ B) Modelo-Vista-Controlador (MVC)
 - ☐ C) Microservicios
 - ☐ D) Pipe and Filter

2. ¿Cuál es el objetivo principal de buscar una "Alta Cohesión" en el diseño de módulos?
 - ☐ A) Que el módulo dependa de la mayor cantidad de clases posibles.
 - ☐ B) Que las responsabilidades dentro de un mismo módulo estén estrechamente relacionadas.
 - ☐ C) Que el módulo sea difícil de modificar.
 - ☐ D) Reducir la cantidad de memoria que utiliza el módulo.

3. Usted necesita crear un objeto que tenga una única instancia en toda la aplicación (por ejemplo, un gestor de conexiones a BD). ¿Qué patrón de diseño de GoF (Gang of Four) debe usar?
 - ☐ A) Factory Method
 - ☐ B) Singleton
 - ☐ C) Observer
 - ☐ D) Adapter



4. ¿A qué concepto se refiere el "Bajo Acoplamiento" en el diseño de software?

- ☐ A) Aumentar la interdependencia entre módulos.
- ☐ B) Minimizar las dependencias y el conocimiento que un módulo tiene de otro.
- ☐ C) Agrupar todas las funciones en una sola clase.
- ☐ D) Eliminar el uso de interfaces.

5. En el diseño de bases de datos, ¿qué proceso se realiza para eliminar redundancias y evitar anomalías de inserción, actualización y borrado?

- ☐ A) Indexación
- ☐ B) Normalización
- ☐ C) Desnormalización
- ☐ D) Replicación



6. ¿Qué patrón de diseño estructural permite que dos interfaces incompatibles trabajen juntas traduciendo las llamadas de una a la otra?

- ☐ A) Proxy
- ☐ B) Facade (Fachada)
- ☐ C) Adapter (Adaptador)
- ☐ D) Decorator

7. Se requiere diseñar un sistema donde un objeto (Sujeto) notifique automáticamente a otros objetos (Observadores) cuando cambie su estado. ¿Qué patrón es el más adecuado?

- ☐ A) Strategy
- ☐ B) Observer
- ☐ C) Command
- ☐ D) Iterator

8. En una arquitectura de Microservicios, ¿cómo se comunican típicamente los servicios para mantener la independencia?

- ☐ A) Compartiendo la misma base de datos.
- ☐ B) A través de protocolos ligeros como HTTP/REST o colas de mensajes.
- ☐ C) Mediante llamadas directas a memoria.
- ☐ D) No se comunican entre sí.

9. ¿Qué principio de diseño SOLID establece que "una clase debe tener una, y solo una, razón para cambiar"?

- ☐ A) Principio de Abierto/Cerrado (OCP)
- ☐ B) Principio de Responsabilidad Única (SRP)
- ☐ C) Principio de Inversión de Dependencias (DIP)
- ☐ D) Principio de Segregación de Interfaces (ISP)

10. Al diseñar la persistencia de una aplicación, se define una "Llave Foránea" (Foreign Key). ¿Cuál es su función principal?

- ☐ A) Identificar de forma única cada registro de una tabla.
- ☐ B) Establecer una relación y asegurar la integridad referencial entre dos tablas.
- ☐ C) Acelerar las consultas de búsqueda.
- ☐ D) Cifrar los datos sensibles.

11. ¿Qué vista de la arquitectura de software (según el modelo 4+1) se enfoca en la organización de los módulos de software en el entorno de desarrollo?

- ☐ A) Vista Lógica
- ☐ B) Vista de Proceso
- ☐ C) Vista de Implementación (o de Desarrollo)
- ☐ D) Vista de Despliegue (Física)

12. ¿Qué patrón de diseño se utiliza para definir una familia de algoritmos, encapsular cada uno y hacerlos intercambiables en tiempo de ejecución?

- ☐ A) State
- ☐ B) Strategy
- ☐ C) Template Method
- ☐ D) Bridge

13. En el diseño de interfaces de usuario (UI), ¿qué principio busca que los elementos similares tengan comportamientos y apariencias similares en todo el sistema?

- ☐ A) Visibilidad
- ☐ B) Consistencia
- ☐ C) Retroalimentación
- ☐ D) Simplicidad



14. Un sistema requiere que los datos pasen a través de una serie de componentes de procesamiento de forma secuencial. ¿Qué estilo arquitectónico es el más representativo?

- ☐ A) Capas (Layered)
- ☐ B) Filtros y tuberías (Pipes and Filters)
- ☐ C) Basado en eventos
- ☐ D) Peer-to-Peer

15. ¿Qué tipo de diagrama UML se utiliza en la fase de diseño para mostrar la distribución física de los nodos de hardware y los componentes de software que residen en ellos?

- ☐ A) Diagrama de Paquetes
- ☐ B) Diagrama de Despliegue
- ☐ C) Diagrama de Componentes
- ☐ D) Diagrama de Objetos

16. ¿Cuál es la función del patrón "Facade" (Fachada)?

- ☐ A) Proporcionar una interfaz simplificada a un conjunto complejo de clases o subsistema.
- ☐ B) Permitir añadir funcionalidades a un objeto dinámicamente.
- ☐ C) Clonar objetos existentes sin depender de sus clases.
- ☐ D) Separar la abstracción de su implementación.

17. ¿Qué nivel de normalización de bases de datos requiere que no existan dependencias transitivas (que todos los atributos dependan solo de la llave primaria)?

- ☐ A) Primera Forma Normal (1FN)
- ☐ B) Segunda Forma Normal (2FN)
- ☐ C) Tercera Forma Normal (3FN)
- ☐ D) Forma Normal de Boyce-Codd

18. En el diseño orientado a objetos, ¿qué es una interfaz?

- ☐ A) Una clase que implementa todos sus métodos.
- ☐ B) Un contrato que define qué métodos debe tener una clase, pero no cómo los implementa.
- ☐ C) Una base de datos temporal.
- ☐ D) Un tipo de variable global.

19. ¿Qué técnica de diseño permite manejar errores y situaciones excepcionales sin interrumpir abruptamente el flujo del programa?

- ☐ A) Refactorización
- ☐ B) Manejo de excepciones (Try-Catch)
- ☐ C) Recursividad
- ☐ D) Casting de tipos

20. El patrón "Factory Method" pertenece a la categoría de patrones de:

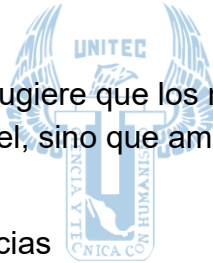
- ☐ A) Comportamiento
- ☐ B) Estructurales
- ☐ C) Creacionales
- ☐ D) Arquitectónicos

21. ¿Qué principio de diseño sugiere que los módulos de alto nivel no deben depender de módulos de bajo nivel, sino que ambos deben depender de abstracciones?

- ☐ A) Inyección de Dependencias
- ☐ B) Inversión de Dependencias (DIP)
- ☐ C) Segregación de Interfaces
- ☐ D) Encapsulamiento de datos

22. En una arquitectura de N-Capas, ¿cuál es la responsabilidad típica de la "Capa de Acceso a Datos" (DAL)?

- ☐ A) Mostrar gráficos al usuario.
- ☐ B) Validar las reglas de negocio complejas.
- ☐ C) Ejecutar consultas SQL y mapear resultados a objetos.
- ☐ D) Gestionar las sesiones de usuario en el navegador.



23. ¿Cuál es el beneficio principal de utilizar un Patrón de Diseño probado?
- ☐ A) Hace que el código sea más corto.
 - ☐ B) Ofrece una solución estandarizada y reutilizable a un problema común.
 - ☐ C) Evita que el programa necesite pruebas.
 - ☐ D) Reduce el costo del hardware.
24. En el diseño de APIs REST, ¿qué método HTTP se utiliza generalmente para actualizar un recurso existente de forma total?
- ☐ A) GET
 - ☐ B) POST
 - ☐ C) PUT
 - ☐ D) DELETE
25. Al diseñar un sistema distribuido, el teorema CAP establece que es imposible garantizar simultáneamente más de dos de las siguientes tres propiedades. ¿Cuáles son?
- ☐ A) Consistencia, Disponibilidad, Tolerancia al particionamiento.
 - ☐ B) Cohesión, Acoplamiento, Portabilidad.
 - ☐ C) Capacidad, Agilidad, Performance.
 - ☐ D) Concurrencia, Almacenamiento, Privacidad.

Respuestas (Clave para Área 2)

1-B, 2-B, 3-B, 4-B, 5-B, 6-C, 7-B, 8-B, 9-B, 10-B, 11-C, 12-B, 13-B, 14-B, 15-B, 16-A, 17-C, 18-B, 19-B, 20-C, 21-B, 22-C, 23-B, 24-C, 25-A.

PREGUNTAS PARA FORTALECER LOS CONOCIMIENTOS DEL MODULO 3

1. Un equipo de desarrollo está utilizando la metodología Test-Driven Development (TDD). ¿Cuál es el orden correcto de los pasos a seguir para implementar una nueva funcionalidad?
 - ☐ A) Escribir código -> Refactorizar -> Escribir prueba.
 - ☐ B) Escribir prueba -> Ejecutar y fallar -> Escribir código -> Ejecutar y pasar -> Refactorizar.
 - ☐ C) Diseñar interfaz -> Escribir código -> Realizar pruebas de integración.
 - ☐ D) Escribir prueba -> Escribir código -> Documentar -> Refactorizar.

 2. ¿Qué tipo de prueba de software se realiza para verificar que los componentes individuales de código (como funciones o métodos) funcionan correctamente de forma aislada?
 - ☐ A) Pruebas de Sistema
 - ☐ B) Pruebas de Integración
 - ☐ C) Pruebas Unitarias
 - ☐ D) Pruebas de Aceptación
- 
3. Usted está revisando un código y encuentra una función que tiene una complejidad ciclomática de 25. ¿Qué indica este valor según las buenas prácticas de ingeniería de software?
 - ☐ A) Que el código es altamente eficiente y rápido.
 - ☐ B) Que el código es difícil de probar y mantener debido a un exceso de caminos lógicos.
 - ☐ C) Que el código no tiene errores de sintaxis.
 - ☐ D) Que la función es pequeña y fácil de entender.

4. En el control de versiones con Git, ¿qué comando se utiliza para integrar los cambios de una rama (branch) secundaria en la rama principal?

- ☐ A) git push
- ☐ B) git commit
- ☐ C) git merge
- ☐ D) git checkout

5. ¿Qué técnica de pruebas de "Caja Blanca" asegura que cada instrucción del programa se haya ejecutado al menos una vez?

- ☐ A) Cobertura de sentencias (Statement Coverage)
- ☐ B) Cobertura de condiciones
- ☐ C) Partición de equivalencia
- ☐ D) Análisis de valores frontera

6. Un desarrollador desea mejorar la estructura interna de un código existente sin alterar su comportamiento externo. ¿Cómo se le denomina a este proceso?

- ☐ A) Reingeniería
- ☐ B) Refactorización (Refactoring)
- ☐ C) Ingeniería Inversa
- ☐ D) Debugging

7. ¿Cuál de las siguientes es una característica fundamental de las metodologías ágiles como Scrum en el desarrollo de software?

- ☐ A) Documentación exhaustiva antes de codificar.
- ☐ B) Entregas de software funcional en iteraciones cortas.
- ☐ C) Seguimiento estricto de un plan lineal de cascada.
- ☐ D) Ausencia total de pruebas durante la codificación.



8. En el manejo de excepciones, ¿cuál es el propósito del bloque finally?
- ☐ A) Capturar un error específico.
 - ☐ B) Ejecutar código obligatoriamente, ocurra o no una excepción (ej. cerrar un archivo).
 - ☐ C) Lanzar una nueva excepción personalizada.
 - ☐ D) Evitar que el programa se compile si hay errores.
9. ¿Qué herramienta se utiliza para analizar el código fuente sin ejecutarlo, con el fin de encontrar posibles errores, vulnerabilidades o incumplimiento de estándares?
- ☐ A) Debugger
 - ☐ B) Analizador estático de código (ej. SonarQube)
 - ☐ C) Profiler
 - ☐ D) Framework de pruebas unitarias
10. Se están realizando pruebas de "Caja Negra" para un campo de edad que acepta valores de 18 a 65 años. ¿Qué valores elegiría para una técnica de "Análisis de Valores Frontera"?
- ☐ A) 10, 40, 80
 - ☐ B) 17, 18, 65, 66
 - ☐ C) 20, 30, 40, 50
 - ☐ D) 18, 19, 20, 21
11. ¿Qué concepto de la Programación Orientada a Objetos permite que una sola interfaz represente diferentes formas subyacentes (tipos de datos)?
- ☐ A) Encapsulamiento
 - ☐ B) Herencia
 - ☐ C) Polimorfismo
 - ☐ D) Abstracción

12. ¿Cuál es la función de un sistema de Integración Continua (CI)?
- ☐ A) Generar el manual de usuario automáticamente.
 - ☐ B) Automatizar la compilación y ejecución de pruebas cada vez que se sube código al repositorio.
 - ☐ C) Reemplazar a los programadores en la escritura de código.
 - ☐ D) Vender el software a los clientes finales.
13. En Git, ¿qué acción se debe realizar para guardar los cambios locales en el repositorio local con un mensaje descriptivo?
- ☐ A) git pull
 - ☐ B) git fetch
 - ☐ C) git commit
 - ☐ D) git add
14. ¿Qué tipo de prueba se enfoca en verificar que el sistema puede manejar una carga de trabajo específica o volúmenes de datos masivos?
- ☐ A) Prueba de Humo
 - ☐ B) Prueba de Rendimiento (Performance Testing)
 - ☐ C) Prueba Funcional
 - ☐ D) Prueba de Regresión
15. ¿Qué es un "Code Smell" (Olor de código)?
- ☐ A) Un error de sintaxis que impide la compilación.
 - ☐ B) Un síntoma en el código fuente que indica un posible problema de diseño o mala práctica.
 - ☐ C) Un virus informático.
 - ☐ D) Una técnica de cifrado de datos.



16. Durante el desarrollo, un programador utiliza una "Librería". ¿Cuál es la principal diferencia entre una Librería y un Framework?

- ☐ A) Las librerías son gratis y los frameworks no.
- ☐ B) En la librería el código del programador tiene el control; en el framework, el framework controla el flujo (Inversión de Control).
- ☐ C) Las librerías solo sirven para bases de datos.
- ☐ D) No existe diferencia, son sinónimos.

17. ¿Qué estándar de codificación recomienda que los nombres de las variables sean descriptivos (ej. impuestoVenta en lugar de iv)?

- ☐ A) Notación Húngara
- ☐ B) Código Limpio (Clean Code)
- ☐ C) Código Ofuscado
- ☐ D) Compilación en JIT



18. ¿Cuál es el propósito de las "Pruebas de Regresión"?

- ☐ A) Probar el sistema por primera vez.
- ☐ B) Asegurar que los nuevos cambios o correcciones no hayan roto funcionalidades que antes funcionaban.
- ☐ C) Evaluar la velocidad de la base de datos.
- ☐ D) Verificar que el cliente esté satisfecho con el diseño.

19. En el desarrollo web, ¿qué significa que una API sea RESTful?

- ☐ A) Que solo usa bases de datos relacionales.
- ☐ B) Que utiliza los métodos HTTP (GET, POST, PUT, DELETE) de forma estandarizada para manipular recursos.
- ☐ C) Que el servidor guarda el estado de cada cliente (Stateful).
- ☐ D) Que solo puede devolver datos en formato XML.

20. ¿Qué es la "Deuda Técnica"?
- ☐ A) El dinero invertido en servidores.
 - ☐ B) El costo de mantenimiento debido a decisiones de desarrollo rápidas y sucias en lugar de usar mejores prácticas.
 - ☐ C) El tiempo que se tarda en compilar un proyecto grande.
 - ☐ D) Un préstamo bancario para startups de software.
21. ¿Qué tipo de prueba de software es realizada por los usuarios finales en un entorno controlado antes del lanzamiento oficial?
- ☐ A) Prueba Alfa
 - ☐ B) Prueba Beta
 - ☐ C) Prueba de Componente
 - ☐ D) Prueba de Estrés
22. ¿Qué principio establece que una función o método debe realizar una sola tarea y hacerla bien?
- ☐ A) Principio de Responsabilidad Única (SRP)
 - ☐ B) Principio de Abierto/Cerrado
 - ☐ C) Inyección de Dependencias
 - ☐ D) Ley de Demeter
23. ¿Para qué sirve el archivo .gitignore en un proyecto de software?
- ☐ A) Para guardar las contraseñas del sistema.
 - ☐ B) Para especificar qué archivos o carpetas no deben ser rastreados por el sistema de control de versiones.
 - ☐ C) Para listar las dependencias del proyecto.
 - ☐ D) Para configurar el servidor de producción.



24. ¿Cuál es el objetivo de un "Code Review" (Revisión de Código)?
- ☐ A) Criticar el trabajo de los compañeros de forma negativa.
 - ☐ B) Identificar errores, mejorar la calidad del código y compartir conocimiento entre el equipo.
 - ☐ C) Contar cuántas líneas de código escribió cada programador.
 - ☐ D) Decidir quién recibirá un aumento de sueldo.
25. En el ciclo de vida del software, ¿en qué fase se suelen crear los ejecutables y se preparan los ambientes de staging?
- ☐ A) Fase de Análisis
 - ☐ B) Fase de Diseño
 - ☐ C) Fase de Construcción / Implementación
 - ☐ D) Fase de Mantenimiento

Respuestas (Clave para Área 3)

1-B, 2-C, 3-B, 4-C, 5-A, 6-B, 7-B, 8-B, 9-B, 10-B, 11-C, 12-B, 13-C, 14-B, 15-B, 16-B, 17-B, 18-B, 19-B, 20-B, 21-A, 22-A, 23-B, 24-B, 25-C.



PREGUNTAS PARA FORTALECER LOS CONOCIMIENTOS DEL MODULO 4

1. En la gestión de proyectos, ¿cuál es el documento formal que otorga al director del proyecto la autoridad para utilizar los recursos de la organización y define el inicio oficial del proyecto?
 - ☐ A) Plan de Gestión de Riesgos
 - ☐ B) Acta de Constitución del Proyecto (Project Charter)
 - ☐ C) Especificación de Requerimientos
 - ☐ D) Enunciado del Alcance

2. Un equipo Scrum está trabajando en un Sprint de 4 semanas. ¿Cuál es la duración máxima recomendada para la reunión de Daily Scrum?
 - ☐ A) 5 minutos
 - ☐ B) 15 minutos
 - ☐ C) 30 minutos
 - ☐ D) 1 hora

3. ¿Qué técnica de estimación utiliza valores de "Pesimista", "Optimista" y "Más Probable" para calcular la duración esperada de una actividad?
 - ☐ A) Estimación por Analogía
 - ☐ B) Estimación de Tres Puntos (PERT)
 - ☐ C) Juicio de Expertos
 - ☐ D) Puntos de Función



4. Usted identifica un riesgo que tiene una probabilidad baja de ocurrir pero un impacto muy alto. Decide comprar un seguro para cubrir los daños en caso de que ocurra. ¿Qué estrategia de respuesta al riesgo está aplicando?

- ☐ A) Mitigar
- ☐ B) Evitar
- ☐ C) Transferir
- ☐ D) Aceptar

5. ¿Qué herramienta gráfica se utiliza para mostrar la relación entre las actividades del proyecto, sus dependencias y el camino crítico a lo largo del tiempo?

- ☐ A) Diagrama de Pareto
- ☐ B) Diagrama de Gantt
- ☐ C) Diagrama de Dispersión
- ☐ D) Mapa de Empatía



6. En el modelo CMMI, ¿en qué nivel de madurez los procesos se miden y controlan mediante métricas estadísticas y cuantitativas?

- ☐ A) Nivel 2: Gestionado
- ☐ B) Nivel 3: Definido
- ☐ C) Nivel 4: Gestionado Cuantitativamente
- ☐ D) Nivel 5: Optimizado

7. ¿Cuál es la principal responsabilidad del Scrum Master dentro de un equipo ágil?

- ☐ A) Definir las prioridades del negocio.
- ☐ B) Escribir el código de las funcionalidades más difíciles.
- ☐ C) Eliminar impedimentos y asegurar que se sigan los valores y prácticas de Scrum.
- ☐ D) Aprobar el presupuesto del proyecto.

8. ¿Qué métrica de tamaño de software se basa en la funcionalidad proporcionada al usuario (entradas, salidas, consultas, archivos lógicos) en lugar de líneas de código?

- ☐ A) KLOC (Miles de líneas de código)
- ☐ B) Puntos de Función
- ☐ C) Complejidad Ciclomática
- ☐ D) Cohesión y Acoplamiento

9. Durante el monitoreo y control, se observa que el proyecto tiene un SPI (Índice de Desempeño del Cronograma) de 0.8. ¿Qué significa este valor?

- ☐ A) El proyecto va adelantado respecto al cronograma.
- ☐ B) El proyecto va retrasado respecto al cronograma.
- ☐ C) El proyecto está exactamente a tiempo.
- ☐ D) El proyecto está gastando menos dinero de lo planeado.

10. ¿A qué se refiere el término "Corrupción del Alcance" (Scope Creep)?

- ☐ A) A la eliminación de funciones para terminar antes.
- ☐ B) Al crecimiento descontrolado del alcance del proyecto sin ajustes en tiempo, costo y recursos.
- ☐ C) Al proceso de cerrar un proyecto prematuramente.
- ☐ D) Al uso de software pirata en el desarrollo.



11. En un diagrama de red, ¿qué es la "Holgura" (Float/Slack) de una actividad?

- ☐ A) El tiempo que una actividad puede retrasarse sin afectar la fecha de finalización del proyecto.
- ☐ B) El costo adicional de una actividad.
- ☐ C) La duración total de la actividad más el tiempo de descanso.
- ☐ D) El número de personas asignadas a una tarea.

12. ¿Cuál es el objetivo de una "Retrospectiva del Sprint"?

- ☐ A) Mostrar el software funcionando al cliente.
- ☐ B) Que el equipo analice qué se hizo bien y qué se puede mejorar en el proceso para el siguiente Sprint.
- ☐ C) Definir qué tareas se harán en el próximo mes.
- ☐ D) Actualizar el presupuesto del proyecto.

13. Según la norma ISO/IEC 25010, ¿qué característica de calidad se refiere a la capacidad del software para proteger información y datos?

- ☐ A) Mantenibilidad
- ☐ B) Usabilidad
- ☐ C) Seguridad
- ☐ D) Portabilidad

14. Usted debe seleccionar un modelo de ciclo de vida para un proyecto donde los requerimientos son muy vagos y se espera que cambien frecuentemente.

¿Cuál es la mejor opción?

- ☐ A) Cascada (Waterfall)
- ☐ B) Modelo Incremental o Ágil
- ☐ C) Modelo en V
- ☐ D) Modelo de Prototipado Desechable

15. ¿Qué técnica de compresión del cronograma consiste en realizar actividades en paralelo que normalmente se harían en secuencia?

- ☐ A) Crashing (Intensificación)
- ☐ B) Fast Tracking (Ejecución rápida)
- ☐ C) Nivelación de recursos
- ☐ D) Outsource

16. En la gestión de la calidad, ¿cuál es la diferencia entre Aseguramiento de Calidad (QA) y Control de Calidad (QC)?

- ☐ A) Son exactamente lo mismo.
- ☐ B) QA se enfoca en los procesos; QC se enfoca en el producto final (detección de defectos).
- ☐ C) QA es para el cliente y QC es para los programadores.
- ☐ D) QA se hace al final del proyecto y QC al principio.

17. ¿Qué es el "Product Backlog" en Scrum?

- ☐ A) Una lista priorizada de todo lo que podría ser necesario en el producto.
- ☐ B) El diagrama de clases del sistema.
- ☐ C) El manual técnico de instalación.
- ☐ D) La lista de errores reportados por el cliente.

18. ¿Cuál es la función principal de la Gestión de la Configuración de Software (SCM)?

- ☐ A) Configurar los servidores de la empresa.
- ☐ B) Identificar, controlar y rastrear los cambios en los elementos de software (código, documentos, datos) a lo largo del tiempo.
- ☐ C) Comprar hardware nuevo cada año.
- ☐ D) Contratar personal calificado.

19. En la matriz de asignación de responsabilidades (RACI), ¿qué significa la letra "A"?

- ☐ A) Autor del código (Author)
- ☐ B) Responsable de la rendición de cuentas (Accountable / El que aprueba)
- ☐ C) Asistente (Assistant)
- ☐ D) Analista (Analyst)

20. ¿Qué es un "Stakeholder" (Interesado)?

- ☐ A) Solo los programadores del proyecto.
- ☐ B) Cualquier persona u organización que pueda afectar o ser afectada por el proyecto.
- ☐ C) Únicamente el dueño de la empresa.
- ☐ D) Los competidores del mercado.

21. ¿Qué modelo de estimación de costos de software utiliza fórmulas matemáticas basadas en el tamaño del software y otros multiplicadores de esfuerzo (cost drivers)?

- ☐ A) Delphi
- ☐ B) COCOMO (Constructive Cost Model)
- ☐ C) Planificación de Poker
- ☐ D) Top-Down

22. ¿Cuál es el propósito de realizar una "Estructura de Desglose del Trabajo" (EDT/WBS)?

- ☐ A) Asignar salarios a los empleados.
- ☐ B) Descomponer el proyecto en componentes más pequeños y manejables para facilitar la planificación.
- ☐ C) Escribir el código fuente del sistema.
- ☐ D) Definir los colores de la interfaz de usuario.



23. ¿Qué principio de la agilidad establece que "el software funcionando es la medida principal de progreso"?

- ☐ A) Manifiesto Ágil
- ☐ B) ITIL
- ☐ C) PMBOK
- ☐ D) ISO 9000

24. En un proyecto, si el costo real (AC) es mayor al valor ganado (EV), se puede concluir que el proyecto está:

- ☐ A) Por debajo del presupuesto.
- ☐ B) Por encima del presupuesto (gastando más de lo planeado).
- ☐ C) Justo en el presupuesto.
- ☐ D) No se puede determinar con esos datos.

25. Al finalizar un proyecto, ¿cuál es la actividad clave para mejorar el desempeño de la organización en proyectos futuros?

- ☐ A) Borrar todos los archivos temporales.
- ☐ B) Realizar una sesión de lecciones aprendidas y documentarlas.
- ☐ C) Despedir al equipo de desarrollo.
- ☐ D) Cambiar el nombre del software.

Respuestas (Clave para Área 4)

1-B, 2-B, 3-B, 4-C, 5-B, 6-C, 7-C, 8-B, 9-B, 10-B, 11-A, 12-B, 13-C, 14-B, 15-B, 16-B, 17-A, 18-B, 19-B, 20-B, 21-B, 22-B, 23-A, 24-B, 25-B.

Elaborada

ALUMNOS

ACEVES ARIAS CRISTIAN ERNESTO
GASPAR HERNANDEZ CARLOS EDUARDO
JACOBO GARCIA AARON
RAMIREZ AGUILAR ERICK JOVANNY
TORRES SANCHEZ BRANDON ALEXIS

PROFESORES

FLORES JIMENEZ VICTOR HUGO
GALICIA ORTIZ ANA DEYCY
HERNANDEZ PRIETO JESUS MANUEL
VEGA RUIZ ESEQUIEL

DIRECTOR ACADEMICO

LUJAN SALAZAR ULISES IVAN



Recursos Digitales

<https://egeluv.dgaeuv.com/2025/1/GUIAS-EXAMEN/Economico/GUIA-EGEL-ISOFT.pdf>

<https://guias-egel.ceneval.edu.mx/login.php?src=isoft>

<https://chatgpt.com/>

<https://gemini.google.com>



Bibliografía

Área 1. Análisis de sistemas de software

- Booch, G., Rumbaugh, J., & Jacobson, I. (2005). The unified modeling language user guide (2a ed.). Addison-Wesley Professional.
- Davis, A. M. (2013). Just enough requirements management: Where software development meets marketing. Dorset House Publishing.
- Gómez, J. (2014). Guía práctica de estimación y medición de proyectos Software: ¿Por qué?, ¿Para qué? y ¿Cómo? Autor.
- Hatton, S. (2008). "Choosing the right Prioritisation method. Proceedings of the Australian Software Engineering Conference", 19th Australian Software Engineering Conference, ASWEC 2008, pp. 517-526.
<http://doi.ieeecomputersociety.org/10.1109/ASWEC.2008.22>
- Kerzner, H. (2022). Project management: A systems approach to planning, scheduling, and controlling (13a ed.). Wiley.
- McConnell, S. (s.f.). Code complete checklist (2a ed.) [versión Adobe PDF].
https://www.matthewjmillar.net/files/cc2e_checklists.pdf
- Patton, J., & Economy, P. (2014). User story mapping: Discover the whole story, build the right product. O'Reilly.
- Pressman, R. S., & Maxim, B. (2020). Software engineering: A practitioner's approach (9a ed.). McGraw Hill.
- Rohrer, C. (2022). "When to use which user-experience research methods". Nielsen Norman Group, disponible en: <https://www.nngroup.com/articles/which-ux-research-methods/> [consulta: junio de 2024].
- Sommerville, I. (2015). Software engineering (10a ed.). Pearson.
- SysML (s.f.). "SysML Open Source Project: What is SysML? Who created SysML?", SysML, disponible en: <https://sysml.org/Visual> [consulta: junio de 2024].

- TenStep Latinoamérica (2009). "Técnicas de estimación". Blog de Proyectum, disponible en: <https://www.proyectum.com/sistema/blog/tecnicas-de-estimacion/> [consulta: junio de 2024].
- Usman, M., Mendes, E., & Börstler, J. (2015). "Effort estimation in agile software development: A survey on the state of the practice". EASE '15: Proceedings of the 19th International Conference on Evaluation and Assessment in Software Engineering, pp. 1-10. ACM Digital Library. <https://doi.org/10.1145/2745802.2745813> Visual-Paradigm (s.f.). "What is user story?"
- Visual-Paradigm, disponible en: <https://www.visual-paradigm.com/guide/agile-software-development/what-is-user-story/> [consulta: junio de 2024].
- Wiegers, K., & Hokanson, C. (2023). Software requirements. Core Practices for Successful Business Analysis. Addison-Wesley.
- Wong, S. (2017). Análisis y requerimientos de software. Universidad Continental. https://repositorio.continental.edu.pe/bitstream/20.500.12394/4281/1/DO_FIN_103_MAI_UC0939_2018.pdf

Área 2. Diseño de sistemas de software

- Blokdyk, G. (2018). Architecture tradeoff analysis method: A clear and concise reference. Createspace Independent Publishing Platform.
- Brown, S. (2020). Software architecture for developers: A developer-friendly guide to software architecture, technical leadership and the balance with agility (2a ed.). Autor.
- Cervantes, H., Velasco-Elizondo, P., & Castro, L. (2016). Arquitectura de software, conceptos y ciclo de desarrollo. Cengage Learning.
- Clements, P., Bachmann, F., Bass, L., Garlan, D., Ivers, J., Little, R., Merson, P., Nord, R., & Stafford, J. (2010). Documenting software architectures, views and beyond (2a ed.). Addison Wesley.
- Date, C. J. (2006). Introducción a los sistemas de bases de datos (7a ed.). Pearson Educación; Prentice Hall.
- Folmer, E., Gorp, J.V., & Bosch, J. (2003). Scenario-based Assessment of Software Architecture Usability. ICSE Workshop on SE-HCI.
https://www.researchgate.net/publication/2940293_Scenario-based_Assessment_of_Software_Architecture_Usability
- Greenberg, S., Carpendale, S., Marquardt, N., & Buxton, B. (2011). Sketching user experiences: The Workbook. Morgan- Kaufmann.
- Harrington, J. L. (2016). Relational database design and implementation: Clearly explained. Morgan Kaufmann Publishers.
- Ingeno, J. (2018). Software architect's handbook. Packt Publishing.
- Jacko, J. (Ed.) (2012). Human computer interaction handbook: Fundamentals, evolving technologies, and emerging applications (3a ed.). CRS Press.
- Jacobson, I., Spencer, I. & Bittner, K. (2011). USE-CASE 2.0. The guide to succeeding with use cases. Autor.
- Joyanes, L. & Zahonero, I. (2014). Programación en C, C++, Java y UML. McGraw Hill.
- Kruchten, P. (1995). Architectural Blueprints. The "4+1" view model of software architecture. IEEE software (12) 6, pp 42-50.
- McCloskey, M. (2014). "Turn user goals into task scenarios for usability testing". Nielsen-Norman Group, disponible en: <https://www.nngroup.com/articles/task-scenarios-usability-testing/> [consulta: junio de 2024].
- Nielsen, J. (2024). "10 usability heuristics for user interface design". Nielsen-Norman Group, disponible en: <https://www.nngroup.com/articles/ten-usability-heuristics/> [consulta: junio de 2024].

- Pascual, J. R. (2019). “Acoplamiento y cohesión. Disrupción tecnológica”. Blog Disrupción Tecnológica, disponible en: <https://www.disrupciontecnologica.com/acoplamiento-y-cohesion/> [consulta: junio de 2024].
- Pernice, K. (2016). “UX prototypes: Low fidelity vs. high fidelity”. Nielsen-Norman Group, disponible en: <https://www.nngroup.com/articles/ux-prototype-hi-lo-fidelity/> [consulta: junio de 2024].
- Suárez, J., & Gutiérrez, L. E. (2016). Tipificación de dominios de requerimientos para la aplicación de patrones arquitectónicos. Información Tecnológica, 27(4), pp.193-202. <https://dx.doi.org/10.4067/S0718-07642016000400021>.
- Sweeting, C. (2017). “How utterances & slot samples affect intent-matching in Alexa skills”. Medium, disponible en: <https://medium.com/voiceflow/how-utterances-slot-samples-affectintent-matching-in-alexa-skills-dcb9b5f7a9ae> [consulta: junio de 2024].
- Ullman, J., Alatorre, E., & Widom, J. (2016). Introducción a los sistemas de bases de datos. Prentice Ha



Área 3. Desarrollo de sistemas de software

- Bağ, A. (2020). "PHP vs. Javascript: A short comparison". Scand, disponible en: <https://scand.com/company/blog/php-vs-javascript-difference-between/#:~:text=Just%20like%20Javascript%2C%20PHP%20is,and%20executed%20on%20the%20server.> [consulta: junio de 2024].
- Banahan, M., Brady, D., & Doran, M. (2010). The C Book [versión eBook] https://publications.gbdirect.co.uk/c_book/
- Beaulieu, A. (2020). Learning SQL: Generate, manipulate and retrieve Data. O'Reilly.
- Cane, A. (2019). Programación para principiantes: Guía comprensiva para principiantes aprenda a programar paso a paso de la A a la Z. Autor.
- Date, C. J. (2006). Introducción a los sistemas de bases de datos (7a ed.). Pearson Educación; Prentice Hall.
- Dean, J. S., & Dean, R. H. (2009). Introducción a la programación con Java. McGraw Hill. EcuRed (s.f.). "Lenguaje de programación Haskell".
- EcuRed, disponible en: https://www.ecured.cu/Lenguaje_de_programaci%C3%B3n_Haskell [consulta: junio de 2024].
- Fernández, P., Arce, J., Pérez, R., & Díaz, C. (2018). Introducción a la Lógica de Programación [versión Adobe PDF]. <http://sara.uniquindio.edu.co/wp-content/uploads/2018/07/Introducci%C3%B3n-a-la-L%C3%B3gica-de-Programaci%C3%B3n.pdf>
- Gabbrielli, M., & Martini, S. (2010). Programming Languages: Principles and Paradigms [versión Adobe PDF] <http://160592857366.free.fr/joe/ebooks/ShareData/Programming%20Languages%20-%20Principles%20and%20Paradigms.pdf>
- Juganaru, M. (2014). Introducción a la programación. [versión eBook] https://www.academia.edu/31289773/Introducci%C3%B3n_a_la_programaci%C3%B3n
- Laudon, J., & Laudon, K. (2016). Sistemas de información gerencial: Administración de la empresa digital (14ª ed.). Pearson.
- Loy, M., Niemeyer, P., & Leuck, D. (2023). Learning Java (5a ed). O'Reilly Media.
- Martí-Oliet, N., Ortega, Y., & Verdejo, J. A. (2013). Estructuras de datos y métodos algorítmicos: 213 ejercicios resueltos (2ª ed.). Garceta.
- Meier, A., & Kaufmann, M. (2019). SQL & NoSQL databases: Models, languages, consistency options and architectures for big data management. Springer Vieweg.

- O'Hanlon, P. (2019). Advanced TypeScript Programming Projects: Build 9 different apps with TypeScript 3 and JavaScript frameworks such as Angular, React, and Vue. Packt Publishing.
- Paul, F. (2022). "Python vs Other Programming Languages in 2024: Detailed Comparison". Uvik, disponible en: <https://uvik.net/blog/how-python-is-different-from-other-languages/> [consulta: junio de 2024].
- Sánchez, J., & Martínez, C. R. (2013). Estructura de datos [versión Adobe PDF]. [https:// www.escom.ipn.mx/docs/oferta/matDidacticoISC2009/EDts/Libro_EstructuraDatos.pdf](https://www.escom.ipn.mx/docs/oferta/matDidacticoISC2009/EDts/Libro_EstructuraDatos.pdf)
- Schildt, H., & Coward, D. (2023). Java: The complete reference (13a ed.). McGraw Hill.
- Silberschatz, A. (2014). Fundamentos de bases de datos (6a ed.) McGraw Hill.
- Stackshare (2024). "Trending Comparisons (C++ vs. JavaScript vs. PHP)". Stackshare, disponible en: <https://stackshare.io/stackups/cplusplus-vs-javascript-vs-php> [consulta: junio de 2024].



Área 4. Gestión de proyectos de software

- AltexSoft. (2022). "Technical documentation in software development: Types, best practices, and tools". AltexSoft, disponible en: <https://www.altexsoft.com/blog/technical-documentation-in-software-development-types-best-practices-and-tools/> [consulta: junio de 2024].
- Arciniega, F. (2017). Normas y estándares de calidad para el desarrollo de software [versión Adobe PDF]. http://fcaenlinea.unam.mx/anexos/1728/Unidad_2/u2_act2_1.pdf
- Bass, L., Clements, P., & Kazman, R. (2021). Software architecture in practice (4a ed.). Pearson; Addison Wesley.
- Boar, B. (2001). The art of strategic planning for information technology. John Wiley & Son.
- Booch, G., Rumbaugh, J., & Jacobson, I. (2008). UML el proceso unificado de desarrollo de software (F. J. Durán Muñoz, Trad.). Pearson; Addison Wesley.
- Campo, R., Campo, M., & Raya, V. R. (2013). Gestión de proyectos. RA-MA.
- David, F. R. (2017). Conceptos de Administración Estratégica (15a ed.). Pearson.
- Díaz, J., & Robiolo, G. (2014). "Método de estimación de costos de un producto de software web" [ponencia]. 15th Argentine Symposium on Software Engineering, ASSE, (43), pp. 120-134.
- Eeles, P. (2001). "Capturing architectural requirements", IBM Rational Developer Works, disponible en: https://www.researchgate.net/profile/Peter-Eeles-2/publication/329760910_Capturing_Architectural_Requirements/links/5c197761458515a4c7e8bae1/Capturing-Architectural-Requirements.pdf [consulta: junio de 2024].
- Escudero, X. (2015). Calidad del software: Técnicas de prevención, detección y corrección de defectos [versión Adobe PDF]. <https://dokumen.tips/documents/software-gestin-calidad-del-de-la-calidad-y-mtricas.html?page=1>
- Gamboa-Carrascal, J., & Rosado, A. (enero-diciembre 2015). Diseño de un método ágil de desarrollo de software basado en XP, SCRUM, OPENUP y validado con la herramienta de análisis 4-dat. Revista Ingenio, 8(1), pp. 19-31. <https://revistas.ufps.edu.co/index.php/ingenio/article/view/2045/1998>
- García, J. (2018). Estudio comparativo de metodologías, herramientas y wiki de soporte para la gestión de proyectos de desarrollo software [versión

Adobe PDF].

<https://openaccess.uoc.edu/bitstream/10609/73151/6/jgarcianavarroTFG0118memoria.pdf>.

- Garita-González, G., & Lizano-Madriz, F. (enero, 2018). Estimación de costo de software: Una propuesta de aplicación pedagógica de COCOMO. *Uniciencia*, 32(1), pp. 118-133. <https://dx.doi.org/10.15359/ru.32-1.8>
- Maida, E., & Pacienza, J. (2015). Metodologías de desarrollo de software [tesis], Argentina, Universidad Católica Argentina.
- OpenUp (2013). "Metodología Open Up", OpenUp, disponible en: <https://openupeaojmp.blogspot.com/2013/09/metodologia-open-up.html> [consulta: junio de 2024].
- Paredes, D., Martínez, L., Bermúdez, R., & Mendoza, S. (2019). Análisis de la metodología RUP en el desarrollo de software académico mediante la herramienta DJANGO. *RECIMUNDO*, 3(2), 964-979.
- Sánchez, A., Urban, S., & Rodríguez, D. (2012). Ingeniería del Software: Un enfoque desde la Guía SWebok. Alfaomega.
- Schwaber, K., & Sutherland, J. (2020). La guía de Scrum: La guía definitiva de Scrum, las reglas del juego. Scrum Guides [versión Adobe PDF] <https://scrumguides.org/docs/scrumguide/v2020/2020-Scrum-Guide-Spanish-Latin-South-American.pdf>
- Wells, D. (2013). "Extreme Programming: A gentle introduction". Extreme Programing, disponible en: <http://www.extremeprogramming.org/> [consulta: junio de 2024].